



人工智能

陈科海

计算机科学与技术学院

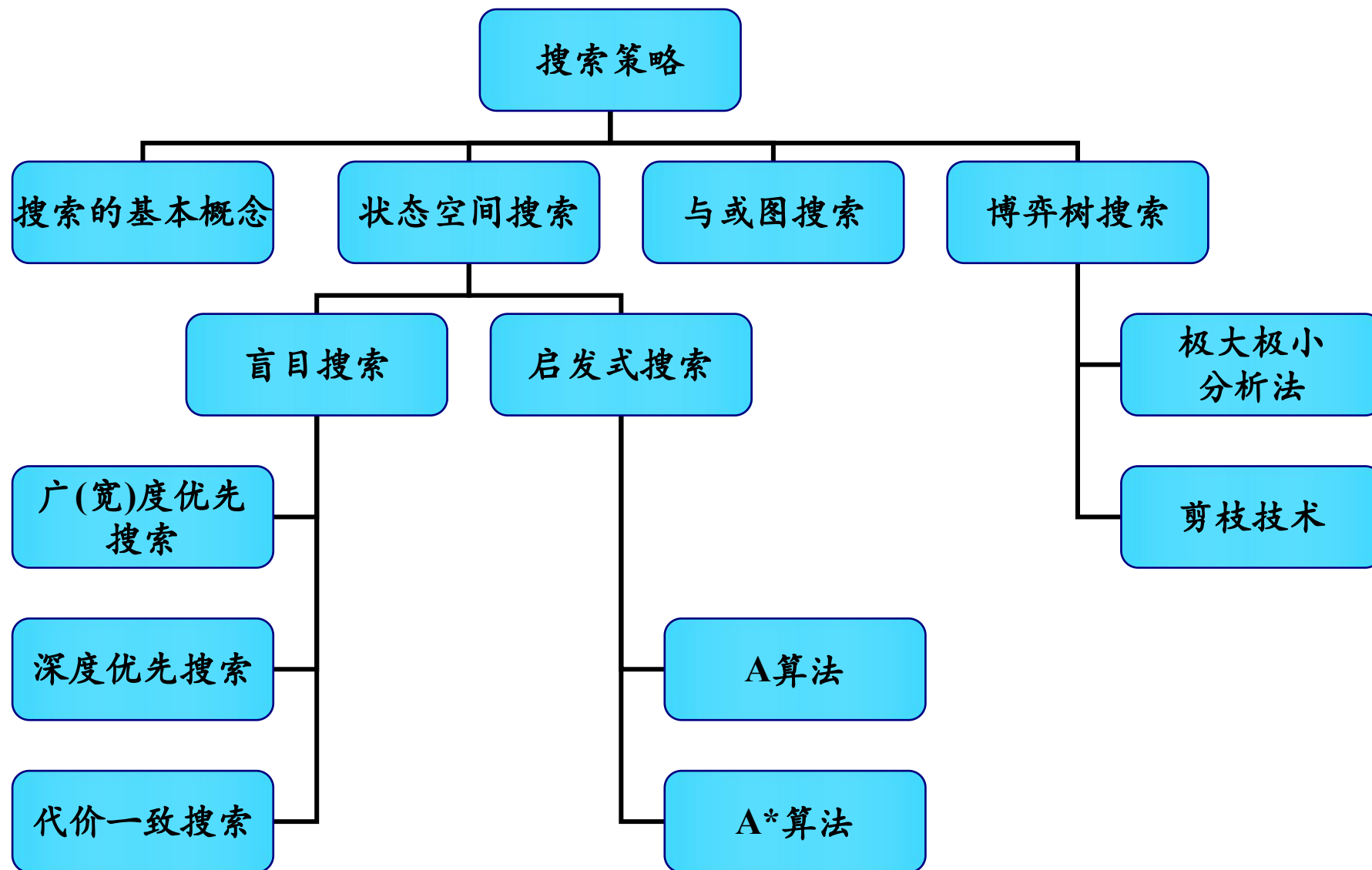
哈尔滨工业大学（深圳）

chenkehai@hit.edu.cn



第4章 搜索策略

本章知识结构



目录

01

4.1 概述

02

4.2 状态空间盲目搜索

03

4.3 状态空间启发式搜索

04

4.4 与/或树搜索

05

4.5 博弈树的启发式搜索

4.1.1 搜索的含义

□ 概念:

依靠经验, 利用已有知识, 根据问题的实际情况, 不断寻找可利用知识, 从而构造一条代价最小的推理路线, 使问题得以解决的过程称为搜索

□ 适用情况:

不良结构或非结构化问题; 难以获得求解所需的全部信息; 更没有现成的算法可供求解使用。结构良好, 理论上算法可依的, 但问题或算法的复杂性较高

□ 搜索的分类

按是否使用启发式信息:

- **盲目搜索:** 按预定的控制策略进行搜索, 在搜索过程中获得的中间信息并不改变控制策略。
- **启发式搜索:** 在搜索中加入了与问题有关的启发性信息, 用于指导搜索朝着最有希望的方向前进, 加速问题的求解过程并找到最优解。

按问题的表示方式:

- **状态空间搜索:** 用状态空间法来求解问题所进行的搜索
- **与或树搜索:** 用问题归约法来求解问题时所进行的搜索

4.1.2 问题的概述

□ 问题可以形式化地定义为三个组成部分

(1) 初始状态 (Initial state) (s_0) ;

(2) 后继函数:

操作函数 (Actions) : $\{a_1, a_2, a_3 \dots\}$

路径耗散函数 (PathCost) : $\text{PathCost} (s_0 \xrightarrow{a_1} s_1 \dots \xrightarrow{a_{i-1}} s_i)$

(3) 目标测试函数 (GoalTest) : $\text{GoalTest}(s) \rightarrow \text{T/F}$

□ 问题的解

初始状态到目标状态的操作序列。 $(s_0 \xrightarrow{a_1} s_1 \dots \xrightarrow{a_{n-1}} s_n)$

4.1.3 状态空间法

□ 状态(State)

表示问题求解过程中每一步问题状况的数据结构，它可形式地表示为：

$$S_k = \{S_{k0}, S_{k1}, \dots\}$$

当对每一个分量都给以确定的值时，就得到了一个具体的状态。

□ 操作(Operator)

称为算符，它是把问题从一种状态变换为另一种状态的手段。操作可以是一个机械步骤，一个运算，一条规则或一个过程。操作可理解为状态集合上的一个函数，它描述了状态之间的关系。

□ 状态空间(State space)

用来描述一个问题的全部状态以及这些状态之间的相互关系。常用一个三元组表示为： (S, F, G)

其中， S 为问题的所有初始状态集合； F 为操作的集合； G 为目标状态的集合。

状态空间也可用一个赋值的有向图来表示，该有向图称为状态空间图。在状态空间图中，节点表示问题的状态，有向边表示操作。

4.1.3 状态空间法

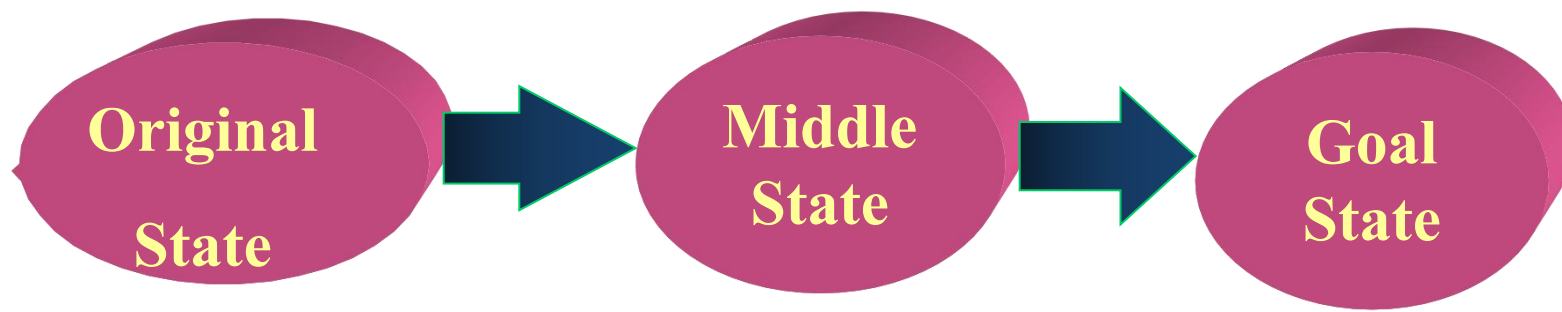
□ 状态空间法求解问题的基本过程:

首先, 为问题选择适当的“**状态**”及“**操作**”的形式化描述方法;

然后, 从某个初始状态出发, 每次使用一个“操作”, 递增地建立起操作序列, 直到达到目标状态为止;

最后, 由初始状态到目标状态所使用的操作序列就是该问题的一个解。

问题: ①最优解问题; ②搜索策略问题。



4.1.3 状态空间法

例：Hanoi塔



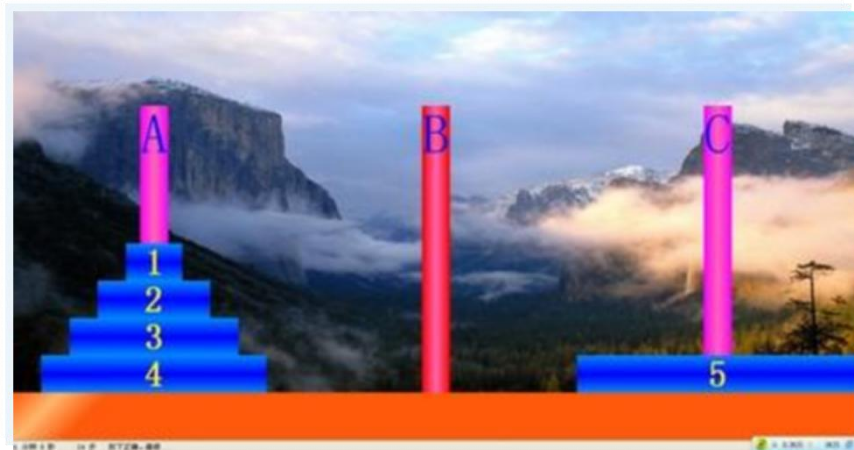
在梵城（Hana）地下有一个僧侣的秘密组织，他们有3个大型的塔柱，左边的塔柱上由方到小套着64个金盘。僧侣们的工作是要把这64个金盘从左边塔柱转移到右边塔柱上去。但转移过程有规定的：

- 1、每次只能搬动一只盘子，盘子只能在3个塔柱上安放，不允许放在地上；
- 2、在每个塔柱上，只允许把小盘子叠在大盘上，反之不允许。

据传说，僧侣们完成这个任务时，世界的末日就来临了。

4.1.3 状态空间法

例：Hanoi塔



19世纪，法国的一位数学家Édouard Lucas (1842–1891) 对该课题进行过研究，他指示，要完成这个任务，僧侣们搬动金盘的总次数：

18446744073709551615 (20位) 假设僧侣们个个身强力壮，每天24小时不知头疲倦地工作，而且一秒钟移动一个金盘，那么，完成这个任务也得花**5800亿**年。

例：二阶梵塔问题

设有三根钢针，它们的编号分别是1号、2号和3号。在初始情况下，1号钢针上穿有A、B两个金片，A比B小，A位于B的上面。要求把这两个金片全部移到另一根钢针上，而且规定每次只能移动一个金片，任何时刻都不能使大的位于小的上面。

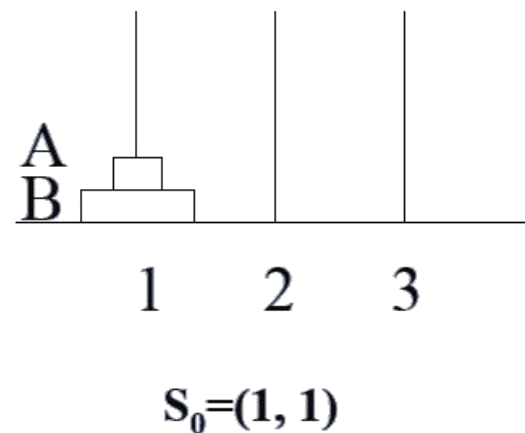
4.1.3 状态空间法

解： 设用 $S_k=(S_{k0}, S_{k1})$ 表示问题的状态，其中， S_{k0} 表示金片A所在的钢针号， S_{k1} 表示金片B所在的钢针号。

全部可能的问题状态共有以下9种：

$$S_0=(1, 1) \ S_1=(1, 2) \ S_2=(1, 3) \ S_3=(2, 1) \ S_4=(2, 2)$$

$$S_5=(2, 3) \ S_6=(3, 1) \ S_7=(3, 2) \ S_8=(3, 3)$$



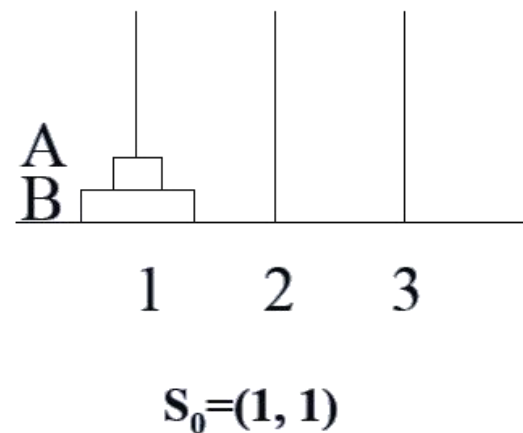
4.1.3 状态空间法

解： 设用 $S_k=(S_{k0}, S_{k1})$ 表示问题的状态，其中， S_{k0} 表示金片A所在的钢针号， S_{k1} 表示金片B所在的钢针号。

全部可能的问题状态共有以下9种：

$$S_0=(1, 1) \ S_1=(1, 2) \ S_2=(1, 3) \ S_3=(2, 1) \ S_4=(2, 2)$$

$$S_5=(2, 3) \ S_6=(3, 1) \ S_7=(3, 2) \ S_8=(3, 3)$$



初始状态集合

$$S=\{S_0\}$$

目标状态集合

$$G=\{S_4, S_8\}$$

初始状态 S_0 和目标状态 S_4 、 S_8 如下图



4.1.3 状态空间法

操作

A_{ij} 表示把金片**A**从第*i*号钢针移到*j*号钢针上;

B_{ij} 表示把金片**B**从第*i*号钢针移到到第*j*号钢针上。

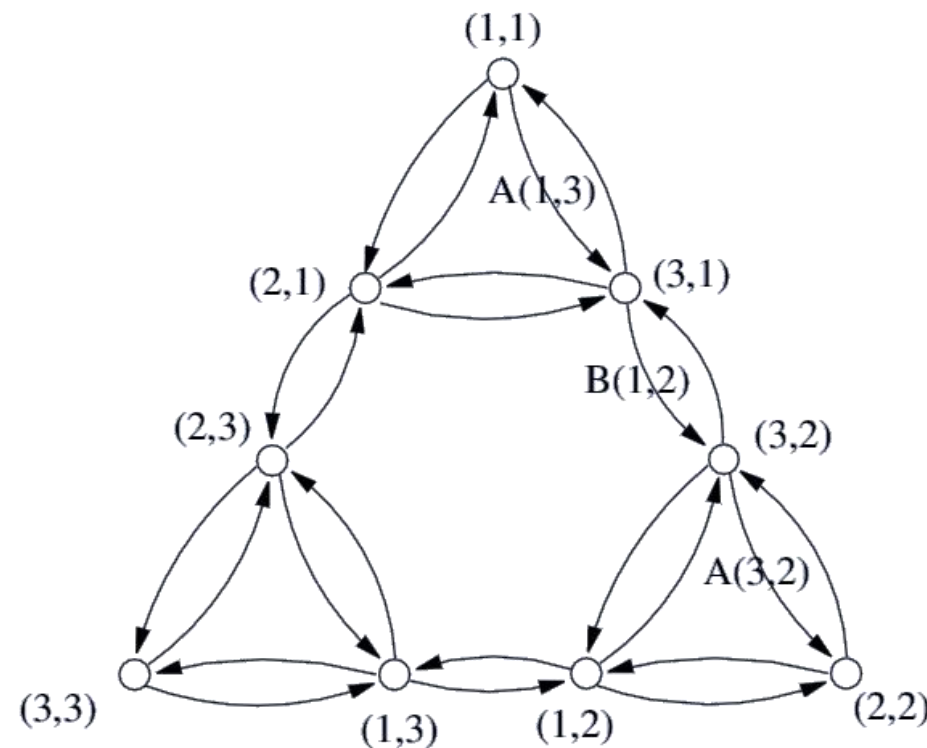
共有**12**种操作，它们分别是：

$A_{12} A_{13} A_{21} A_{23} A_{31} A_{32}$

$B_{12} B_{13} B_{21} B_{23} B_{31} B_{32}$

根据上述**9**种可能的状态和**12**种操作，可构成二阶梵塔问题的状态空间图，如右图所示。

从初始节点**(1, 1)**到目标节点**(2, 2)**及**(3, 3)**的任何一条路径都是问题的一个解。其中，最短的路径长度是**3**，它由**3**个操作组成。例如，从**(1, 1)**开始，通过使用操作 **A_{13}** 、 **B_{12}** 及 **A_{32}** ，可到达**(2, 2)**。



4.1.3 状态空间法

例：修道士和野人问题

在河的左岸有三个修道士、三个野人和一条船，修道士们想用这条船将所有的人运过河去，但受到以下条件的限制：

- 1) 修道士和野人都会划船，但船一次最多只能运两个人；
- 2) 在任何岸边野人数目都不得超过修道士，否则修道士就会被野人吃掉。

4.1.3 状态空间法

例：修道士和野人问题

1、问题的状态

可以用一个三元数组来描述：

$$S = (m, c, b)$$

m: 左岸的修道士数

c: 左岸的野人数

b: 左岸的船数

右岸的状态不必标出，因为：

$$\text{右岸的修道士数 } m' = 3 - m$$

$$\text{右岸的野人数 } c' = 3 - c$$

$$\text{右岸的船数 } b' = 1 - b$$

4.1.3 状态空间法

例：修道士和野人问题

由合理的状态转换规则无法推出

状态	m, c, b	状态	m, c, b	状态	m, c, b	状态	m, c, b
S_0	3 3 1	S_8	1 3 1	S_{16}	3 3 0	S_{24}	1 3 0
S_1	3 2 1	S_9	1 2 1	S_{17}	3 2 0	S_{25}	1 2 0
S_2	3 1 1	S_{10}	1 1 1	S_{18}	3 1 0	S_{26}	1 1 0
S_3	3 0 1	S_{11}	1 0 1	S_{19}	3 0 0	S_{27}	1 0 0
S_4	2 3 1	S_{12}	0 3 1	S_{20}	2 3 0	S_{28}	0 3 0
S_5	2 2 1	S_{13}	0 2 1	S_{21}	2 2 0	S_{29}	0 2 0
S_6	2 1 1	S_{14}	0 1 1	S_{22}	2 1 0	S_{30}	0 1 0
S_7	2 0 1	S_{15}	0 0 1	S_{23}	2 0 0	S_{31}	0 0 0

状态不合法

4.1.3 状态空间法

例：修道士和野人问题

2、操作集 $F = \{L_{01}, L_{10}, L_{11}, L_{02}, L_{20}, R_{01}, R_{10}, R_{11}, R_{02}, R_{20}\}$

操作符	条 件	动 作
L_{01}	$b=1, m=0 \text{ 或 } 3, c \geq 1$	$b=0, c=c-1$
L_{10}	$b=1, (m=3, c=2) \text{ 或 } (m=1, c=1)$	$b=0, m=m-1$
L_{11}	$b=1, m=c, c \geq 1$	$b=0, m=m-1, c=c-1$
L_{02}	$b=1, m=0 \text{ 或 } 3, c \geq 2$	$b=0, c=c-2$
L_{20}	$b=1, (m=3, c=1) \text{ 或 } (m=2, c=2)$	$b=0, m=m-2$
R_{01}	$b=0, m=0 \text{ 或 } 3, c \leq 2$	$b=1, c=c+1$
R_{10}	$b=0, (m=0, c=1) \text{ 或 } (m=2, c=2)$	$b=1, m=m+1$
R_{11}	$b=0, m=c, c \leq 2$	$b=1, m=m+1, c=c+1$
R_{02}	$b=0, m=0 \text{ 或 } 3, c \leq 2$	$b=1, c=c+2$
R_{20}	$b=0, (m=0, c=2) \text{ 或 } (m=1, c=1)$	$b=1, m=m+2$

4.1.3 状态空间法

例：修道士和野人问题

3、状态空间

给出状态和操作的描述之后，该问题的状态空间是：

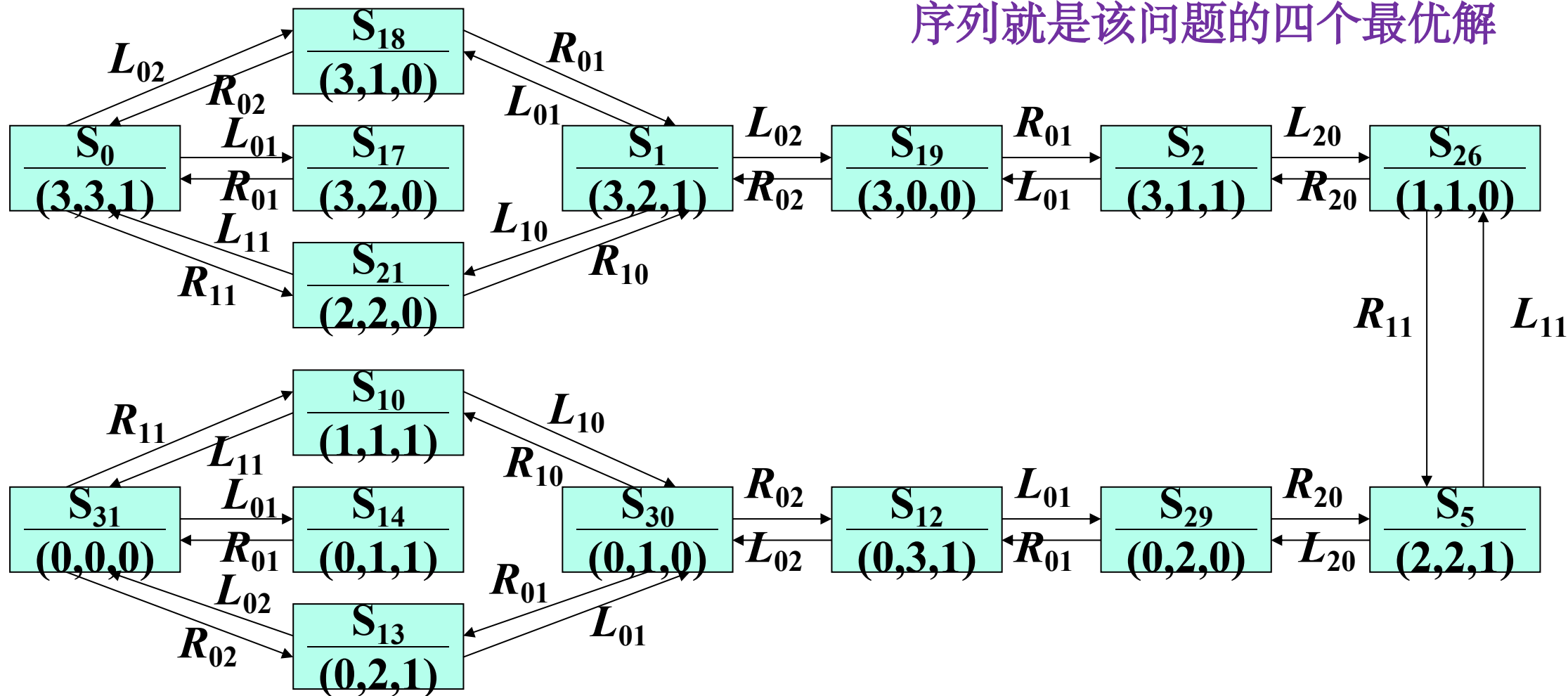
$\{\{S_0\}, \{L_{01}, L_{10}, L_{11}, L_{02}, L_{20}, R_{01}, R_{10}, R_{11}, R_{02}, R_{20}\}, \{S_{31}\}\}$ 。

4.1.3 状态空间法

例：修道士和野人问题

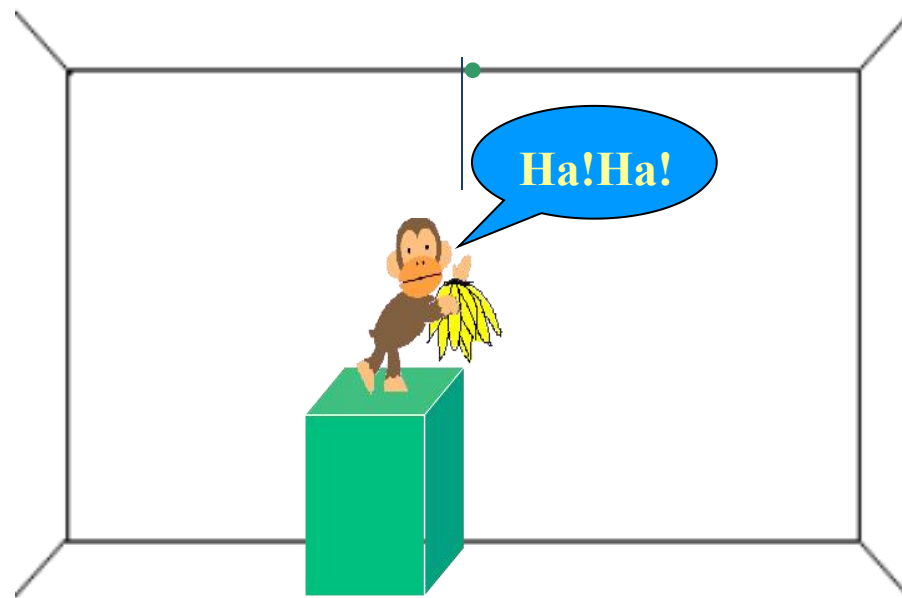
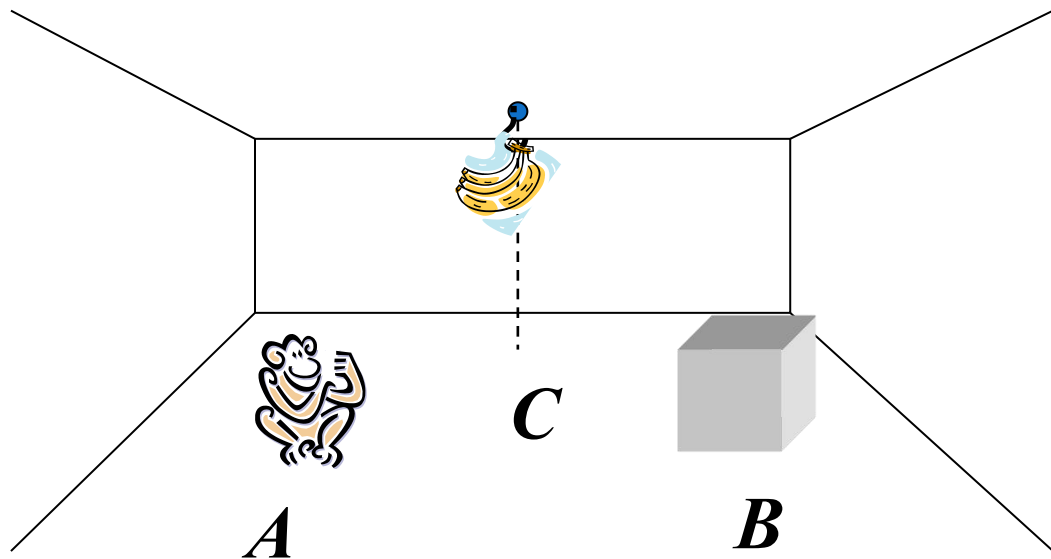
4、状态空间图：

四条 S_0 到 S_{31} 长度相等的最短路径，对应的操作序列就是该问题的四个最优解



4.1.3 状态空间法

例：猴子摘香蕉问题。在讨论知识表示时，我们曾提到过这一问题，现在用状态空间法来解决这一问题。



请画出猴子摘香蕉问题的状态空间图

问题的状态可用4元组 (m, b, ON, H) 表示。

其中， m 表示猴子的位置； b 表示箱子的位置； ON 表示猴子是否在箱子上，当猴子在箱子上时， y 取1，否则 y 取0； H 表示猴子是否拿到香蕉，当拿到香蕉时 z 取1，否则 z 取0。

可能的状态

$S_0 : (a, b, 0, 0)$ 初始状态

$S_1 : (b, b, 0, 0)$

$S_2 : (c, c, 0, 0)$

$S_3 : (c, c, 1, 0)$

$S_4 : (c, c, 1, 1)$ 目标状态

允许的操作为

Goto(u): 猴子走到位置 u ，即

$(w, x, 0, 0) \rightarrow (u, x, 0, 0)$

Pushbox(v): 猴子推着箱子到水平位置 v ，即

$(x, x, 0, 0) \rightarrow (v, v, 0, 0)$

Climbbox: 猴子爬上箱子，即

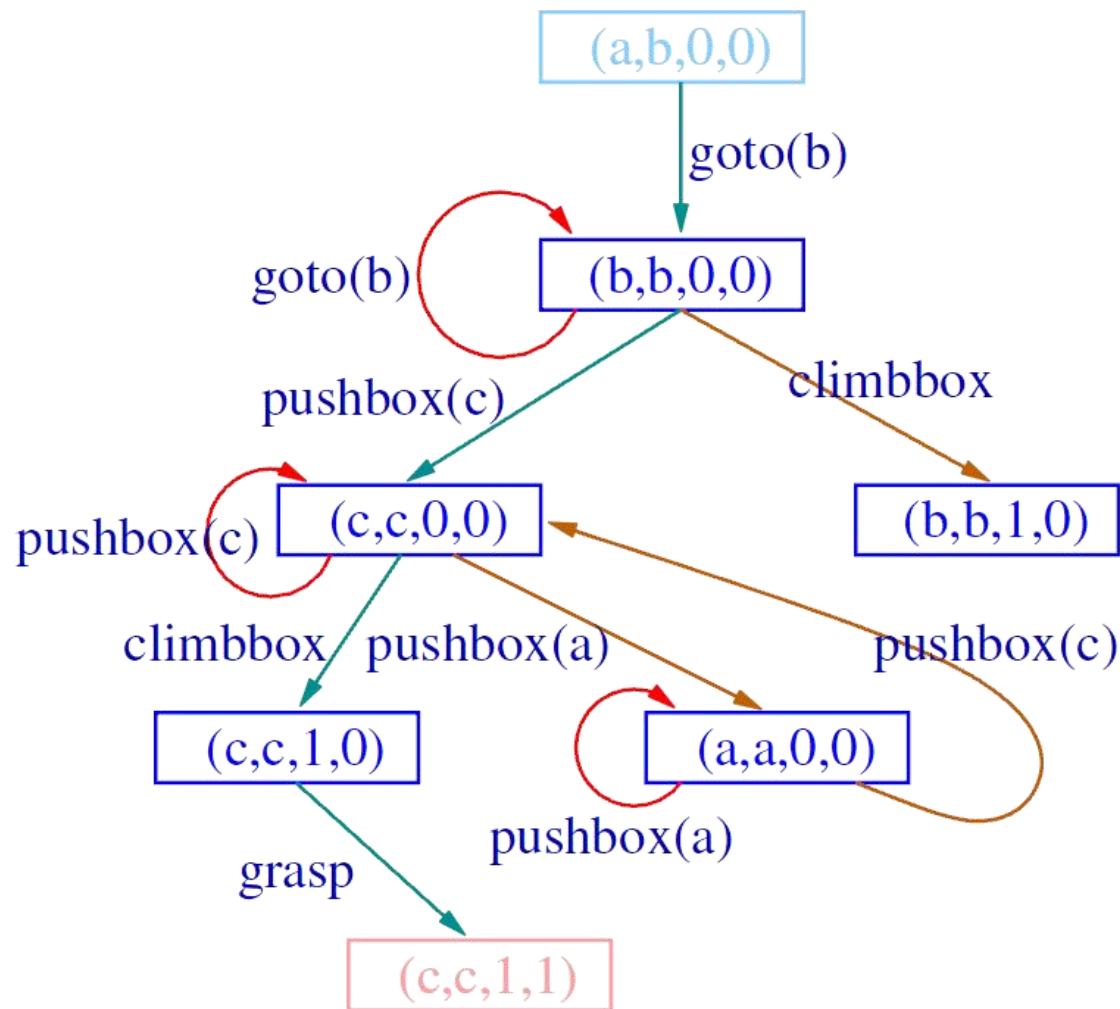
$(x, x, 0, 0) \rightarrow (x, x, 1, 0)$

Grasp: 猴子拿到香蕉，即

$(c, c, 1, 0) \rightarrow (c, c, 1, 1)$

4.1.3 状态空间法

猴子摘香蕉问题的状态空间图



可见，由初始状态 $(a, b, 0, 0)$ 到目标状态 $(c, c, 1, 1)$ 的操作序列为：

$\{\text{Goto}(b), \text{Pushbox}(c), \text{Climbbox}, \text{Grasp}\}$

4.1.4 问题归约法

□ 问题的与/或树表示

➤ 基本思想

当一问题较复杂时，可通过**分解**或**变换**，将其转化为一系列较简单的子问题，然后通过对这些子问题的求解来实现对原问题的求解。

➤ 分解

如果一个问题 P 可以归约为一组子问题 $P_1, P_2, \dots, P_n$ ，并且只有当所有子问题 P_i 都有解时原问题 P 才有解，任何一个子问题 P_i 无解都会导致原问题 P 无解，则称此种归约为问题的分解。

即分解所得到的子问题的“**与**”与原问题 P 等价。

➤ 等价变换

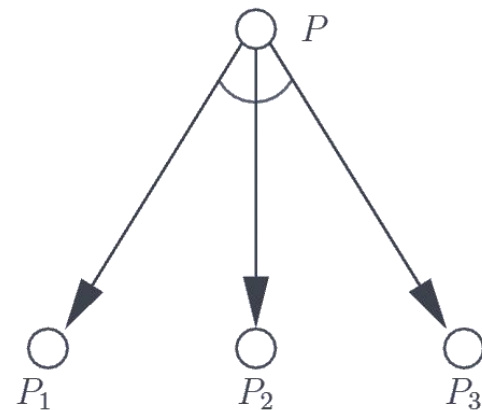
如果一个问题 P 可以归约为一组子问题 $P_1, P_2, \dots, P_n$ ，并且子问题 P_i 中只要有一个有解则原问题 P 就有解，只有当所有子问题 P_i 都无解时原问题 P 才无解，称此种归约为问题的等价变换，简称变换。

即变换所得到的子问题的“**或**”与原问题 P 等价。

4.1.4.1 问题的与/或树表示

1. 问题的分解和“与树”：

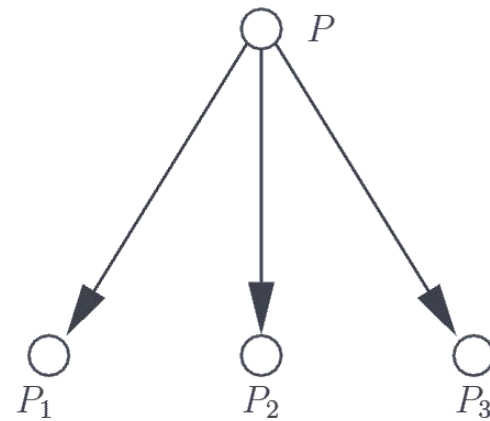
把一个复杂问题分解为若干个子问题时，可以用一个“与树”来表示这种分解。



(a) 与树

2. 问题的等价变换和“或树”：

把一个复杂的问题等价变换为若干个与之等价的新问题时，可用一个“或树”来表示这种变换。

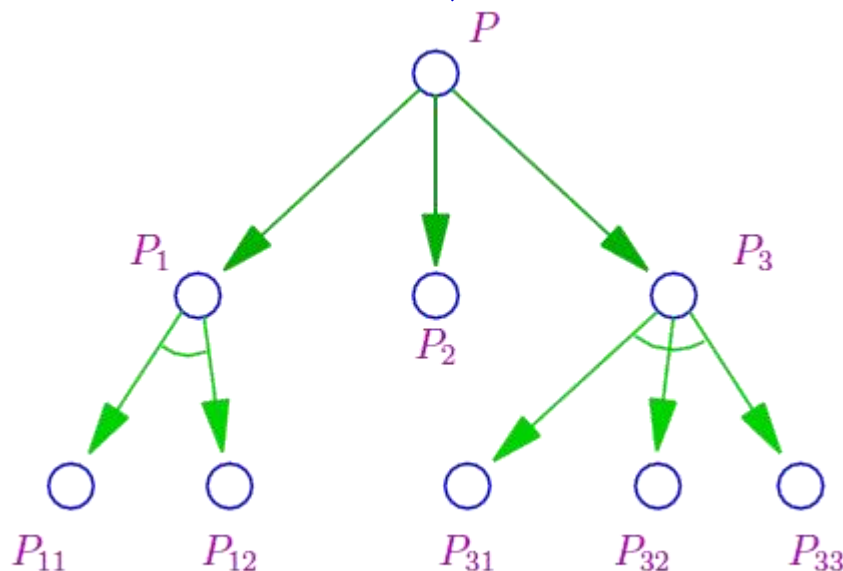


(b) 或树

4.1.4.1 问题的与/或树表示

3. 与/或树:

如果一个问题既需要通过分解, 又需要通过变换才能得到其本原问题, 则其求解过程可用一个“与/或树”表示。所谓本原问题: 指那种不能 (或不需要) 再进行分解或变换, 且可以直接解答的子问题。



问题归约法

与或树

原始问题



起始节点

中间问题



中间节点

本原问题



终止节点

4. 端节点与终止节点:

端节点: 没有子节点的节点。终止节点: 本原问题所对应的节点。终止节点一定是端节点, 但端节点却不一定是终止节点。

4.1.4.1 问题的与/或树表示

5. 可解节点与不可解节点

在与/或树中，满足以下三个条件之一的节点为**可解节点**：

①任何终止节点都是可解节点。

②对“或”节点，当其子节点中至少有一个为可解节点时，则该或节点就是可解节点。

③对“与”节点，只有当其子节点全部为可解节点时，该与节点才是可解节点。

同样，可用类似的方法定义**不可解节点**：

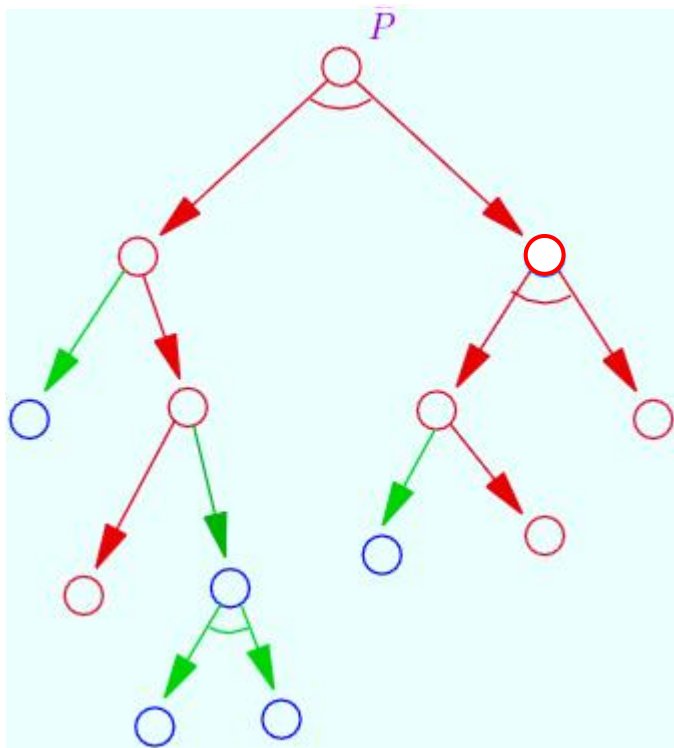
①不为终止节点的端节点是不可解节点。

②对“或”节点，若其全部子节点都为不可解节点，则该或节点是不可解节点。

③对“与”节点，只要其子节点中有一个为不可解节点，则该与节点是不可解节点。

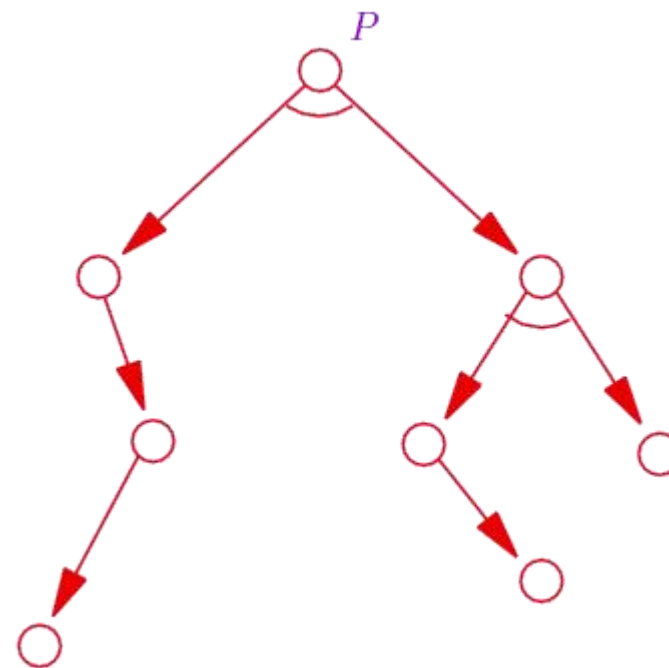
4.1.4.1 问题的与/或树表示

6. **解树**：一个由可解节点构成，并且由这些可解节点可以推出初始节点(对应原始问题)为可解节点(对应原始问题)的子树。问题归约求解过程实际上就是生成解树，即证明原始节点是可解节点的过程。



可解标记过程

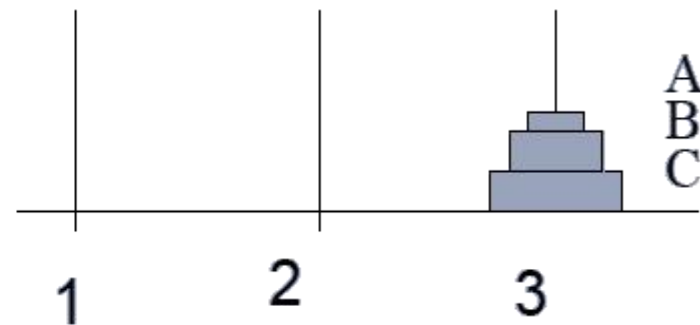
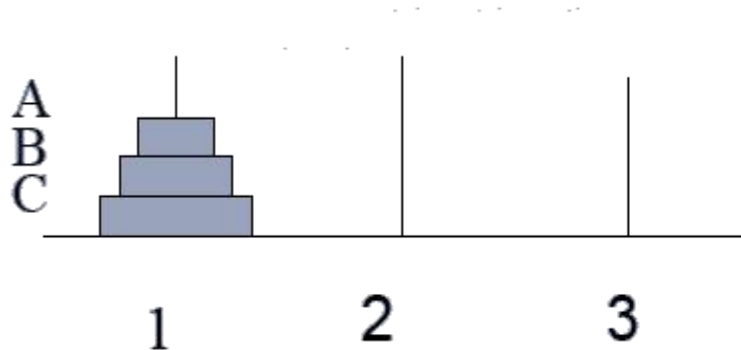
○ 可解节点
○ 端节点



解树

4.1.4.1 问题的与/或树表示

例 三阶梵塔问题。要求把1号钢针上的3个金片全部移到3号钢针上，如下图所示。



解： 这个问题也可用状态空间法来解。

为了能够解决这一问题，首先需要定义该问题的形式化表示方法。设用三元组
(i, j, k)

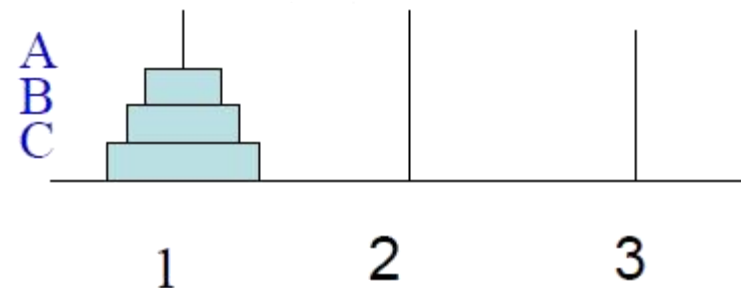
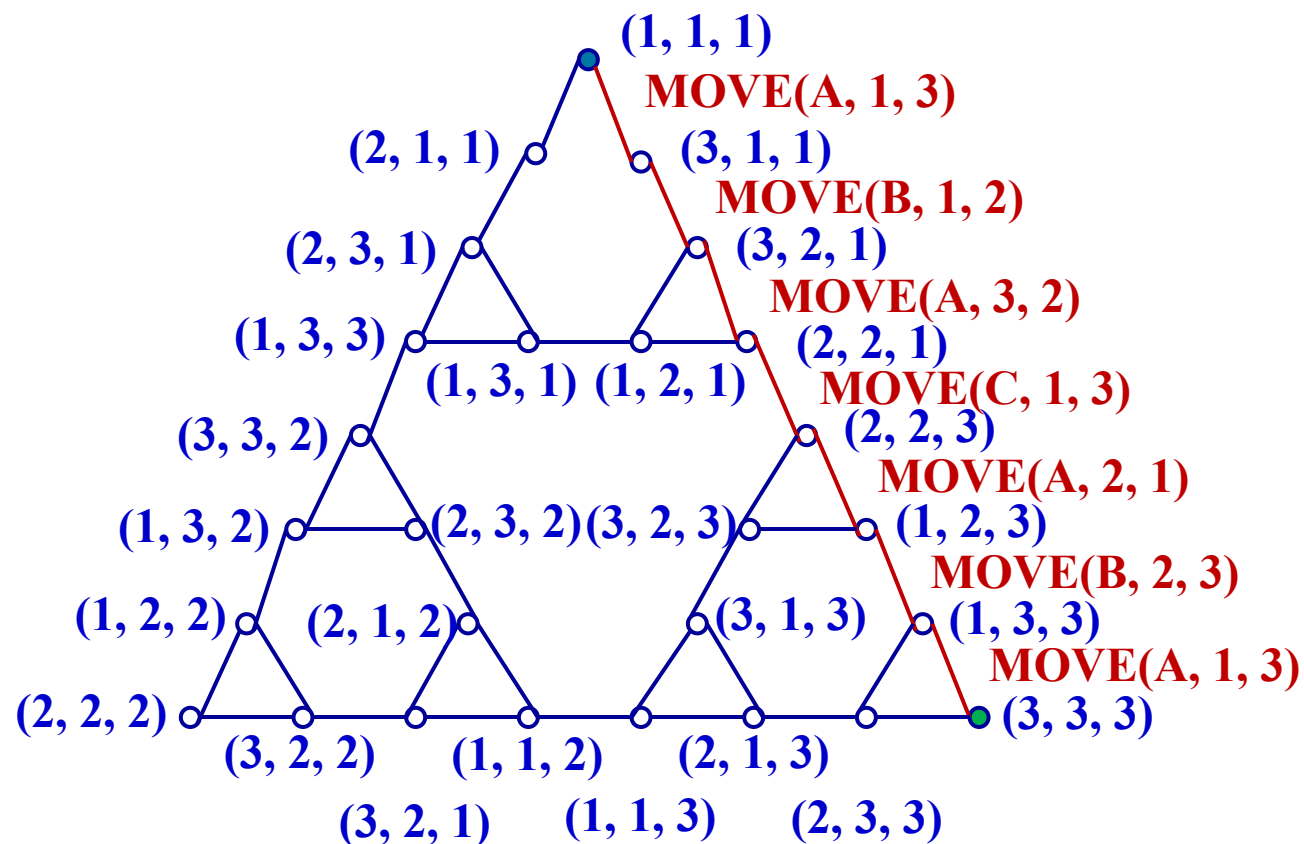
表示问题在任一时刻的状态，用 “ $\rightarrow$ ” 表示状态的转换。上述三元组中

i 代表金片**A**所在的钢针号

j 代表金片**B**所在的钢针号

k 代表金片**C**所在的钢针号

状态空间法



问题归约法

解：其中(i, j, k)表示C在柱i, B在柱j, A在柱k上.

利用归约方法，问题可分解为以下三个子问题：

(1) 把金片A及B移到2号钢针上的双金片移动问题。即

$(1, 1, 1) \rightarrow (1, 2, 2)$

(2) 把金片C移到3号钢针上的单金片移动问题。即

$(1, 2, 2) \rightarrow (3, 2, 2)$

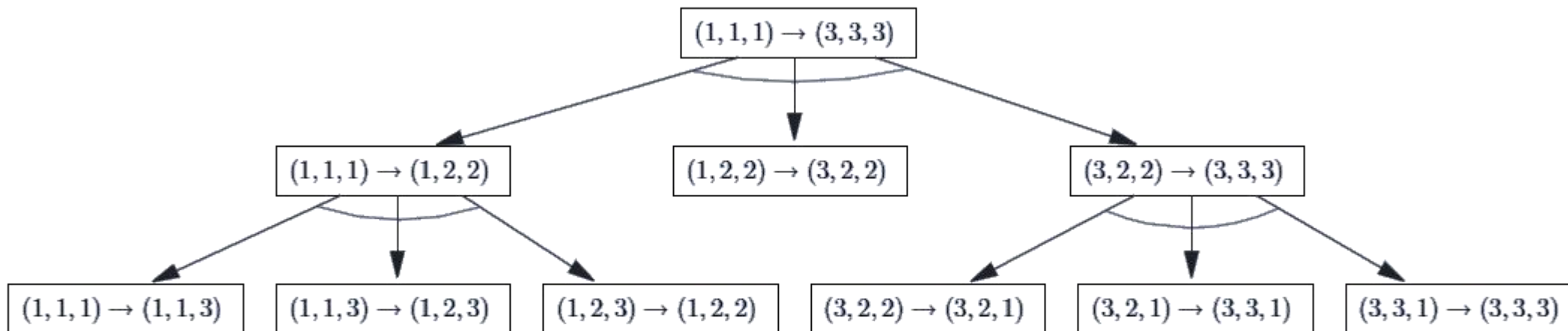
(3) 把金片A及B移到3号钢针上的双金片移动问题。即

$(3, 2, 2) \rightarrow (3, 3, 3)$

其中，子问题 (1) 和 (3) 都是二阶梵塔问题，还可以再分解，(2) 则是本原问题。



三阶梵塔问题的解答



(b) 三阶梵塔问题的解答

在该与/或树中，有7个终止节点，它们分别对应着7个本原问题。如果把这些本原问题从左至右排列起来，即得到了原始问题的解：

$(1, 1, 1) \rightarrow (1, 1, 3)$ $(1, 1, 3) \rightarrow (1, 2, 3)$ $(1, 2, 3) \rightarrow (1, 2, 2)$ $(1, 2, 2) \rightarrow (3, 2, 2)$
 $(3, 2, 2) \rightarrow (3, 2, 1)$ $(3, 2, 1) \rightarrow (3, 3, 1)$ $(3, 3, 1) \rightarrow (3, 3, 3)$

思考：不同方法求解四阶梵塔问题。

目录

01

4.1 概述

02

4.2 状态空间盲目搜索

03

4.3 状态空间启发式搜索

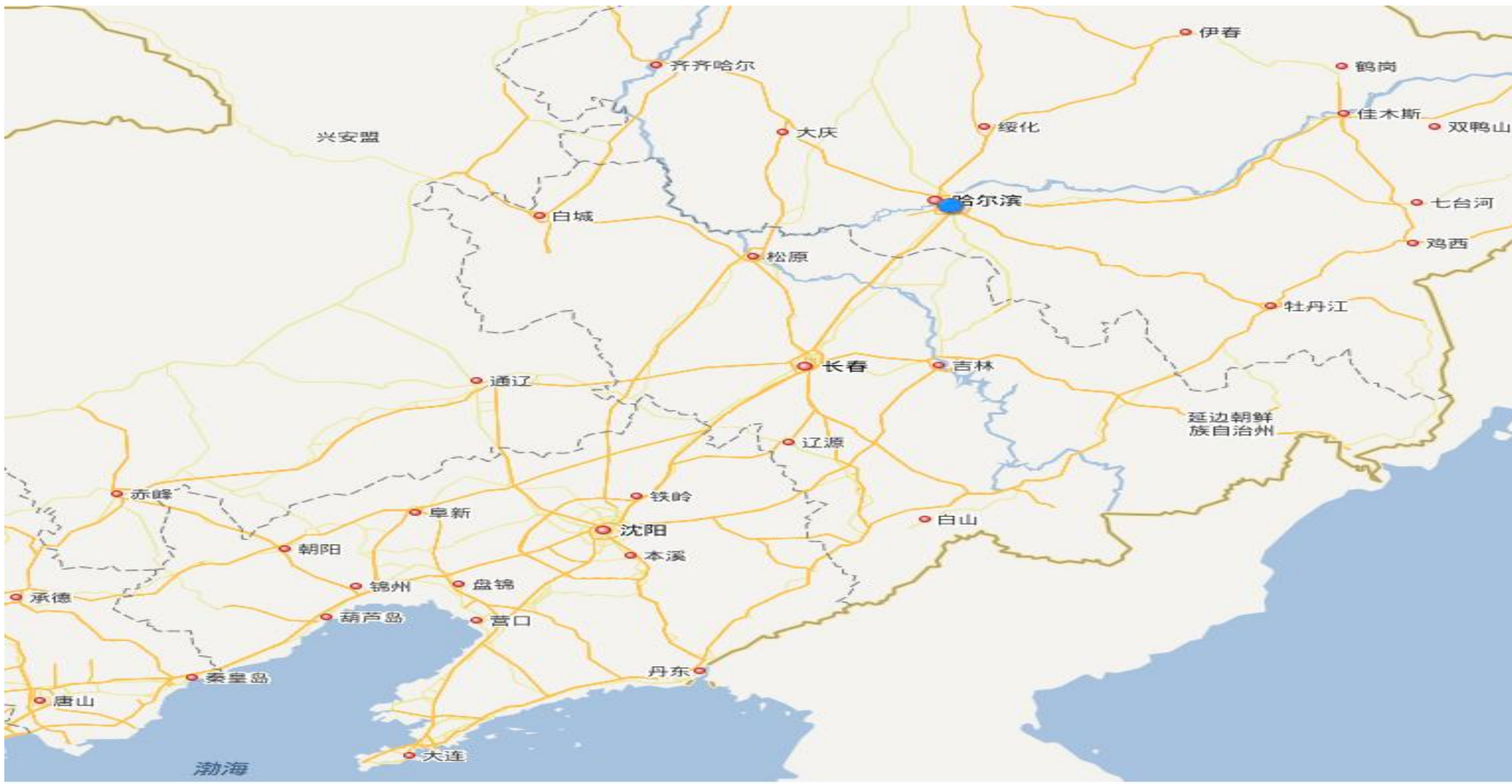
04

4.4 与/或树搜索

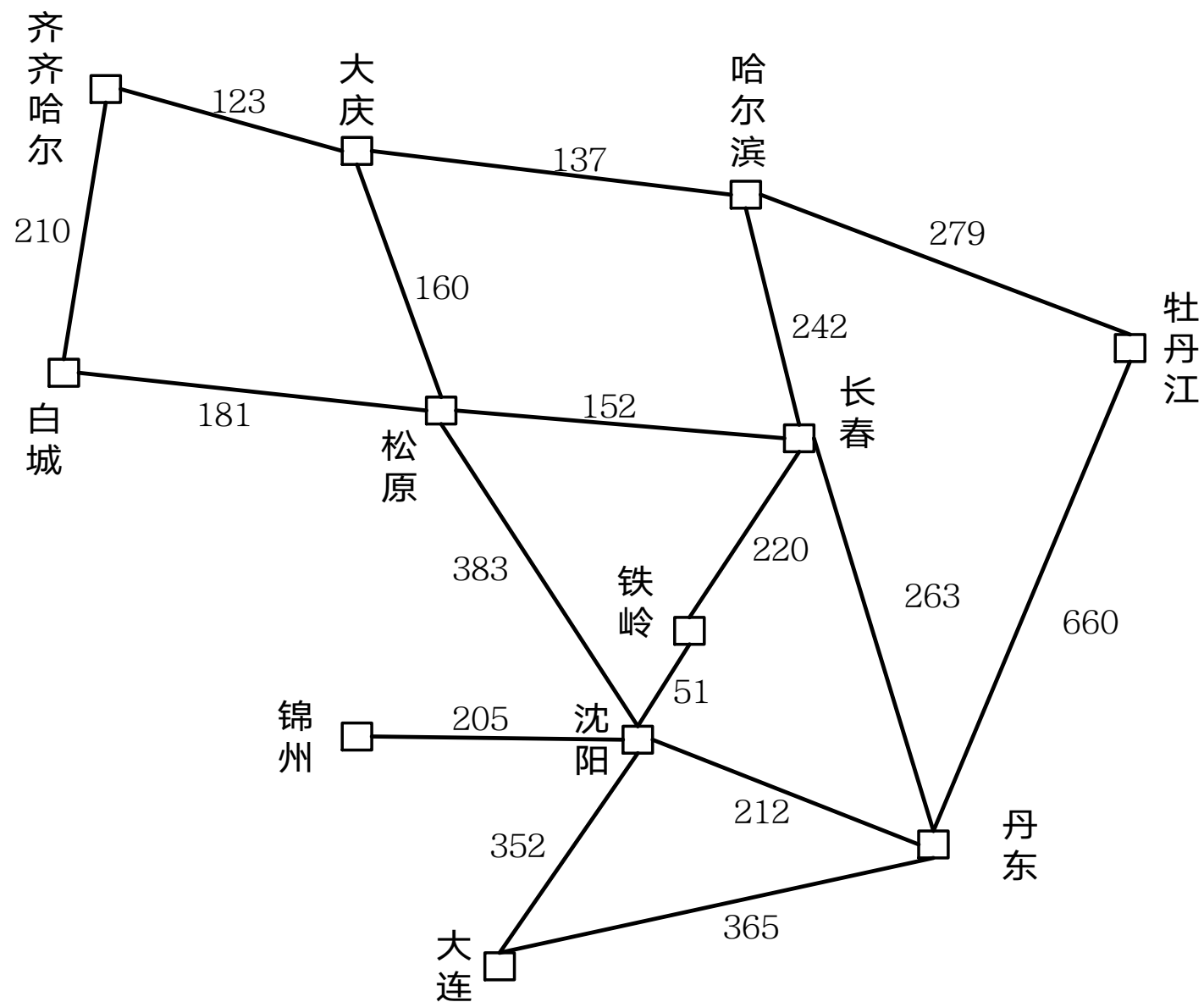
05

4.5 博弈树的启发式搜索

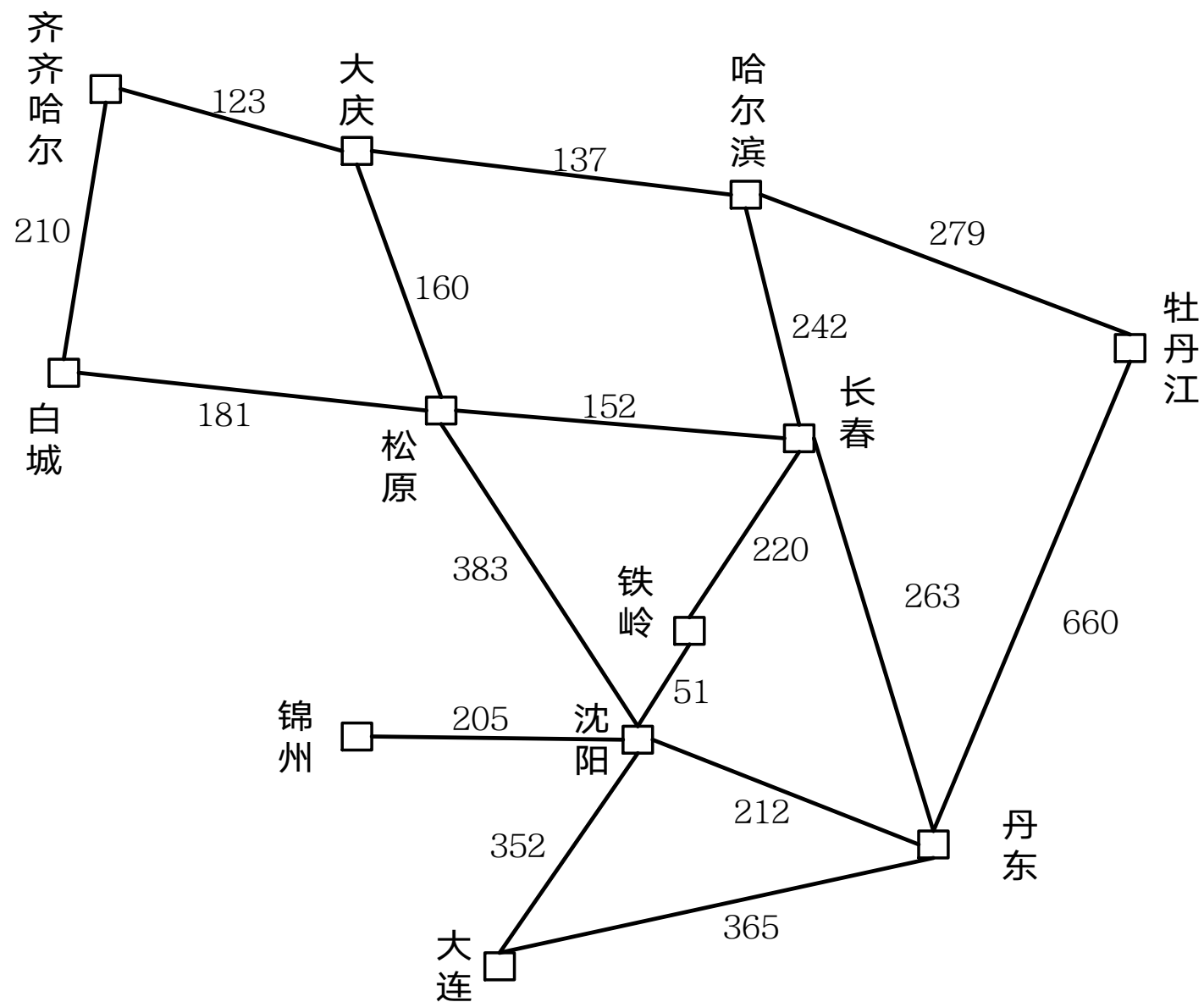
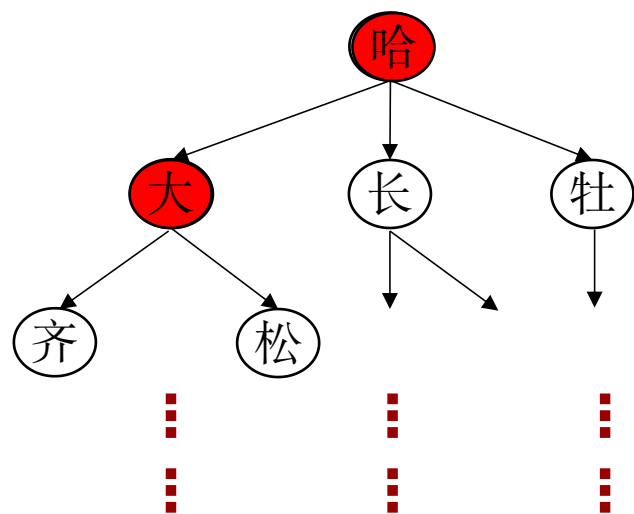
4.2.1 路径搜索问题



4.2.1 路径搜索问题



4.2.1 路径搜索问题



4.2.2 状态空间的盲目搜索

□ 状态空间搜索的基本思想

先把问题的初始状态作为当前扩展节点对其进行扩展，生成一组子节点，然后检查问题的目标状态是否出现在这些子节点中。若出现，则搜索成功，找到了问题的解；若没出现，则再按照某种搜索策略从已生成的子节点中选择一个节点作为当前扩展节点。

重复上述过程，直到目标状态出现在子节点中或者没有可供操作的节点为止。所谓对一个节点进行“**扩展**”是指对该节点用某个可用操作进行作用，生成该节点的一组子节点。

□ 算法的数据结构和符号约定

Open表：用于存放刚生成的节点,未扩展的节点，Open表称为未扩展的节点表。

Closed表：用于存放已经扩展或将要扩展的节点，Closed称为已扩展的节点表。

S₀：用表示问题的初始状态

S_g：用表示问题的目标状态

4.2.2 一般图搜索过程

□ 算法流程

- 1) 把初始节点 S_0 放入Open表，并建立目前仅包 S_0 的图G，建立一个Closed表，置为空；
- 2) 检查Open表是否为空表，若为空，则问题无解，失败退出
- 3) 把Open表的第一个节点取出放入Closed表，并记该节点为n
- 4) 考察节点n是否为目标节点，若是则得到问题的解成功退出。
- 5) 扩展节点n，生成一组子节点。把这些子节点中不是其父节点的那部分子节点计入集合M，并把这些子节点作为节点n的子节点加G中。

4.2.2 一般图搜索过程

(6) 针对**M**中子节点的不同情况，分别作如下处理：

- ① 对那些没有在**G**中出现过的**M**成员设置一个指向其父节点（即节点**n**）的指针，并将他它放入**Open**表；
- ② 对那些原来已经在**G**中出现过，但没有被扩展过的**M**成员，确定是否需要修改它指向父节点的指针；
- ③ 对那些原来已经在**G**中出现过，并已经被扩展过的**M**成员，确定是否需要修改其后继节点指向父节点的指针。

(7) 按某种策略对**Open**表中的节点进行排序

(8) 转第 (2) 步

4.2.2 一般图搜索过程

说明:

(1) 上述过程是状态空间的一般图搜索算法，它具有通用性，后面所要讨论的各种状态空间搜索策略都是上述过程的特例。各种搜索策略的主要区别在于对Open表中节点的排序顺序的不同。

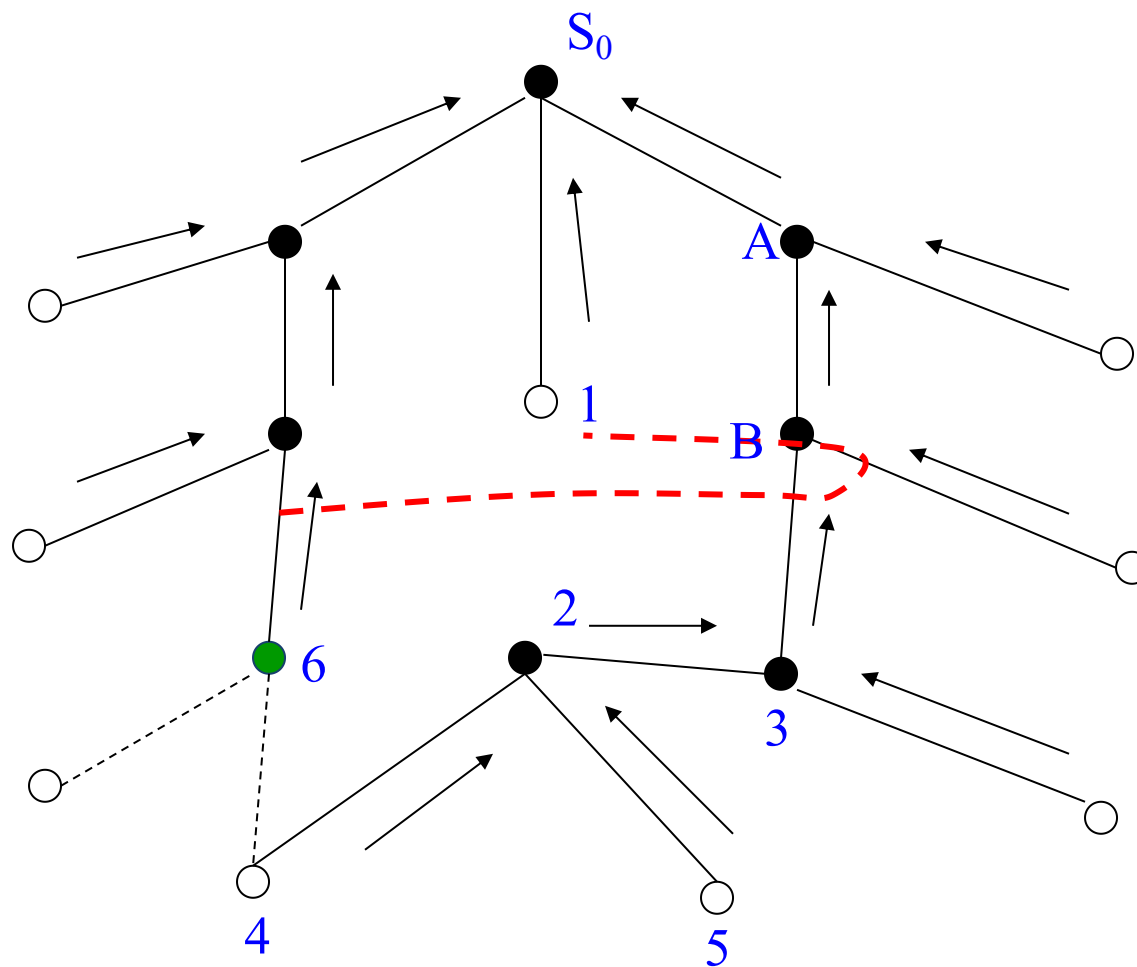
(2) 在第(6)步针对M中子节点的不同情况进行处理时，

如果是第②种情况时，那么，这个M中子节点究竟应该作为哪个节点的后继节点呢？

一般是由原始节点到该节点路径上所付出的代价来决定的，哪条路径付出的代价小，相应的节点就作为它的父节点。所谓由原始节点到该节点路径的代价是指这条路径上所有有向边代价之和。

如果发生③情况，除了需要确定该子节点指向父节点的指针外，还需要确定其后继节点指向父节点的指针。其依据也是由原始节点到该节点路径上的代价。

修改返回指针

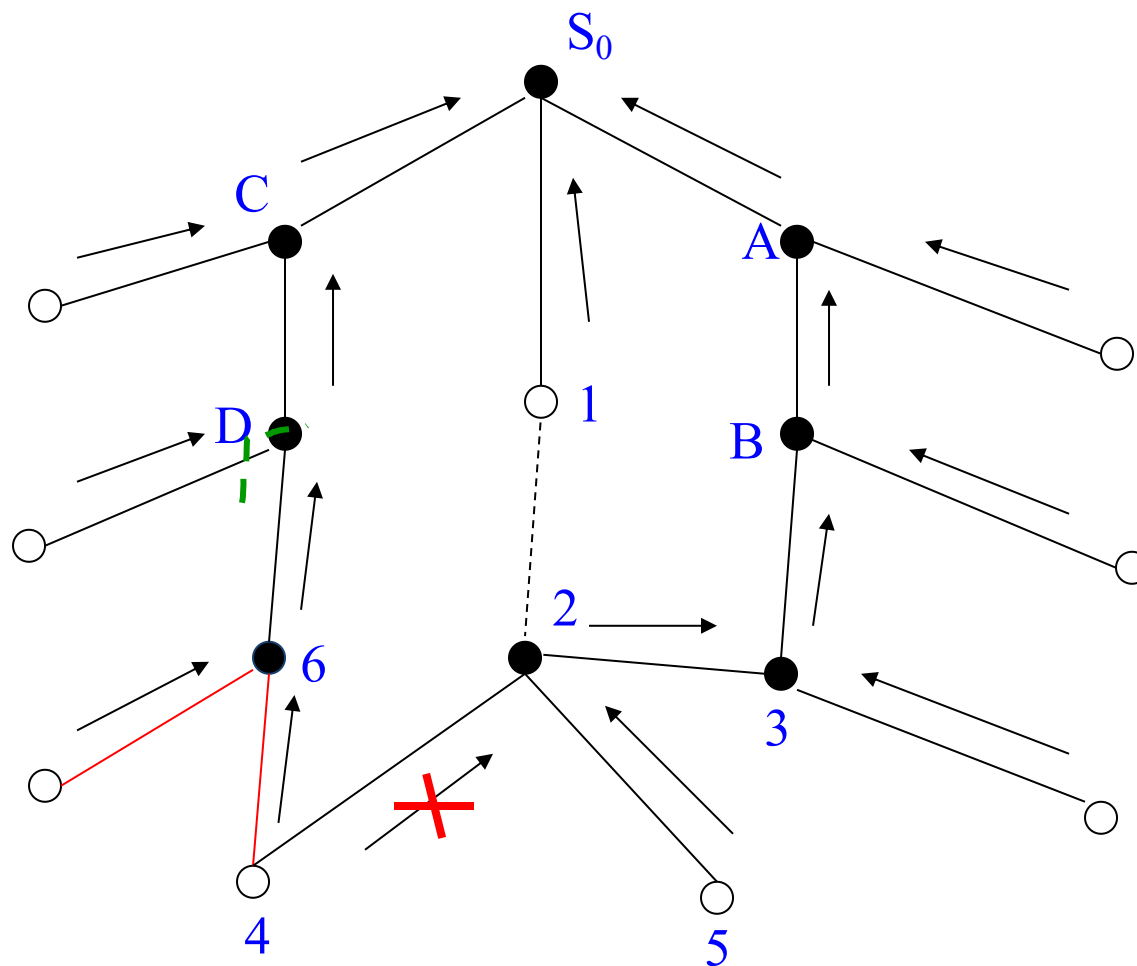


指针返回路径: 4 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ B $\rightarrow$ A $\rightarrow$ S_0

修改返回指针

② 对那些原来已经在G中出现过，但没有被扩展过的M成员，确定是否需要修改它指向父节点的指针。

③ 对那些原来已经在G中出现过，并已经被扩展过的M成员，确定是否需要修改其后继节点指向父节点的指针。



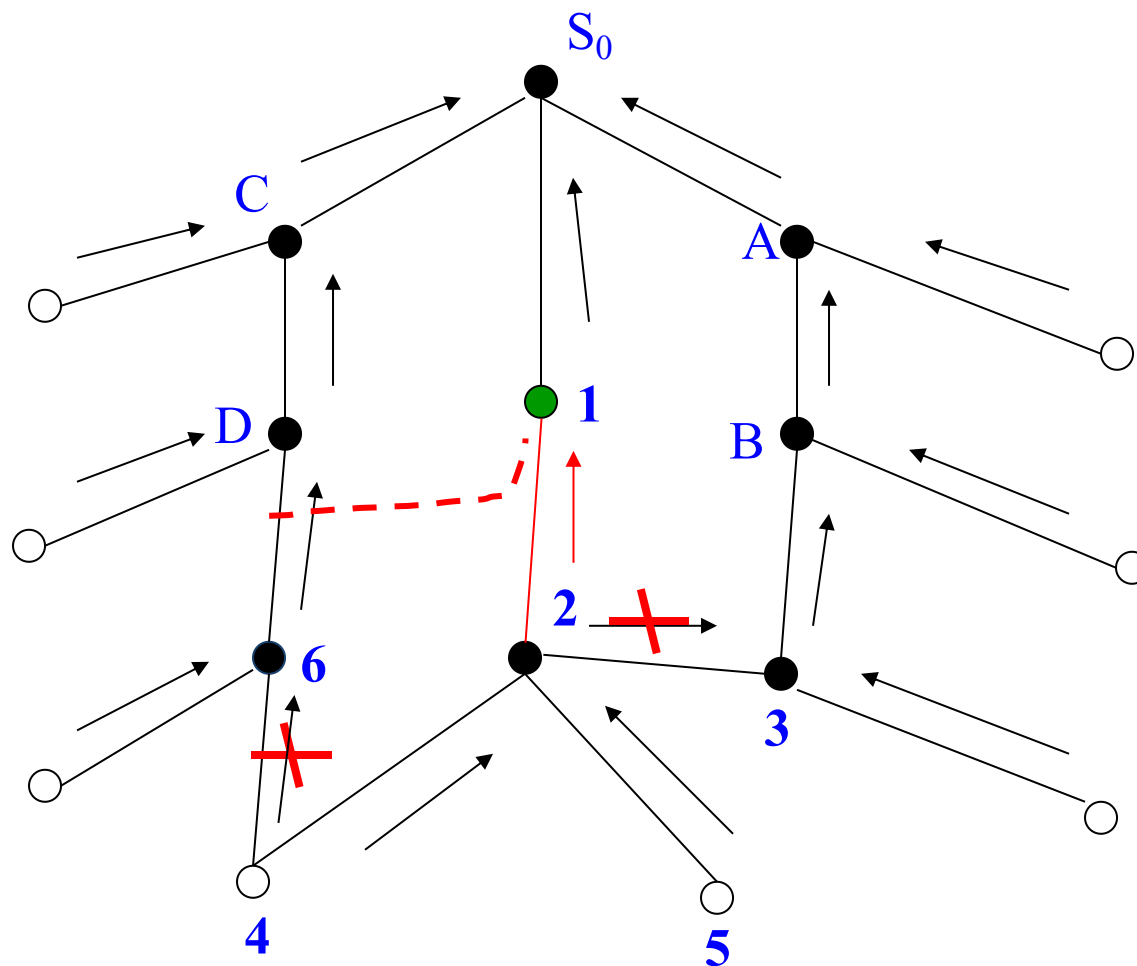
原有指针返回路径：4→2→3→B→A→S0

修改后指针返回路径：4→6→D→C→S0

修改返回指针

② 对那些原来已经在G中出现过，但没有被扩展过的M成员，确定是否需要修改它指向父节点的指针。

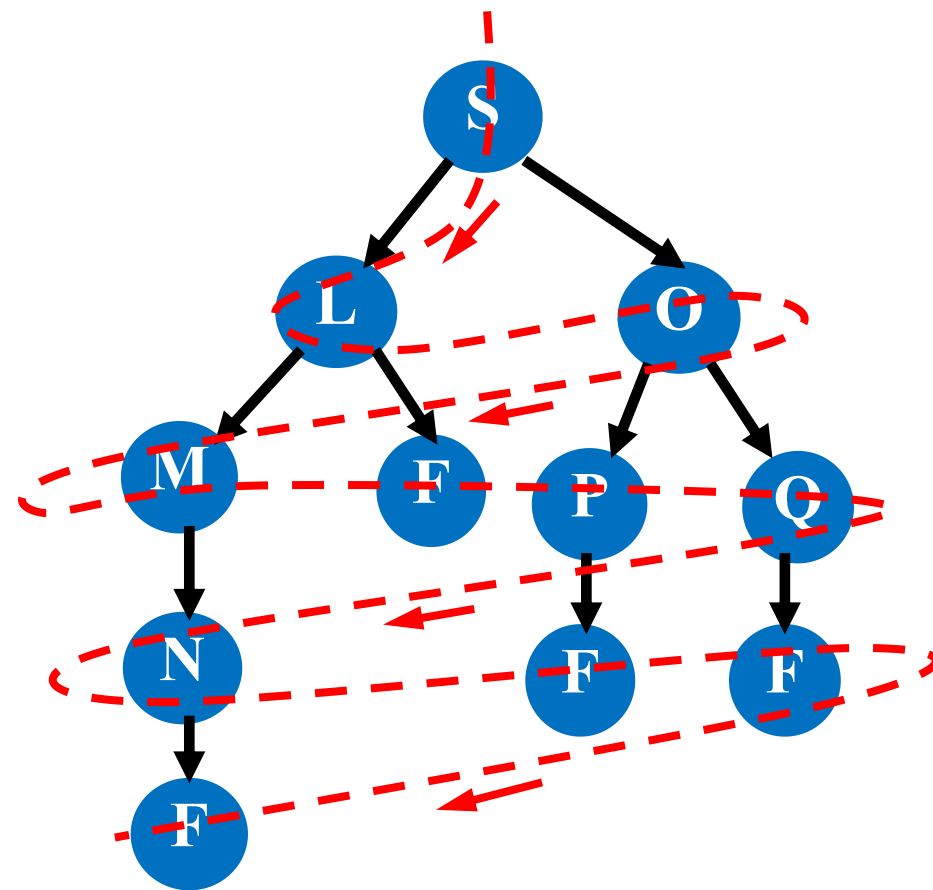
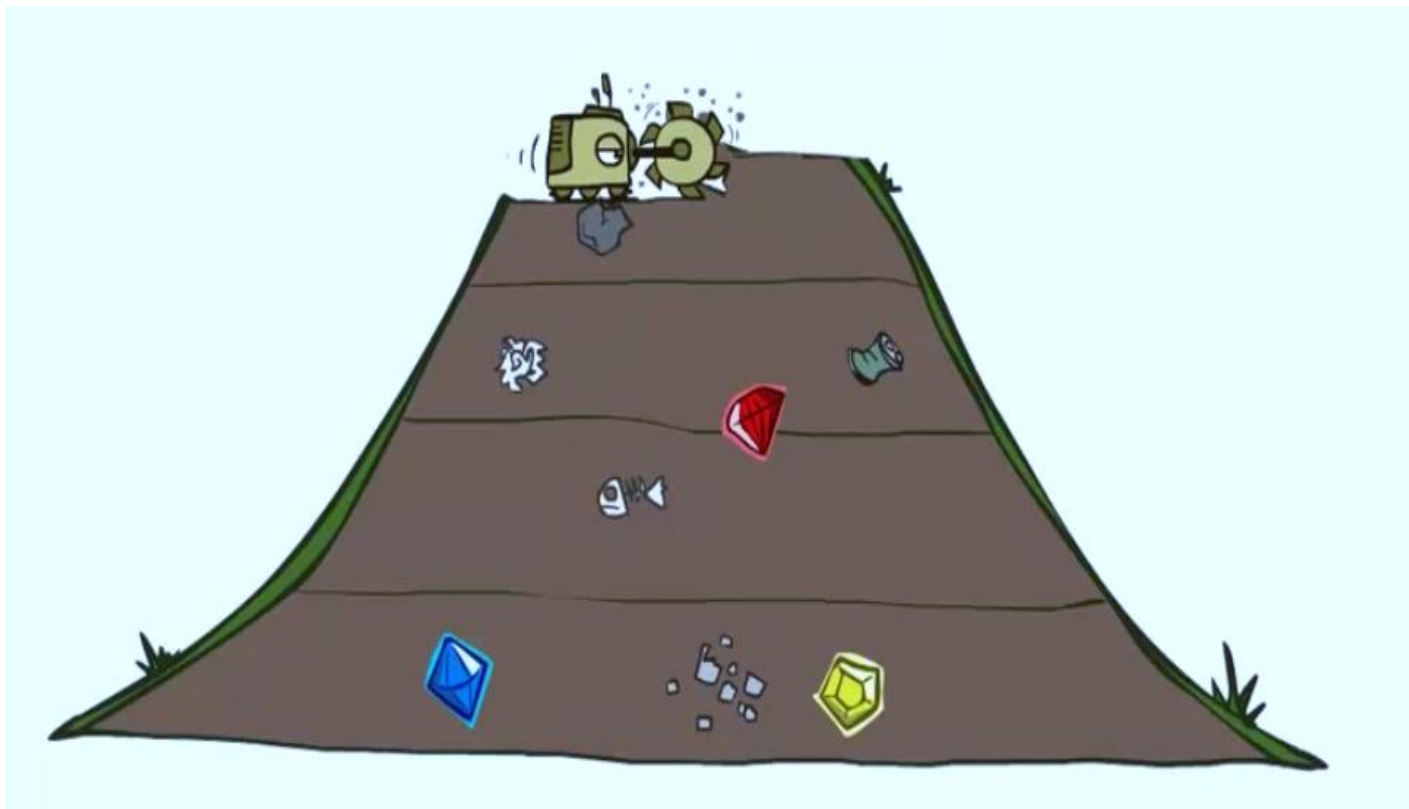
③ 对那些原来已经在G中出现过，并已经被扩展过的M成员，确定是否需要修改其后继节点指向父节点的指针。



原有指针返回路径：4→6→D→C→S0

修改后指针返回路径：4→2→1→S0

4.2.3 广度优先搜索(Breadth-first Search)



4.2.3 广度优先搜索(Breadth-first Search)

又称为宽度优先搜索，是一种先生成的节点先扩展的策略。

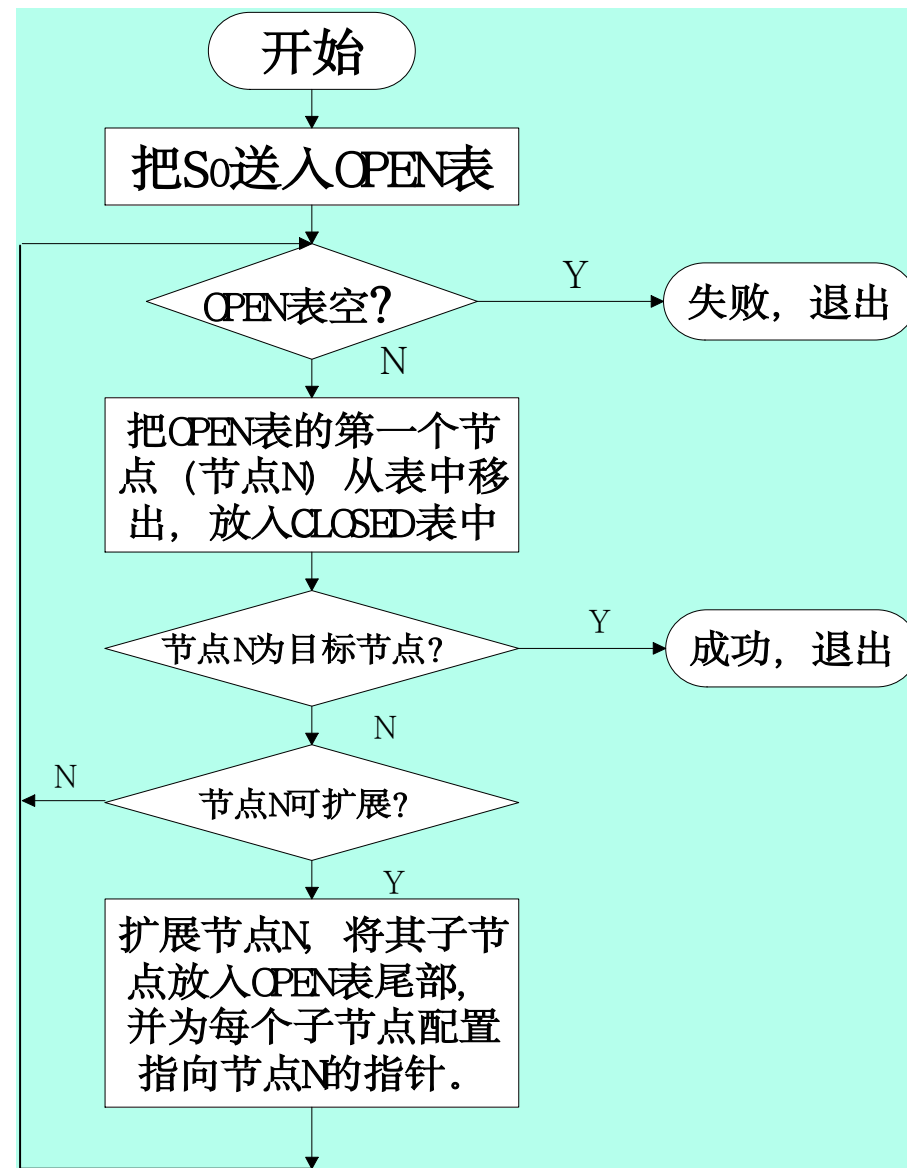
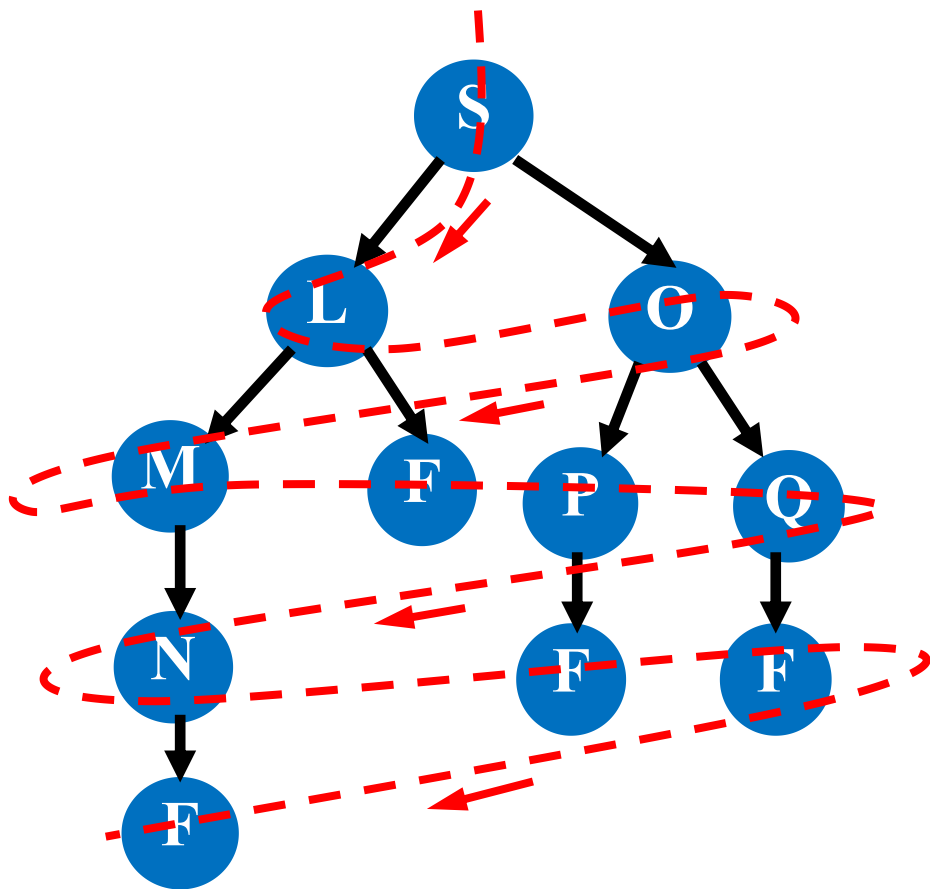
宽度优先搜索的基本思想是：从初始节点 S_0 开始，逐层地对节点进行扩展并考察它是否为目标节点，在第 n 层的节点没有全部扩展并考察之前，不对第 $n+1$ 层的节点进行扩展。

OPEN表中的节点总是按进入的先后顺序排列，先进入的节点排在前面，后进入的排在后面。

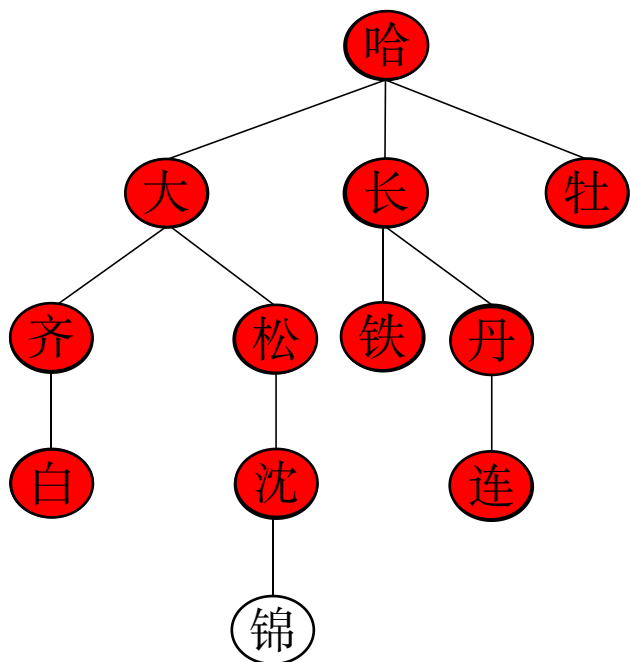
广度优先搜索的一般算法

- (1) 把初始节点 S_0 放入Open表, 建立一个CLOSED表, 置为空;
- (2) 检查Open表是否为空表, 若为空, 则问题无解, 失败退出;
- (3) 把Open表的第一个节点取出放入Closed表, 并记该节点为n;
- (4) 考察节点n是否为目标节点, 若是则得到问题的解成功退出;
- (5) 若节点n不可扩展, 则转第(2)步;
- (6) 扩展节点n, 将其子节点放入Open表的尾部, 并为每个子节点设置指向父节点的指针, 转向第(2)步。

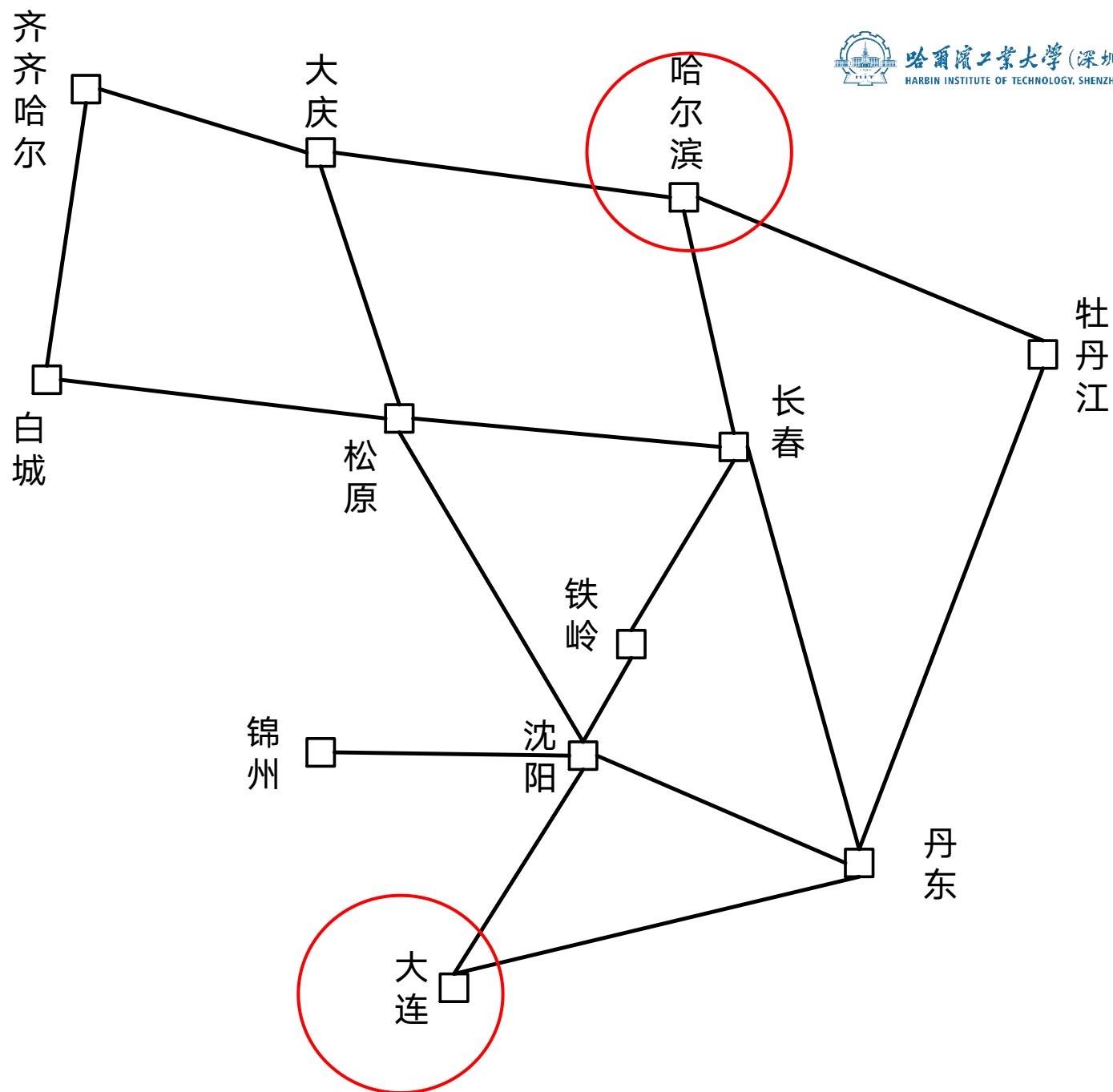
广度优先搜索的一般算法



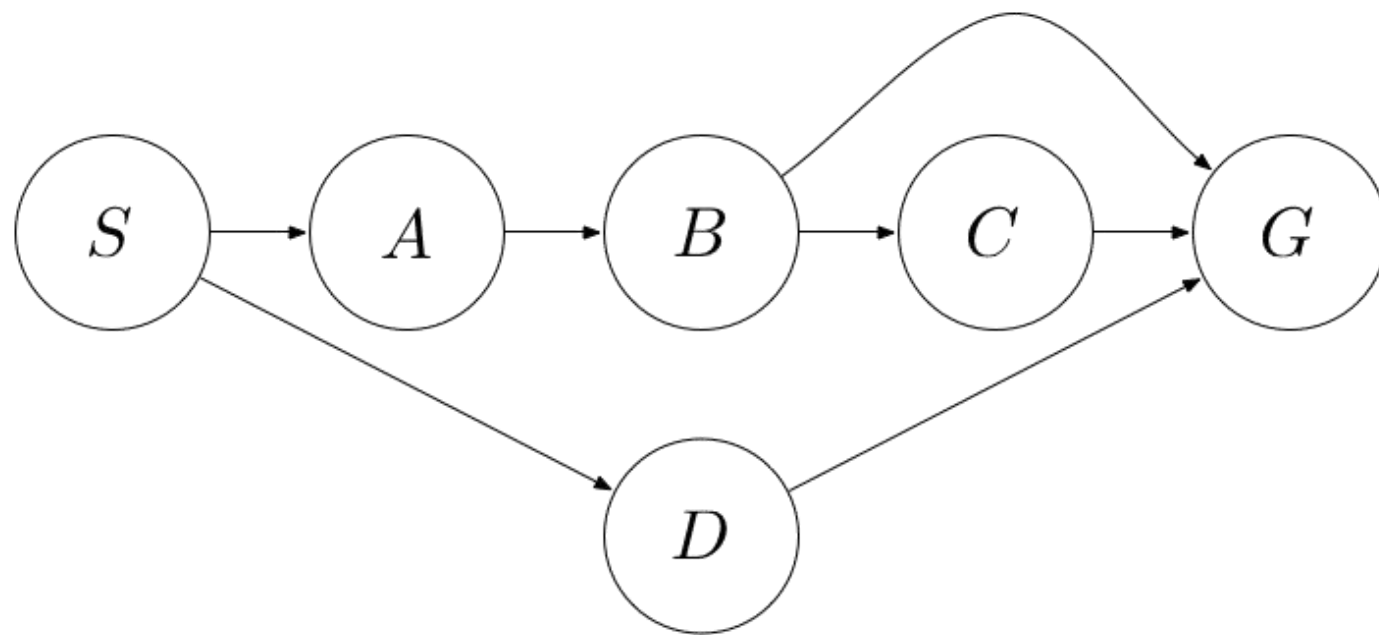
路径搜索问题:



- 广度(宽度)优先搜索
(Breadth-first search)



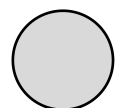
求下图所示从 S 到 G 的广度优先搜索 (Breadth-first search) 树



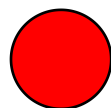
- 假设同一深度扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展

作答

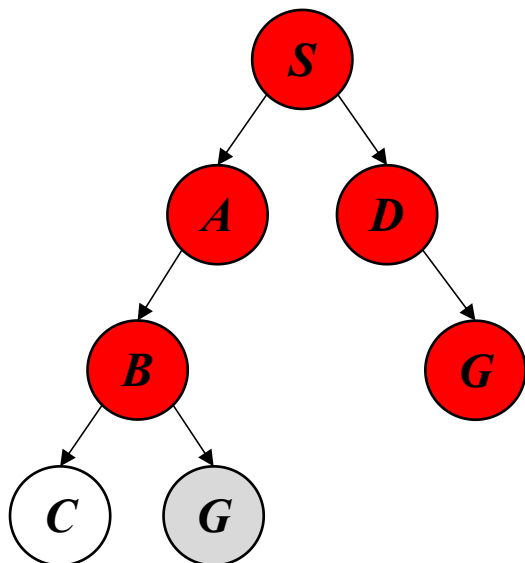
4.2.3 广度优先搜索 (Breadth-first search)



因为修改父节点指向删掉的节点

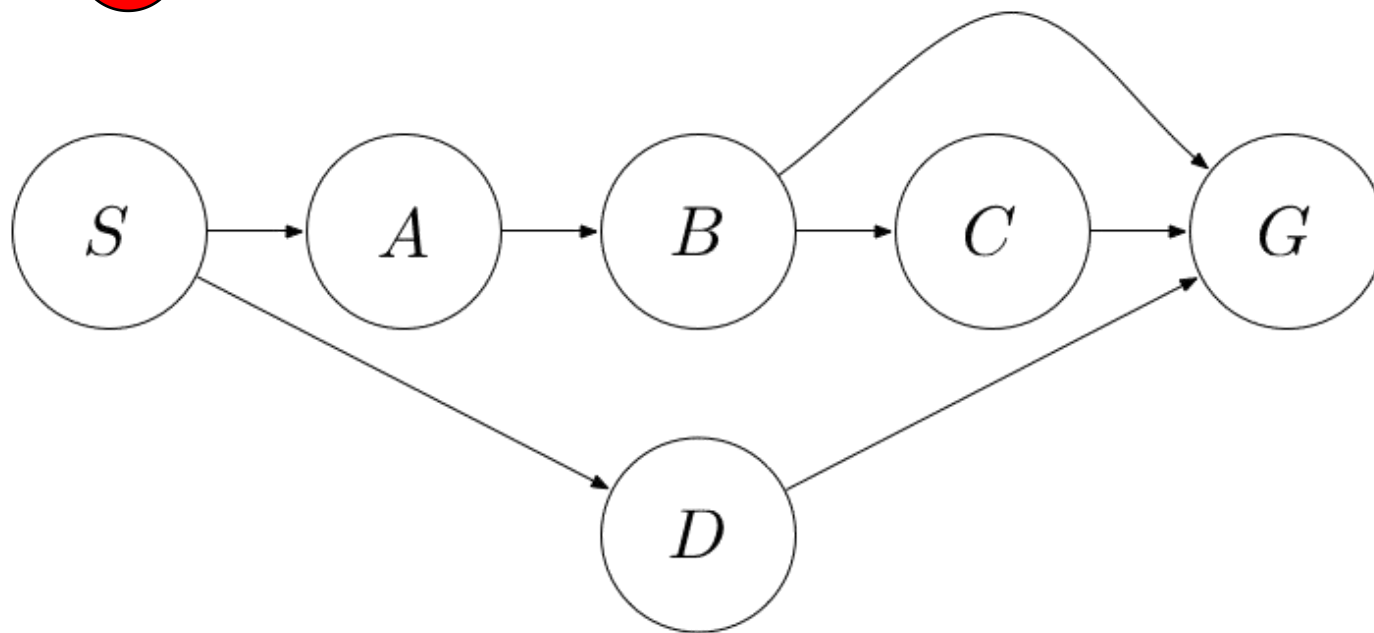


已扩展的节点



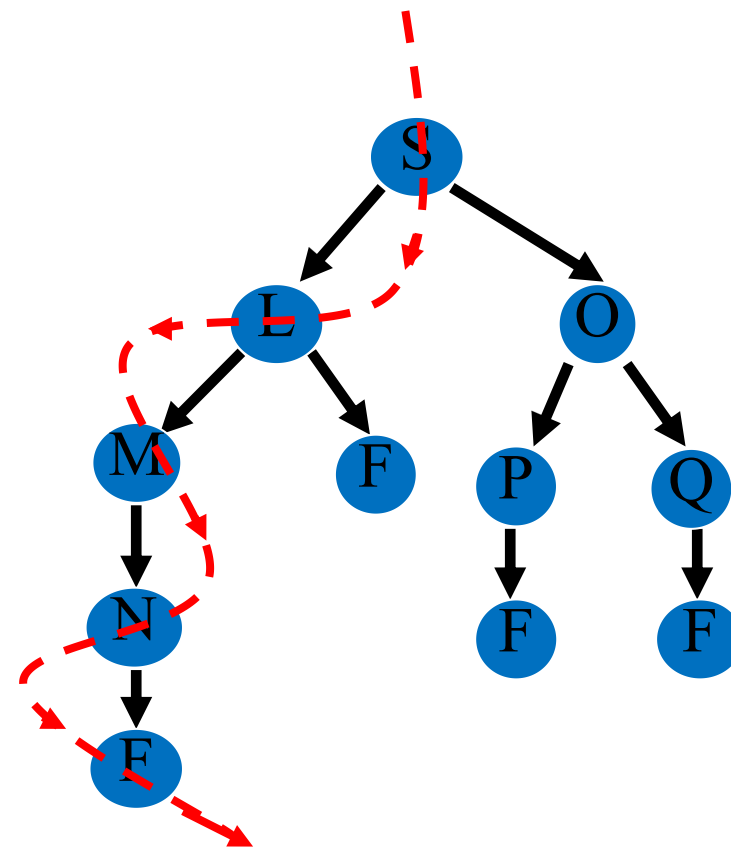
搜索顺序: $S \rightarrow A \rightarrow D \rightarrow B \rightarrow G$

解: $S \rightarrow D \rightarrow G$



- 假设同一深度扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展

4.2.4 深度优先搜索 (Depth-first search)



4.2.4 深度优先搜索 (Depth-first search)

在深度优先搜索中，首先扩展最新产生的(即最深的)节点，也即**后生成的节点先扩展**的策略。

从初始节点 S_0 开始扩展，若没有得到目标节点，则选择最新产生的子节点进行扩展；若还是不能到达目标节点，则再对刚才最新产生的子节点进行扩展，一直如此向下搜索。

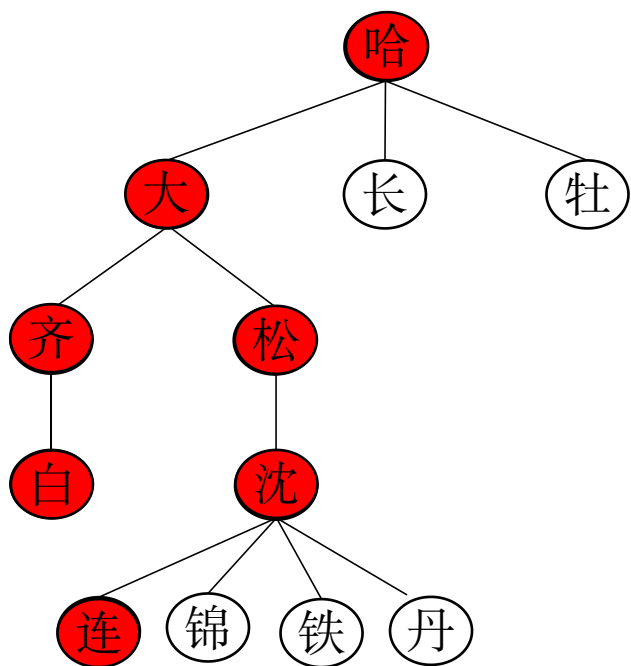
当到达某个子节点，且该子节点既不是目标节点又不能继续扩展时，才选择其兄弟节点进行考察。

Open表是一种栈结构，最先进入的节点排在最后面，最后进入的节点排最前面。

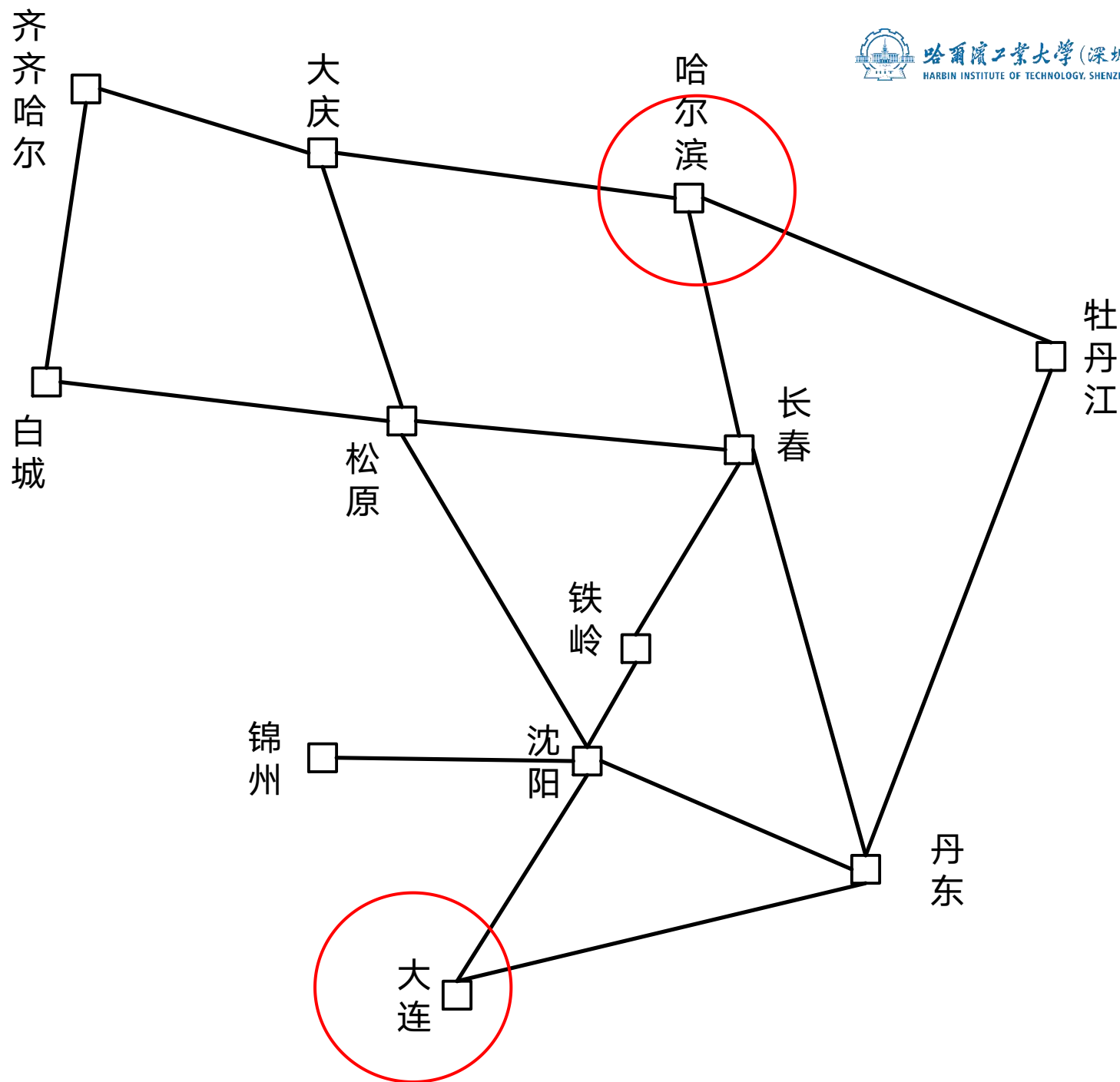
深度优先搜索的一般算法

- (1) 把初始节点 S_0 放入Open表, 建立一个CLOSED表, 置为空;
- (2) 检查Open表是否为空表, 若为空, 则问题无解, 失败退出;
- (3) 把Open表的第一个节点取出放入Closed表, 并记该节点为n;
- (4) 考察节点n是否为目标节点, 若是则得到问题的解成功退出;
- (5) 若节点n不可扩展, 则转第(2)步;
- (6) 扩展节点n, 将其子节点放入Open表的**首**部, 并为每个子节点设置指向父节点的指针, 转向第(2)步。

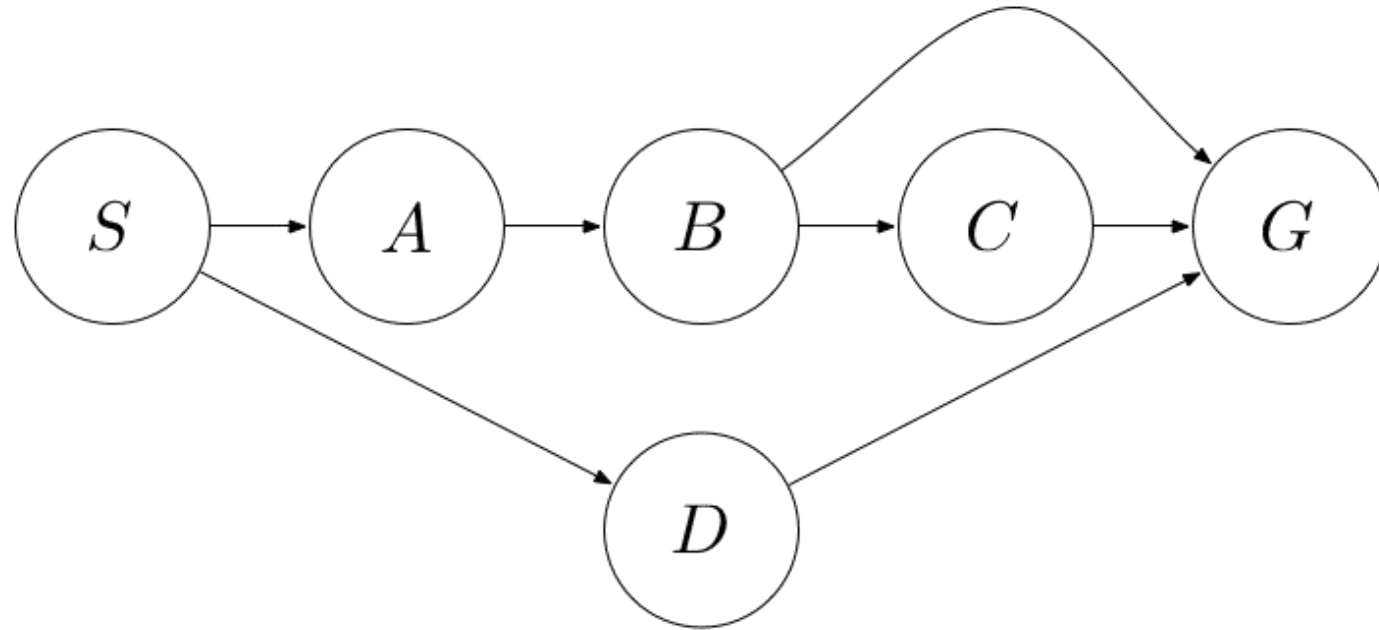
路径搜索问题：



- 深度(宽度)优先搜索
(Depth-first search)



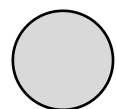
求下图所示从 S 到 G 的深度优先搜索 (Depth-first search) 树



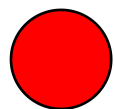
- 假设同一深度扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展

作答

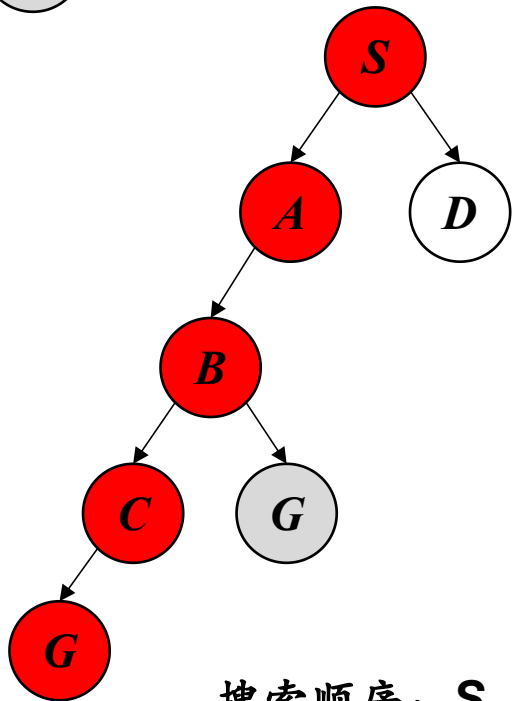
4.2.4 深度优先搜索 (Depth-first search)



因为修改父节点指向删掉的节点



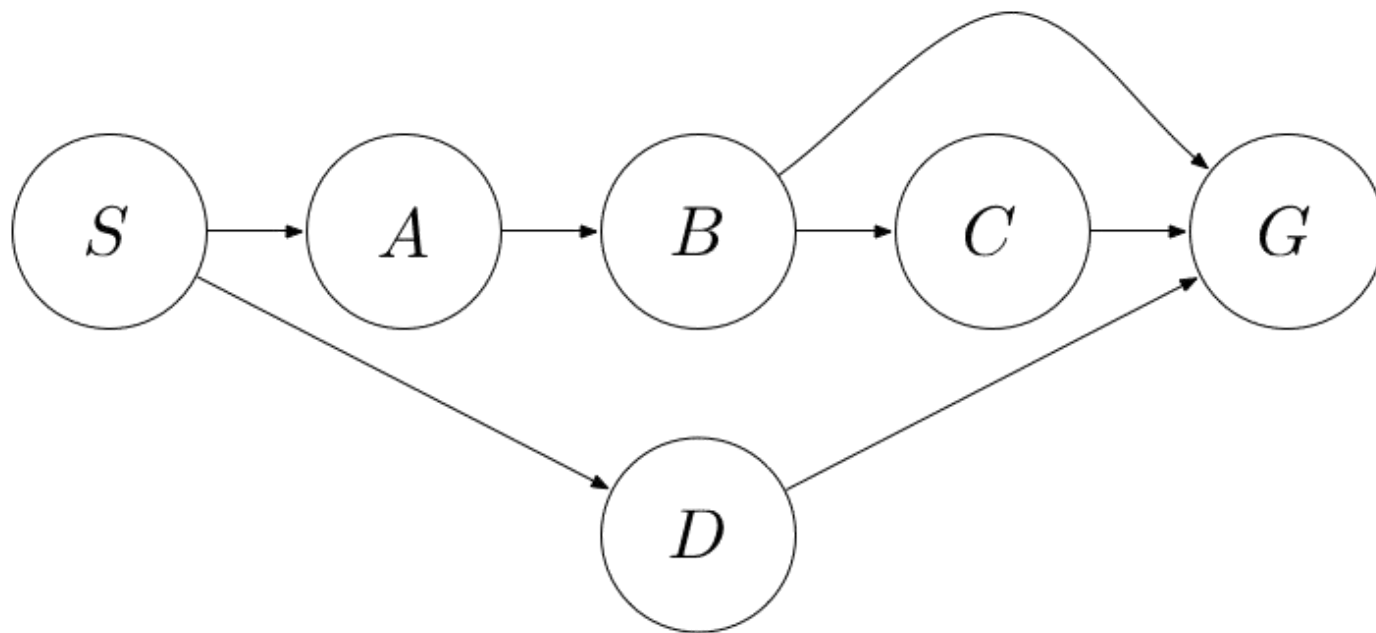
已扩展的节点



搜索顺序: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

解: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

- 假设同一深度扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展



4.2.4 深度优先搜索 (Depth-first search)

在深度优先搜索中，首先扩展最新产生的(即最深的)节点，也即**后生成的节点先扩展**的策略。

从初始节点 S_0 开始扩展，若没有得到目标节点，则选择最新产生的子节点进行扩展；若还是不能到达目标节点，则再对刚才最新产生的子节点进行扩展，一直如此向下搜索。

当到达某个子节点，且该子节点既不是目标节点又不能继续扩展时，才选择其兄弟节点进行考察。

Open表是一种栈结构，最先进入的节点排在最后面，最后进入的节点排最前面。

深度优先搜索的一般算法

- (1) 把初始节点 S_0 放入Open表, 建立一个CLOSED表, 置为空;
- (2) 检查Open表是否为空表, 若为空, 则问题无解, 失败退出;
- (3) 把Open表的第一个节点取出放入Closed表, 并记该节点为n;
- (4) 考察节点n是否为目标节点, 若是则得到问题的解成功退出;
- (5) 若节点n不可扩展, 则转第(2)步;
- (6) 扩展节点n, 将其子节点放入Open表的**首**部, 并为每个子节点设置指向父节点的指针, 转向第(2)步。

4.2.4.1 搜索的性质

广度优先搜索(Breadth-first Search)

- 当问题有解时，一定能找到解
- 搜索效率较低
- 方法与问题无关，具有通用性
- 搜索得到的解是搜索树中路径最短的解 (最优解)
- 属于完备搜索策略

4.2.4.2 搜索的性质

深度优先搜索 (Depth-first search)

- 一般不能保证找到最优解
- 最坏情况时，搜索空间等同于穷举
- 是一个通用的与问题无关的方法
- 为了解决深度优先搜索不完备问题，避免搜索过程陷入无穷分支的死循环，提出了有界深度优先搜索方法。

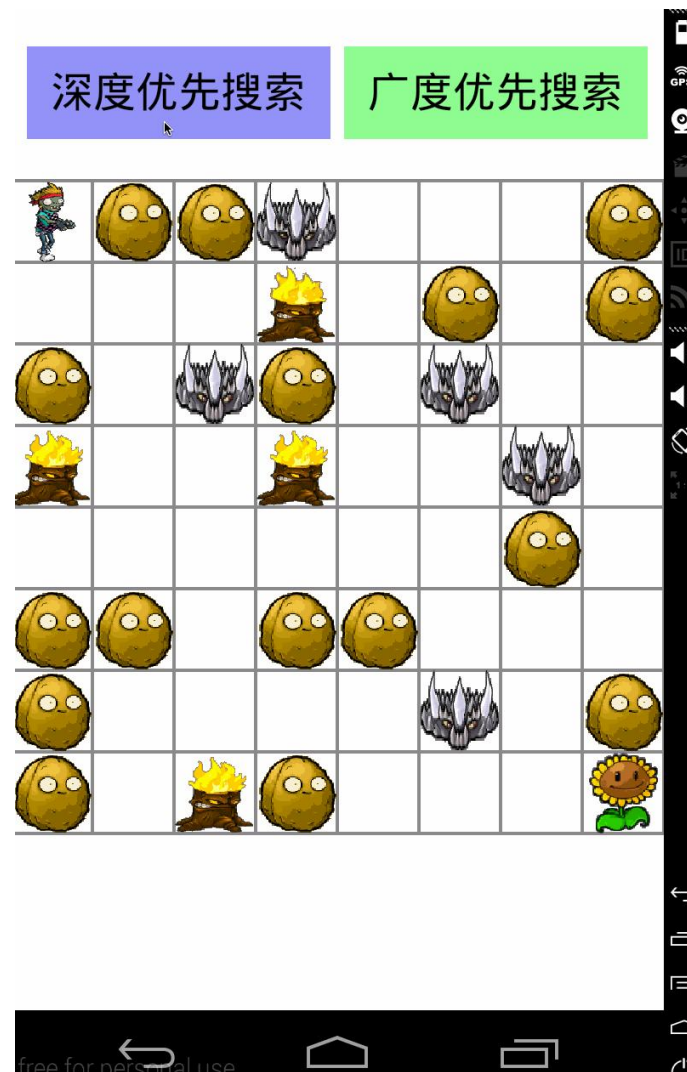
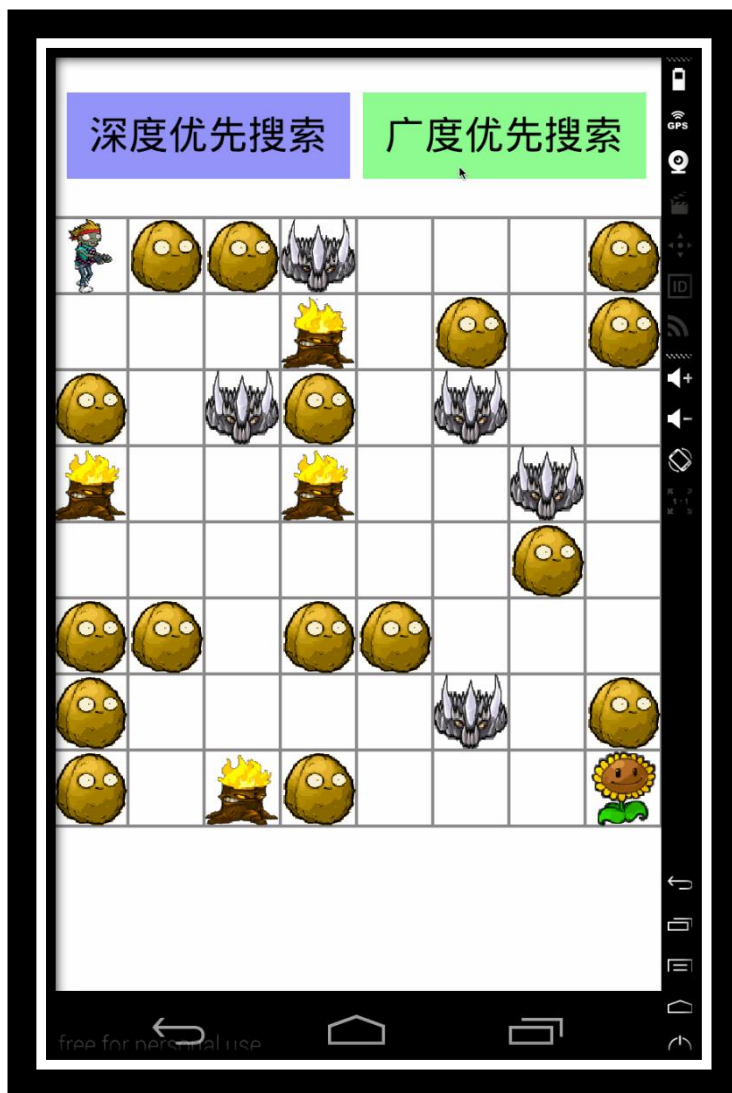
广度优先搜索示例



深度优先搜索示例



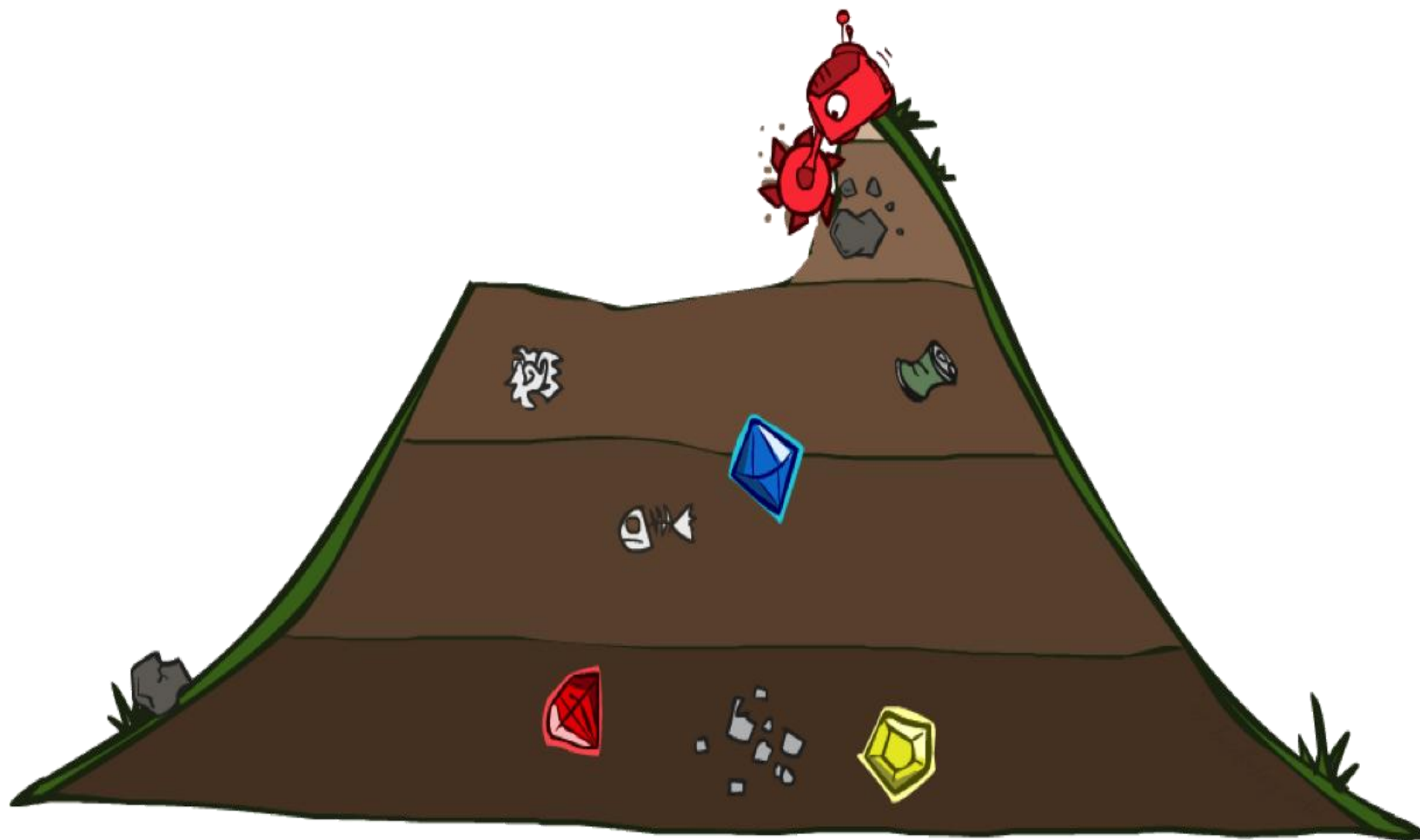
广度优先搜索 VS 深度优先搜索



4.2.4 代价一致搜索(Uniform-cost/Cheapest-first search)



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN



4.2.4 代价一致搜索(Uniform-cost/Cheapest-first search)

前面的各种搜索策略中，实际都假设状态空间中**各边的代价都相同**，且都为一个单位量。从而可用路径长度来代替路径的代价。但实际问题中，这种假设不现实，它们的状态空间中**各个边的代价不可能完相同**。为此，我们需要在搜索树中给每条边标上其代价。这种边上有代价的树称为代价树。

在代价树中，可以用 $g(n)$ 表示从初始节点 S_0 到节点 n 的代价，用 $c(n_1, n_2)$ 表示从父节点 n_1 到 n_2 的代价。这样，对节点 n_2 的代价有

$$g(n_2) = g(n_1) + c(n_1, n_2)$$

在代价树中，**最小代价的路径和最短路径是有可能不同的**，**最短路径不一定是最小代价径**，**最小代价路径也不一定是路径最短**。代价树搜索的目的是为了到最佳解，即找到一条代价最小的解路径。

4.2.4 代价一致搜索 (Uniform-cost/Cheapest-first search)

搜索树中每条连接线上的有关代价,表示时间、距离等花费。

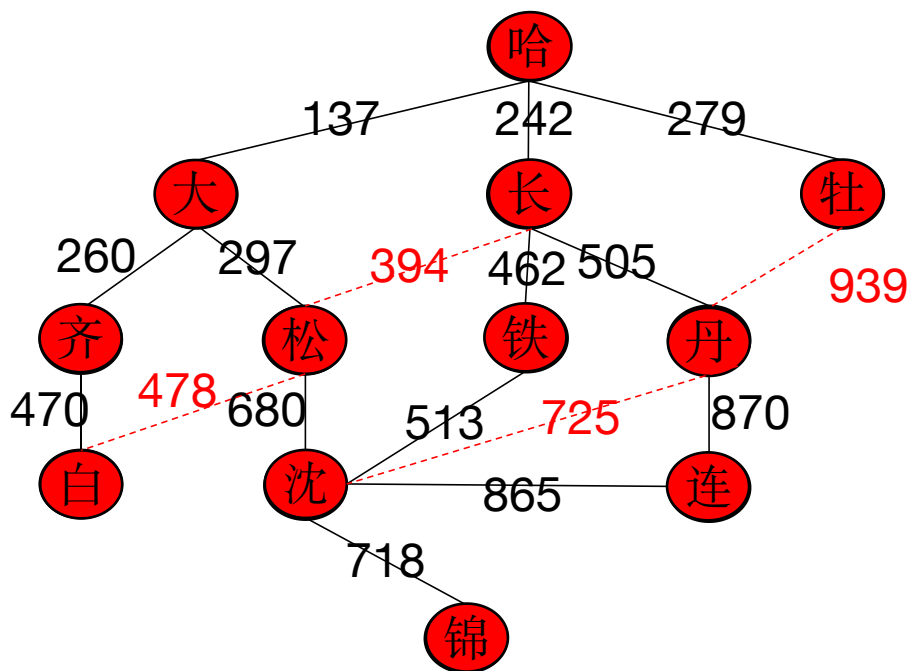
代价一致搜索是宽度优先搜索的一种推广,不是沿着等长度路径断层进行扩展,而是沿着等代价路径断层进行扩展。

代价一致搜索的**基本思想**是:在代价一致搜索算法中,把从起始节点 S_0 到任一节点 i 的路径代价记为 $g(i)$ 。从初始节点 S_0 开始扩展,若没有得到目标节点,则优先扩展最少代价 $g(i)$ 的节点,一直如此向下搜索。

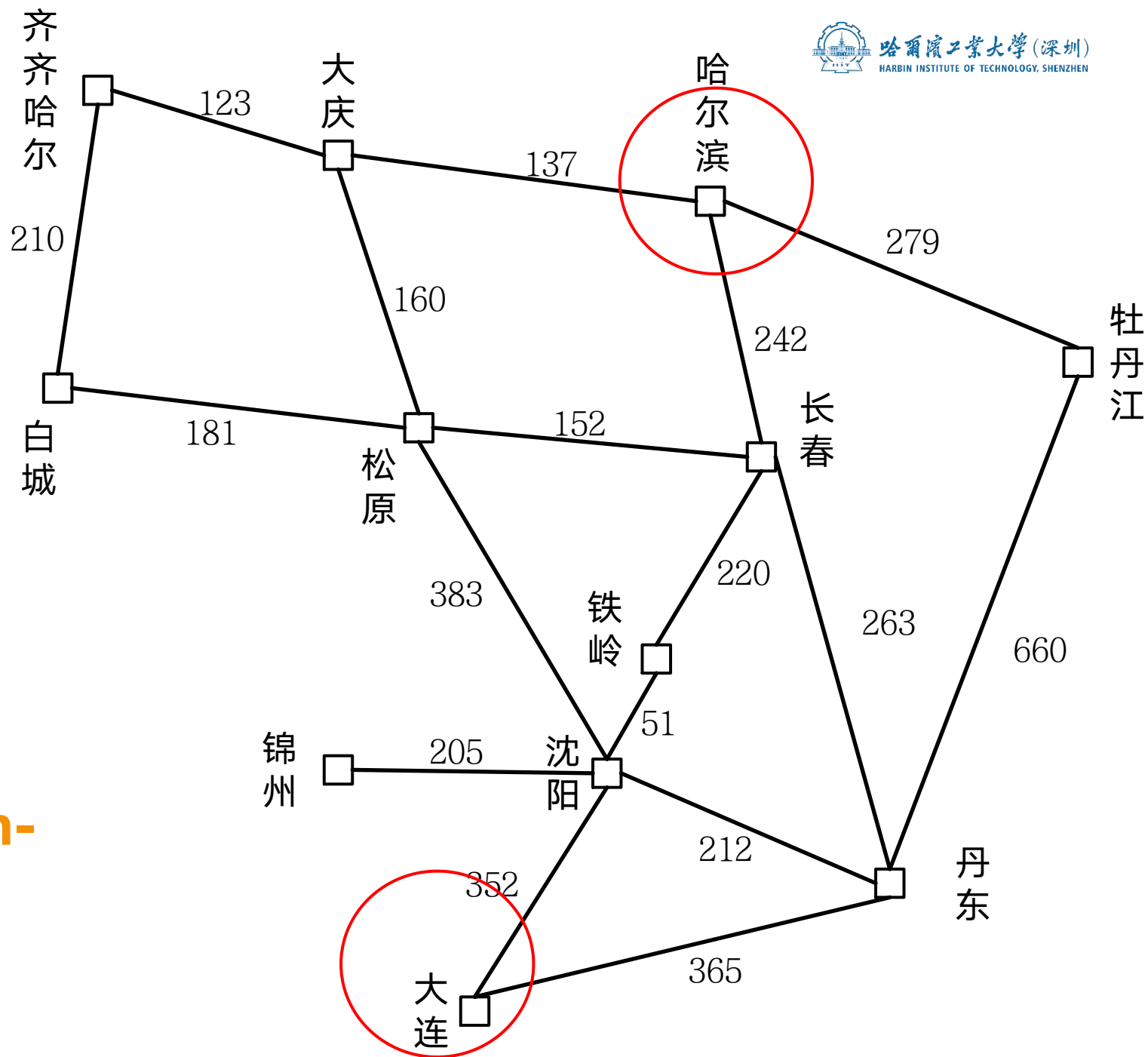
代价一致搜索的一般算法

- (1) 把初始节点 S_0 放入Open表, 设 $g(s_0)=0$, 建立一个CLOSED表, 置为空;
- (2) 检查Open表是否为空表, 若为空, 则问题无解, 失败退出;
- (3) 把Open表的第一个节点取出放入Closed表, 并记该节点为 n ;
- (4) 考察节点 n 是否为目标节点, 若是则得到问题的解成功退出;
- (5) 若节点 n 不可扩展, 则转第(2)步;
- (6) 扩展节点 n , 生成子节点 n_i ($i=1,2,\dots$), 将其子节点放入Open表, 并为每个子节点设置指向父节点的指针, 计算各个节点的代价 $g(n_i)$, 将Open表内的节点按 $g(n_i)$ 从小到大排序, 转向第(2)步。

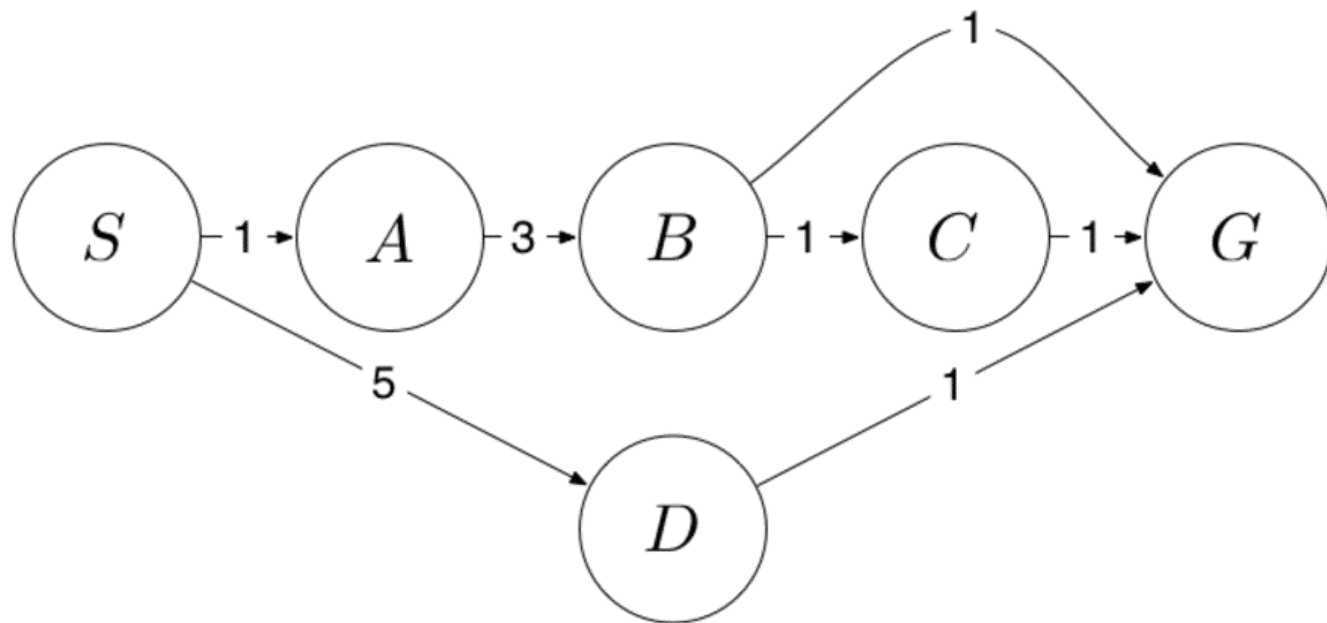
路径搜索问题:



●代价一致搜索 (Uniform-cost/Cheapest-first search)



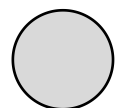
求下图所示从 S 到 G 的代价一致搜索 (UCS) 树



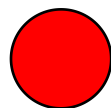
- 假设同一优先级扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展

作答

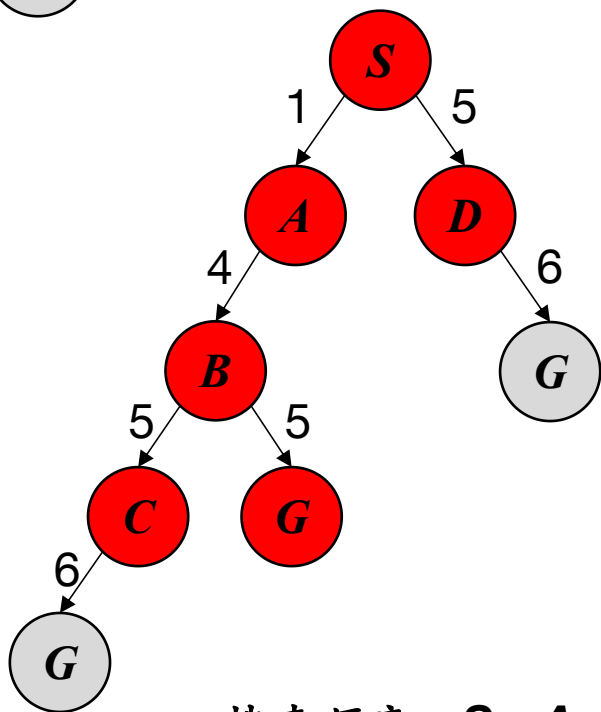
4.2.4代价一致搜索 (UCS)



因为修改父节点指向删掉的节点



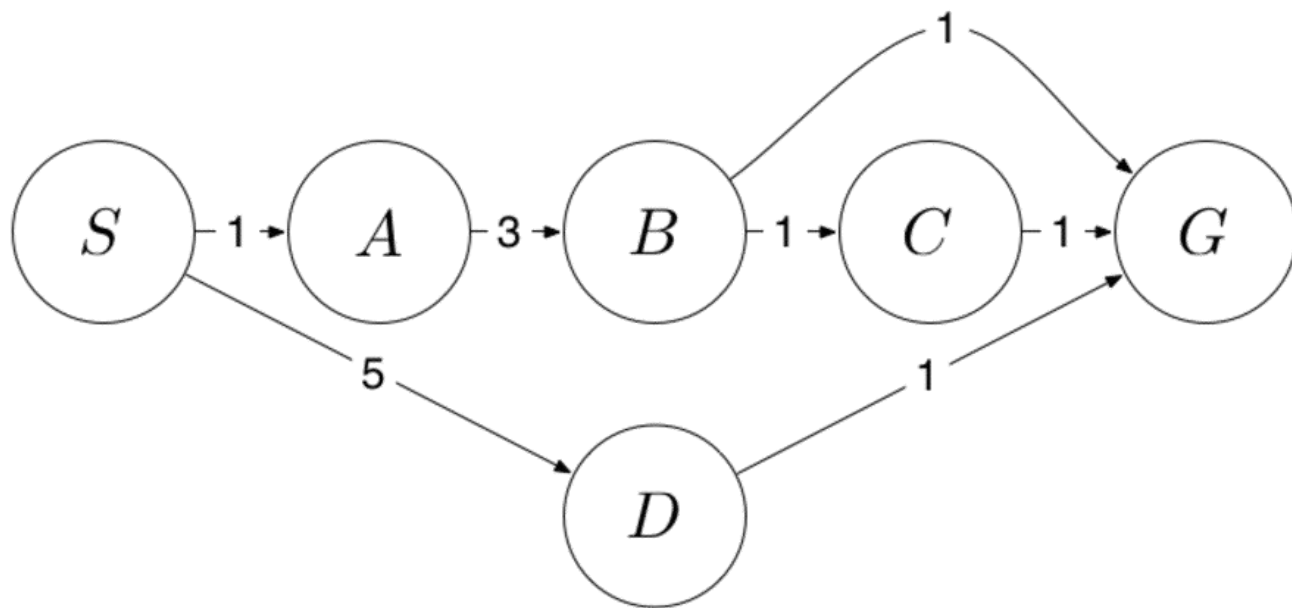
已扩展的节点



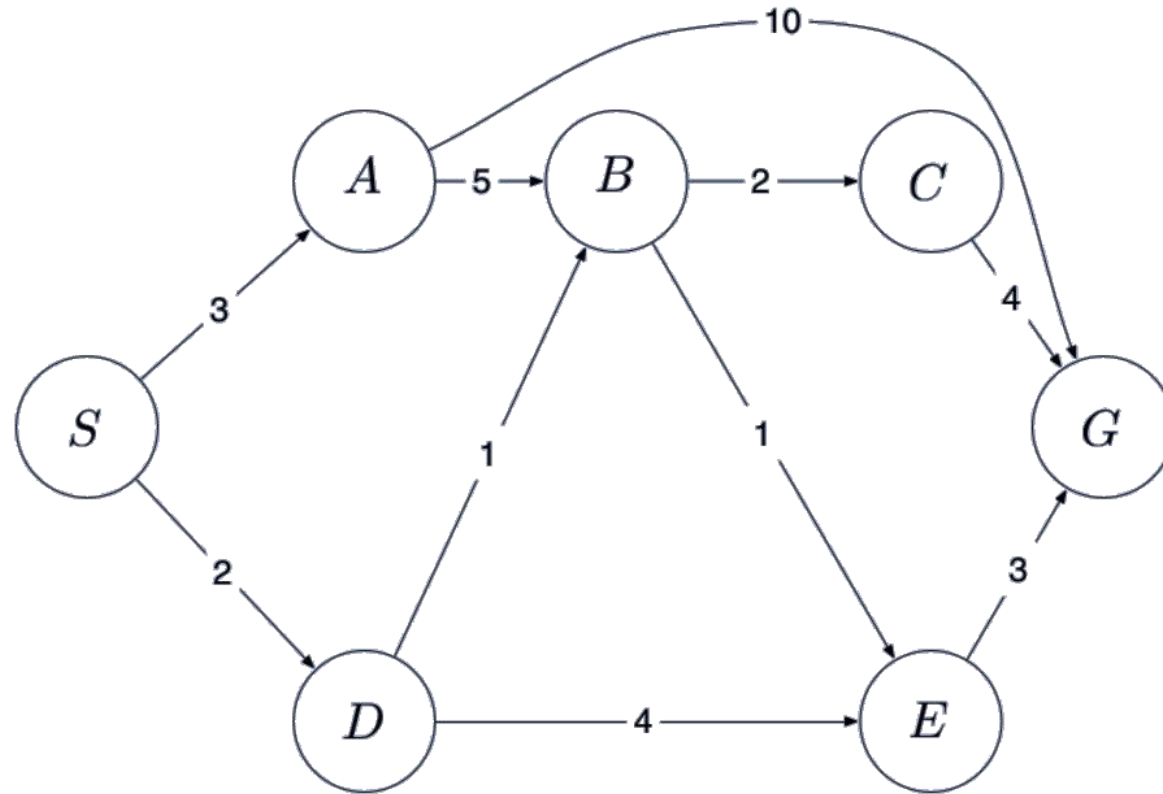
搜索顺序: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$

解: $S \rightarrow A \rightarrow B \rightarrow G$

- 假设同一优先级扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展



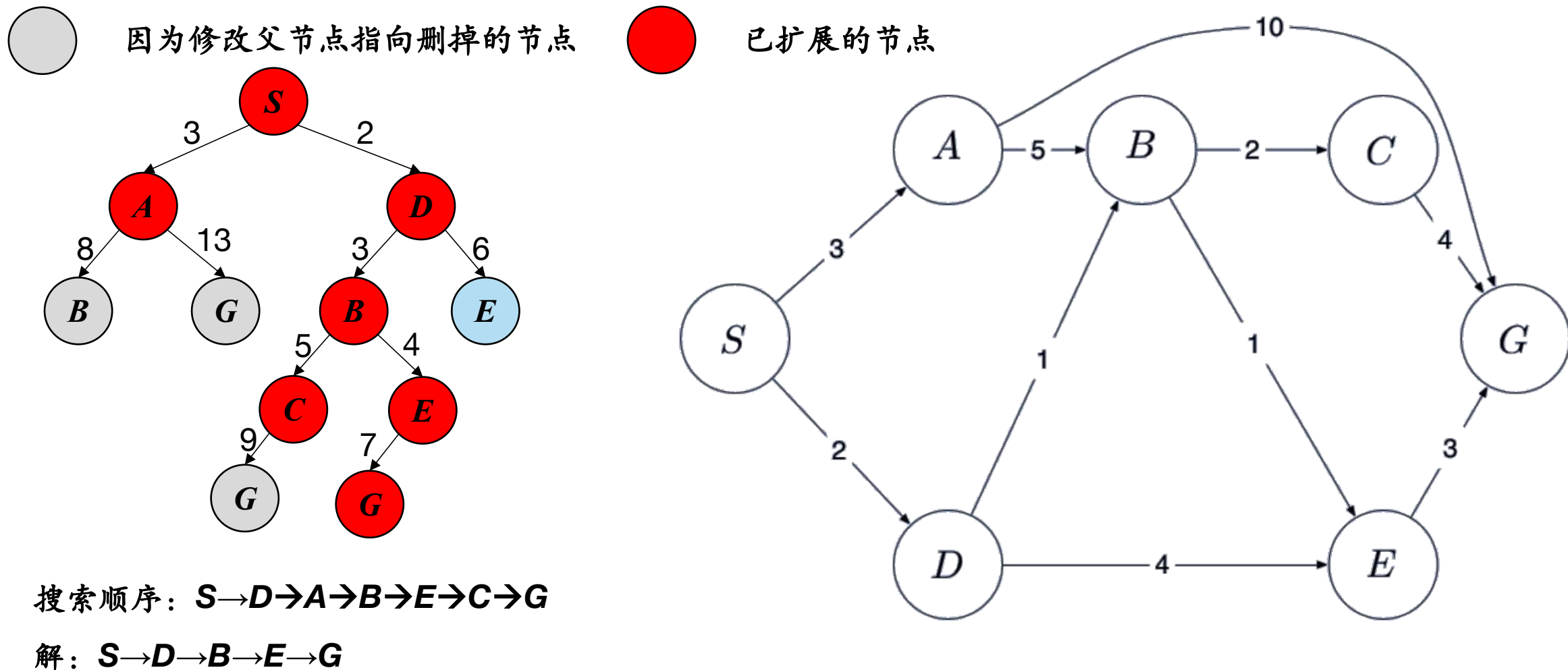
求下图所示从 S 到 G 的代价一致搜索 (UCS) 树



- 假设同一优先级扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展

作答

4.2.4 代价一致搜索 (UCS)



- 假设同一优先级扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展

1 一般图搜索过程

- (1) 把初始节点 S_0 放入Open表，并建立目前仅包 S_0 的图G，建立一个Closed表，置为空；
- (2) 检查Open表是否为空表，若为空，则问题无解，失败退出
- (3) 把Open表的第一个节点取出放入Closed表，并记该节点为n
- (4) 考察节点n是否为目标节点，若是则得到问题的解成功退出。
- (5) 扩展节点n，生成一组子节点。把这些子节点中不是其父节点的那部分子节点计入集合M，并把这些子节点作为节点n的子节点加G中。

1 一般图搜索过程

(6) 针对**M**中子节点的不同情况，分别作如下处理：

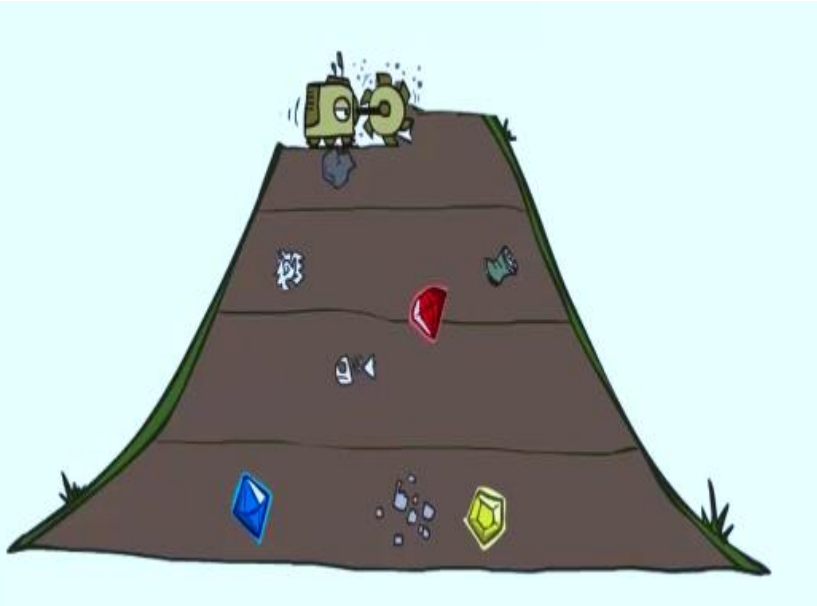
- ① 对那些没有在**G**中出现过的**M**成员设置一个指向其父节点（即节点**n**）的指针，并将他它放入**Open**表。
- ② 对那些原来已经在**G**中出现过，但__没有被扩展过的**M**成员，确定是否需要修改它指向父节点的指针。
- ③ 对那些原来已经在**G**中出现过，并已经被扩展过的**M**成员，确定是否需要修改其后继节点指向父节点的指针。

(7) 按某种策略对**Open**表中的节点进行排序

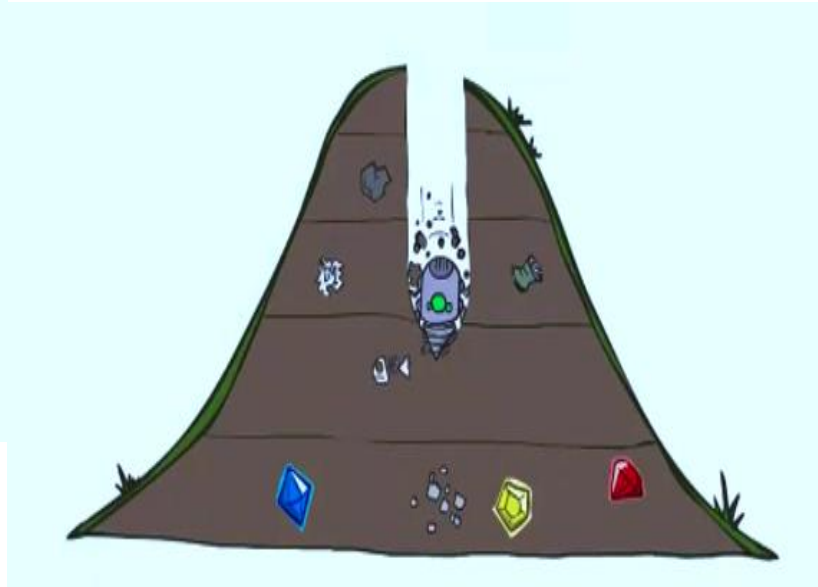
(8) 转第 (2) 步

小结

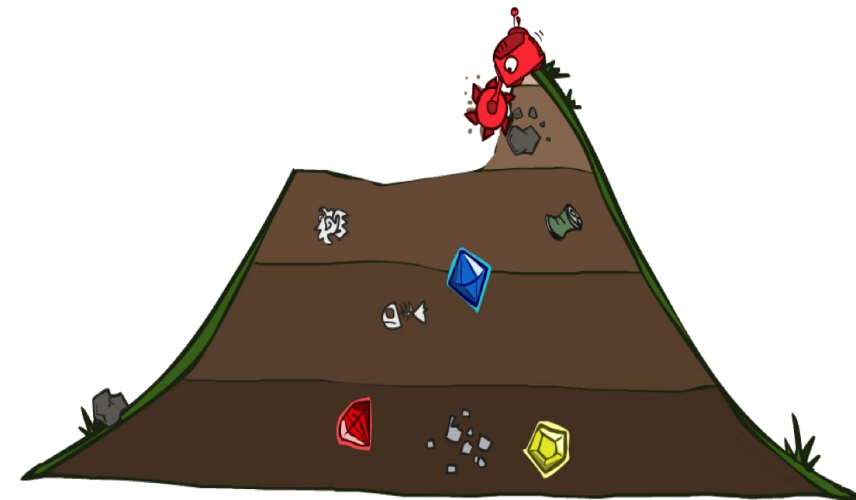
2 搜索策略



广度优先搜索 (BFS)



深度优先搜索 (DFS)



代价一致搜索 (UCS)

目录

- 01 4.1 概述
- 02 4.2 状态空间盲目搜索
- 03 4.3 状态空间启发式搜索
- 04 4.4 与/或树搜索
- 05 4.5 博弈树的启发式搜索

4.3.1 启发性信息

□ 启发性信息

启发性信息是指那种与具体问题求解过程有关的，并可指导搜索过程朝着最有希望方向前进的控制信息。

启发信息的启发能力越强，扩展的无用结点越少，搜索算法越高效。
启发信息包括以下3种：

- ①有效地帮助确定扩展节点的信息；
- ②有效地帮助决定哪些后继节点应被生成的信息；
- ③能决定在扩展一个节点时哪些节点应从搜索树上删除的信息。

□ 估价函数

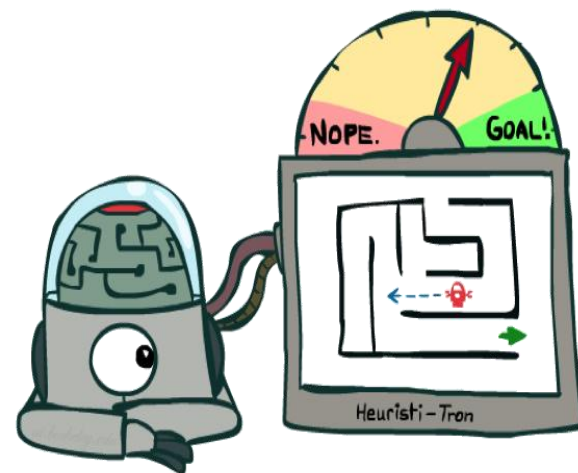
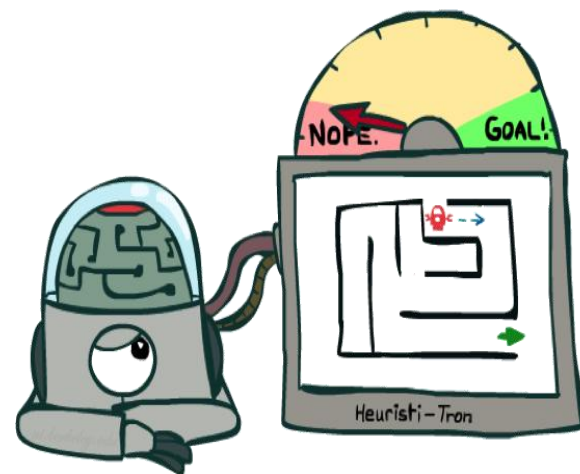
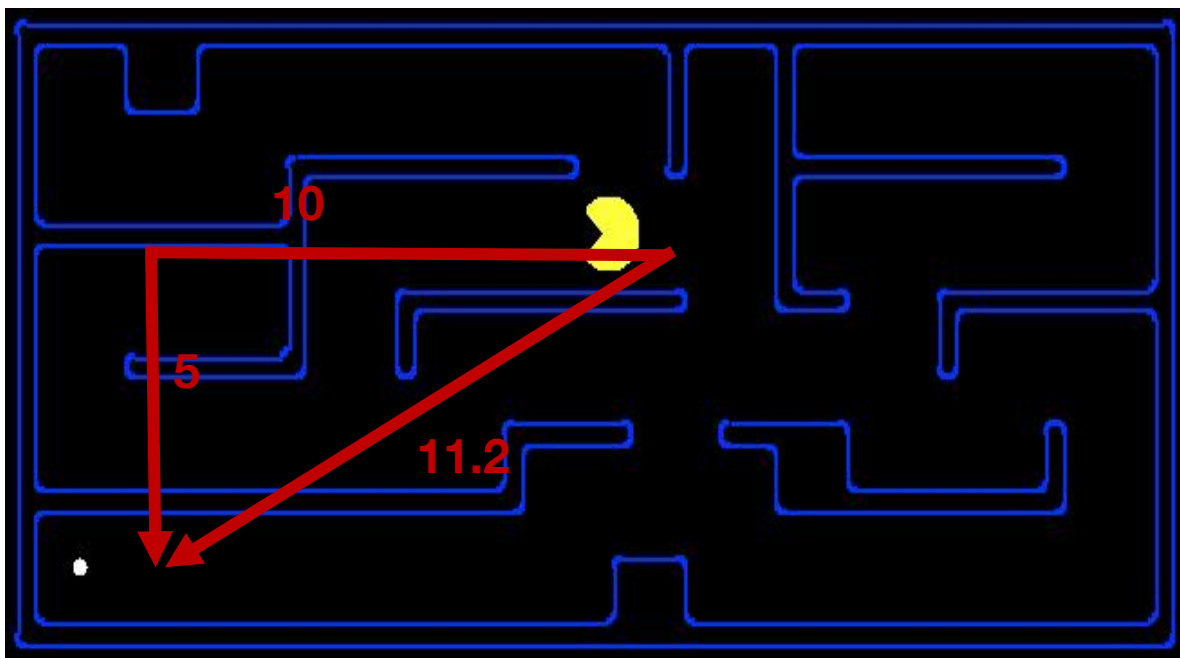
$$f(n) = g(n) + h(n)$$

$g(n)$ 是从初始节点到节点 n 的**实际代价**， $h(n)$ 是从节点 n 到目标节点的最优路径的**估计代价**，需要根据问题自身特性来确定，体现的是问题自身的启发性信息，因此也称为**启发函数**。

4.3.1 启发性信息

□ 启发式函数

- 启发函数用于评估一个状态接近目标的程度
- 为特定搜索问题专门设计
- 用于路径搜索的曼哈顿距离，欧式距离等



4.3.2 贪心算法(Greedy)

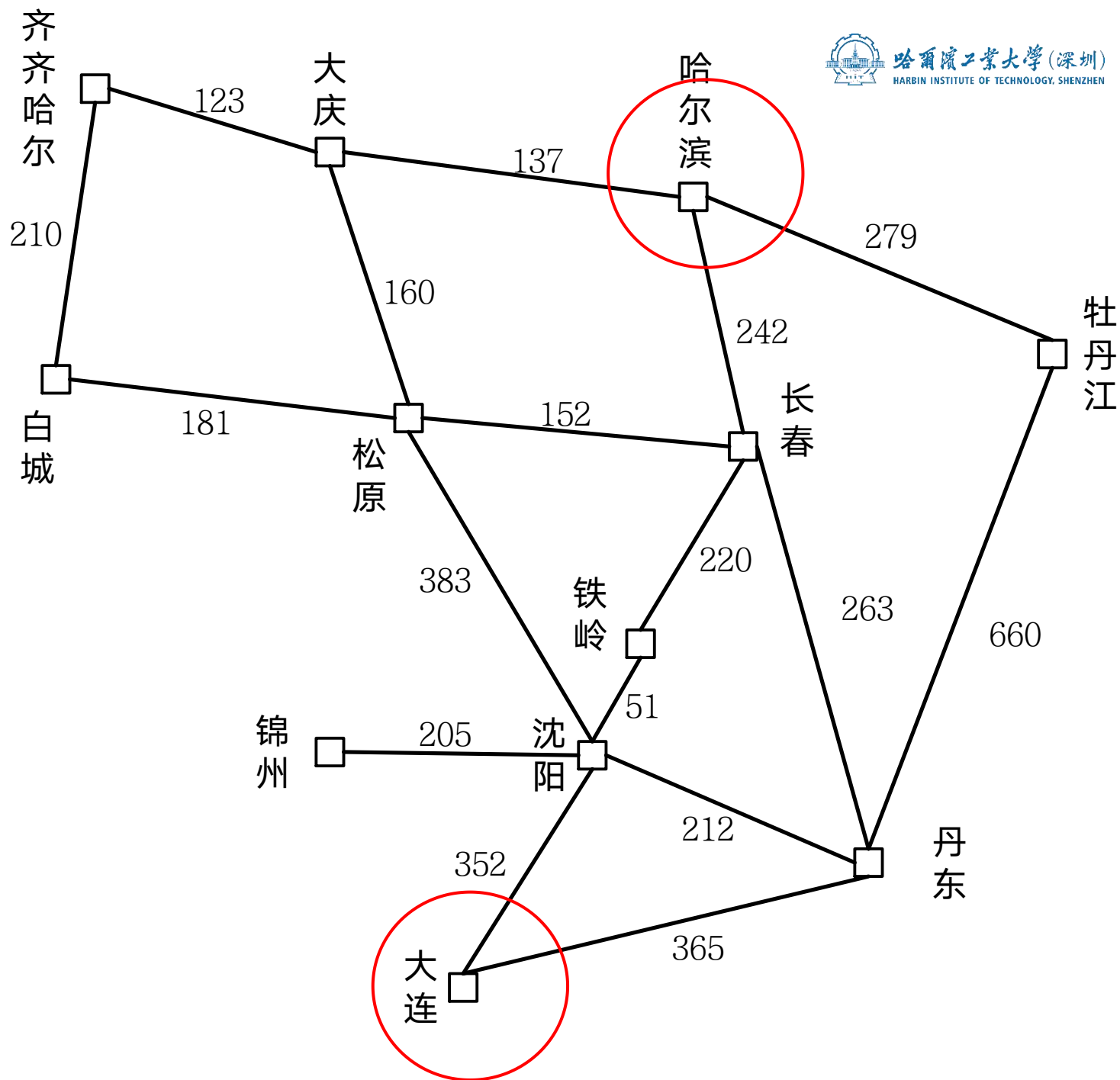
Greedy orders by goal proximity, or *forward cost* $h(n)$



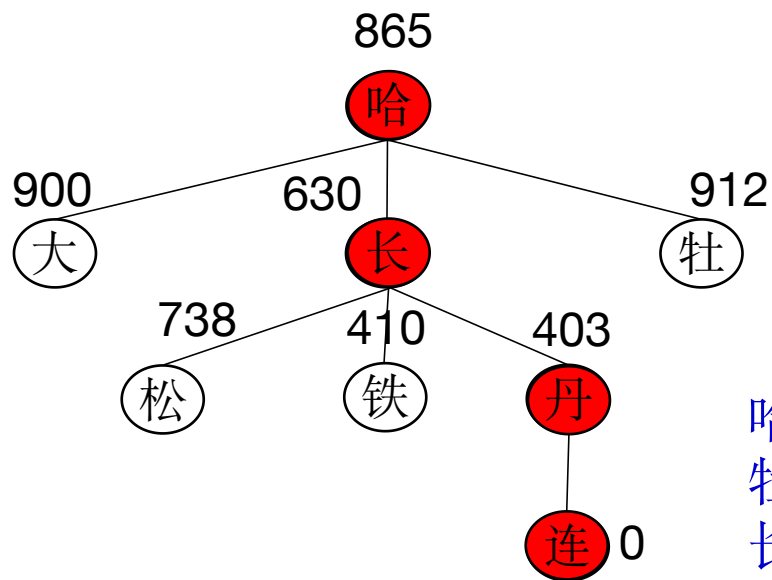
路径搜索问题：

启发式（估价）函数为各城市到达大连的直线距离：

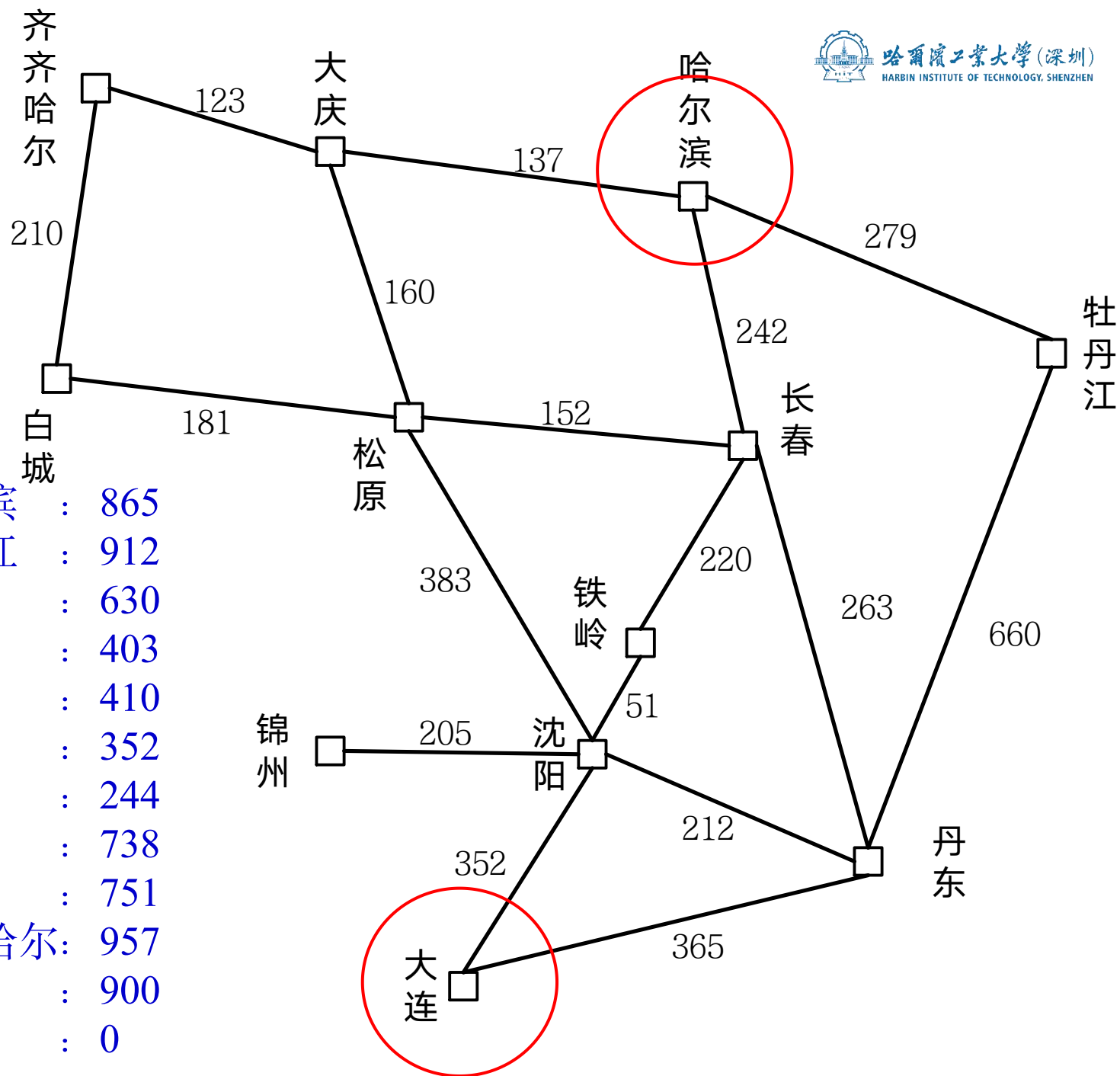
哈尔滨 : 865
牡丹江 : 912
长春 : 630
丹东 : 403
铁岭 : 410
沈阳 : 352
锦州 : 244
松原 : 738
白城 : 751
齐齐哈尔 : 957
大庆 : 900
大连 : 0



路径搜索问题:

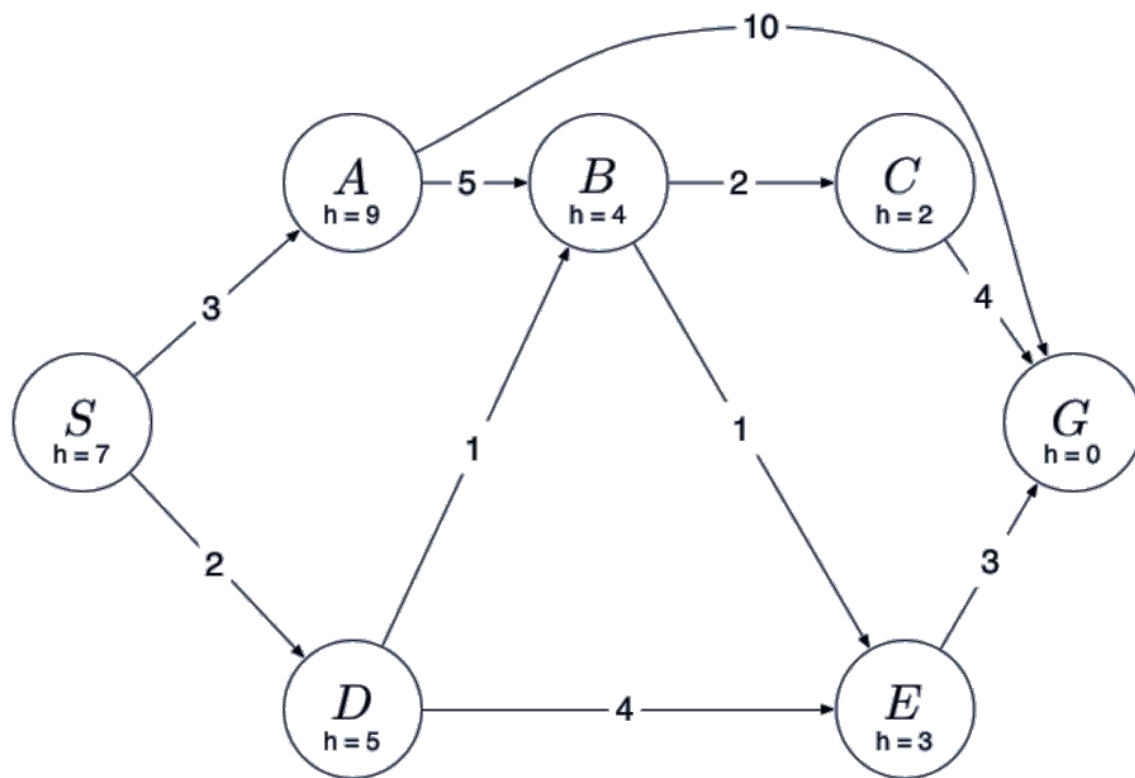


哈尔滨 : 865
牡丹江 : 912
长春 : 630
丹东 : 403
铁岭 : 410
沈阳 : 352
锦州 : 244
松原 : 738
白城 : 751
齐齐哈尔 : 957
大庆 : 900
大连 : 0



到大连的最短路径长度为
865 $(242+220+51+352)$,
贪心算法得到的路径长度为:
 $242+263+365=870$

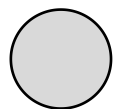
求下图所示从 S 到 G 的贪心(启发式)搜索树



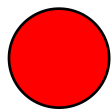
- 假设同一优先级扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展

作答

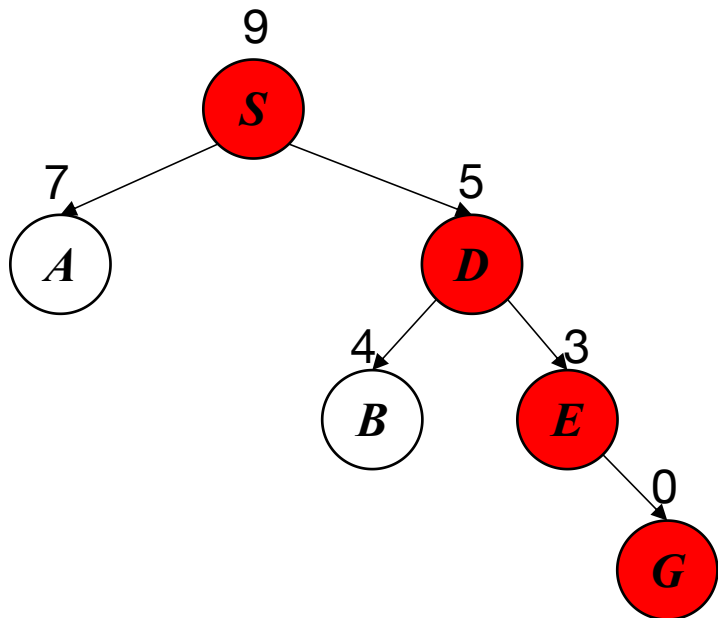
4.3.2 贪心算法



因为修改父节点指向删掉的节点

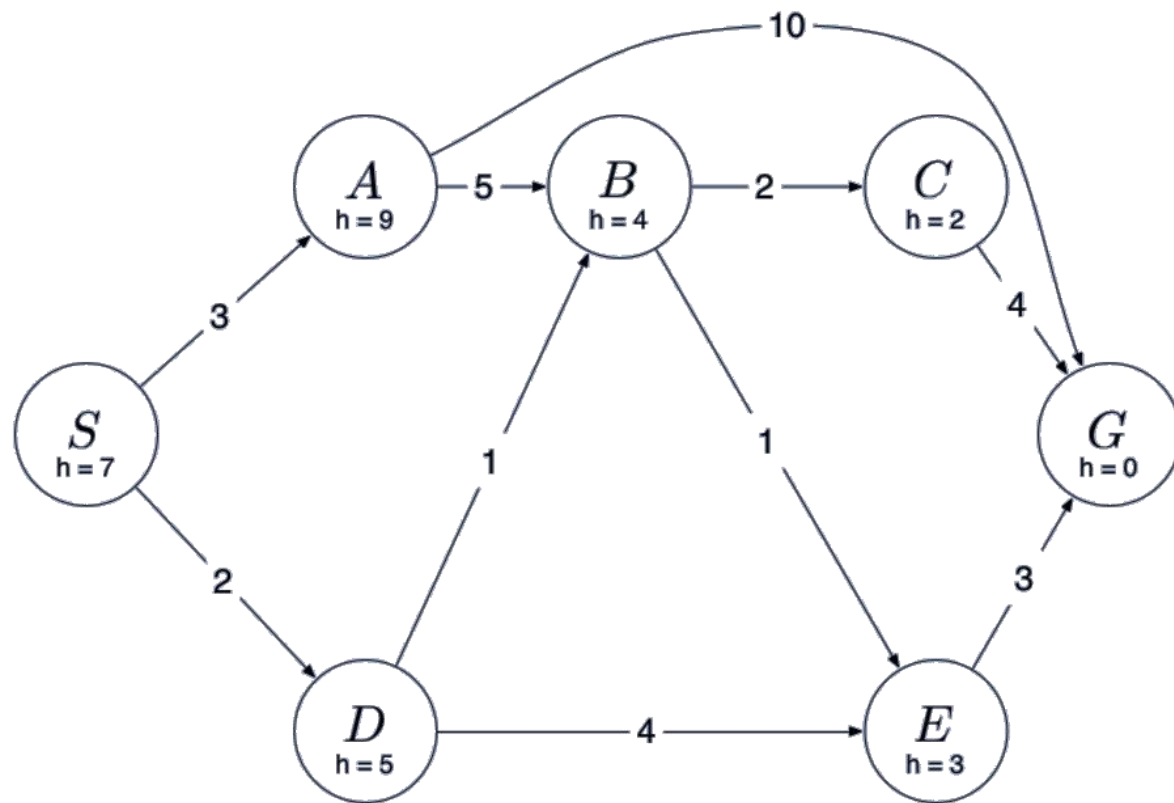


已扩展的节点



搜索顺序: $S \rightarrow D \rightarrow E \rightarrow G$

解: $S \rightarrow D \rightarrow E \rightarrow G$

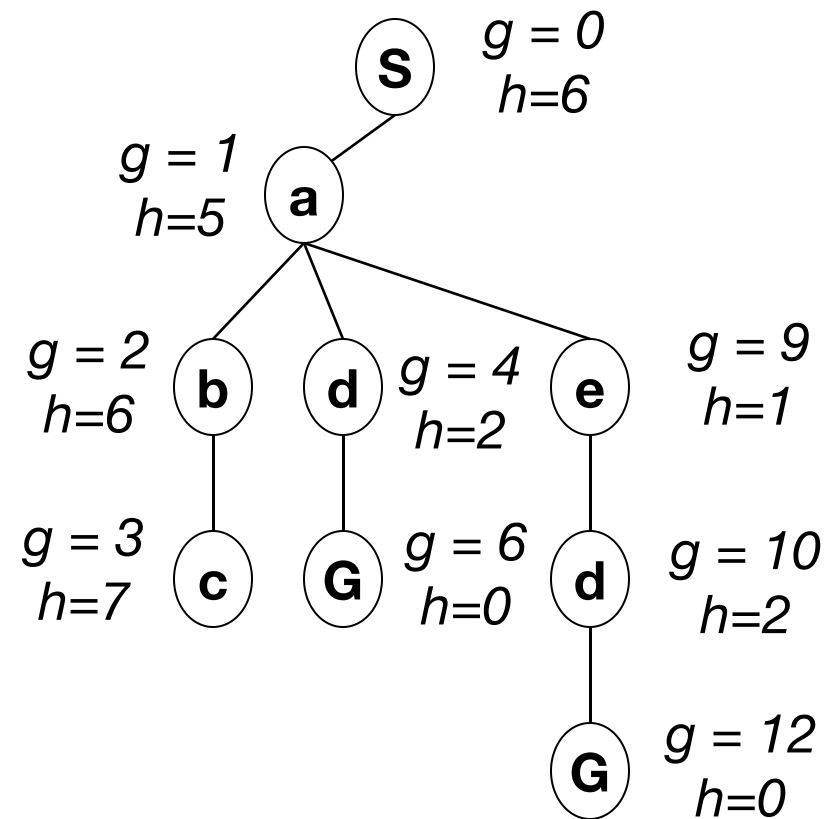
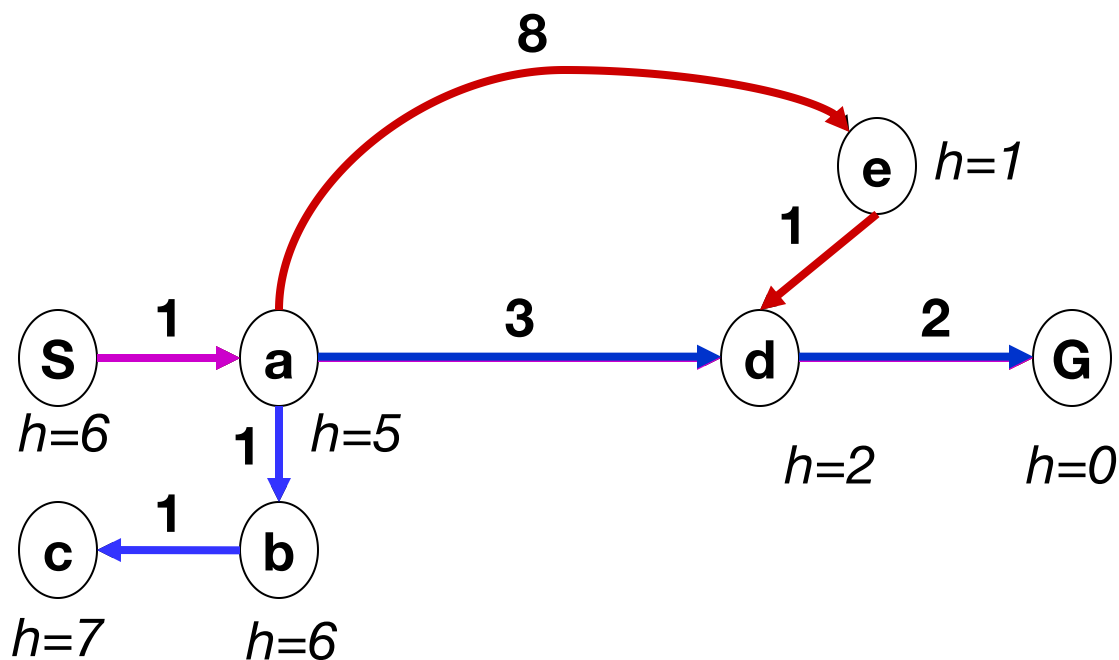


- 假设同一优先级扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展

代价一致 + 贪心

➤ Uniform-cost orders by path cost, or *backward cost* $g(n)$

➤ Greedy orders by goal proximity, or *forward cost* $h(n)$



➤ A Search orders by the sum: $f(n) = g(n) + h(n)$



UCS



Greedy



A^*

□ 估价函数

用来估计节点重要性，定义为从初始节点 S_0 出发，经过节点 n 到达目标节点 S_g 的所有路径中最小路径代价的估计值。一般形式：

$$f(n)=g(n)+h(n)$$

其中， $g(n)$ 是从初始节点 S_0 到节点 n 的实际代价； $h(n)$ 是从节点 n 到目标节点 S_g 的最优路径的估计代价。

4.3.3 A算法

□ **概念：** 在状态空间搜索中，如果每一步都利用估价函数 $f(n)=g(n)+h(n)$ 对**Open**表中的节点进行排序，则称**A**算法。它是一种为启发式搜索算法。

□ **类型：**

全局择优： 从**Open**表的所有节点中选择一个估价函数值最小的进行扩展。

局部择优： 仅从刚生成的子节点中选择一个估价函数值最小的进行扩展。

4.3.3 A算法

❑ **概念：** 在状态空间搜索中，如果每一步都利用估价函数 $f(n)=g(n)+h(n)$ 对**Open**表中的节点进行排序，则称**A**算法。它是一种为启发式搜索算法。

❑ **类型：**

全局择优： 从**Open**表的所有节点中选择一个估价函数值最小的进行扩展。

局部择优： 仅从刚生成的子节点中选择一个估价函数值最小的进行扩展。

❑ **全局择优搜索A算法描述：**

- (1)把初始节点 S_0 放入**Open**表中， $f(S_0)=g(S_0)+h(S_0)$;
- (2)如果**Open**表为空，则问题无解，失败退出;
- (3)把**Open**表的第一个节点取出放入**Closed**表，并记该节点为 n ;
- (4)考察节点 n 是否为目标节点。若是，则找到了问题的解，成功退出;
- (5)若节点 n 不可扩展，则转第(2)步;
- (6)扩展节点 n ，生成其子节点 $n_i (i=1, 2, \dots)$ ，计算每一个子节点的估价值 $f(n_i) (i=1, 2, \dots)$ ，并为每一个子节点设置指向父节点的指针，然后将这些子节点放入**Open**表中;
- (7)根据各节点的估价函数值，对**Open**表中的全部节点按从小到大的顺序重新进行排序;
- (8)转第(2)步。

- (1) 把初始节点 S_0 放入Open表, 设 $g(s_0)=0$, 建立一个CLOSED表, 置为空;
- (2) 检查Open表是否为空表, 若为空, 则问题无解, 失败退出;
- (3) 把Open表的第一个节点取出放入Closed表, 并记该节点为 n ;
- (4) 考察节点 n 是否为目标节点, 若是则得到问题的解成功退出;
- (5) 若节点 n 不可扩展, 则转第(2)步;
- (6) 扩展节点 n , 生成子节点 n_i ($i=1,2,\dots$), 将其子节点放入Open表, 并为Open表和Close中相关节点设置指向父节点的指针 (修改指针指向), 将Open表内的节点按约定的规则进行排序 (重新排序), 转向第(2)步。

广度 (宽度) 优先、深度优先、代价一致搜索、贪心搜索、A搜索=代价一致+贪心

Open表中的几点进行排序

广度（宽度）优先：**level-by-level order**

深度优先：**first order**

代价一致搜索： **$f(n)=g(n)+c(n1,n2)$ order**

贪心搜索： **$f(n)=g(n)$ order**

搜索=代价一致+贪心： **$f(n)=g(n) +h(n)$ order**

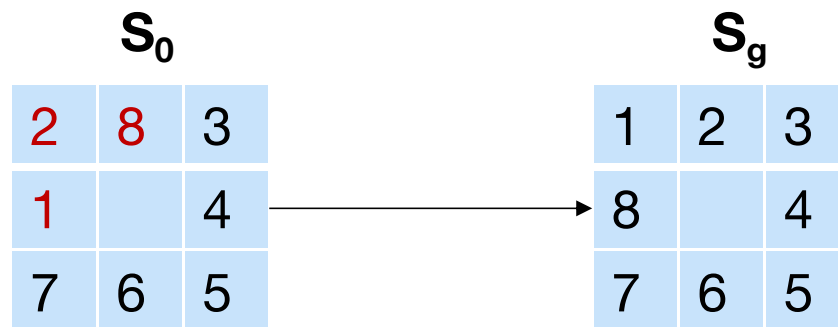
4.3.3 A算法

例4.8八数码难题。设问题的初始状态 S_0 和目标状态 S_g 如下图所示，且估价函数为

$$f(n)=d(n)+W(n)$$

其中： $d(n)$ 表示节点 n 在搜索树中的深度

$W(n)$ 表示节点 n 中“不在位”的数码个数。请用全局择优搜索解决这个问题。

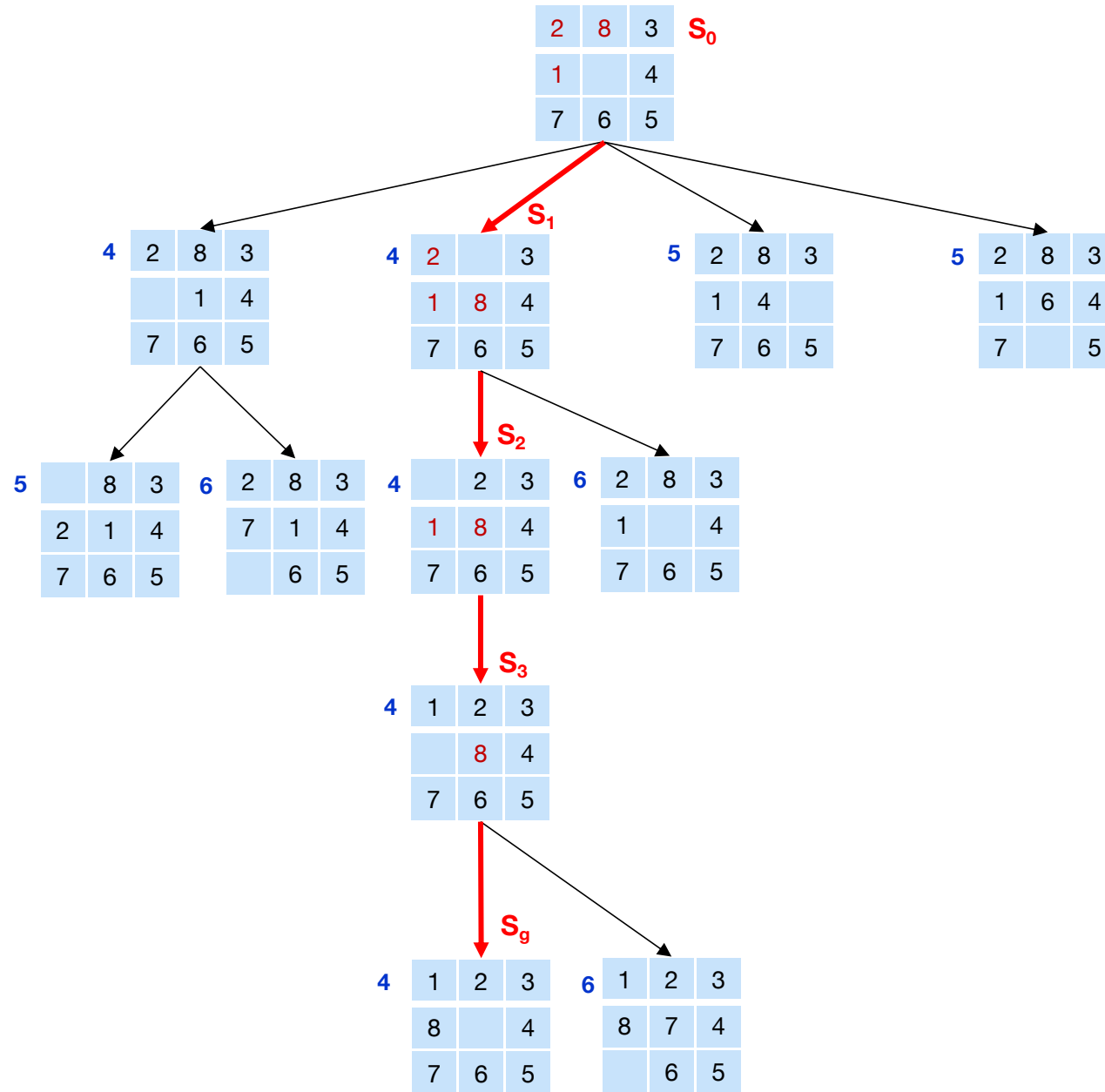


4.3.3 A算法

解：该问题的全局择优搜索树如下图所示。在该图中，每个节点旁边的数字是该节点的估价函数值。

例如，对节点 S_2 ，其估价函数值的计算为：

$$\begin{aligned} f(S_2) &= d(S_2) + W(S_2) \\ &= 2 + 2 = 4 \end{aligned}$$

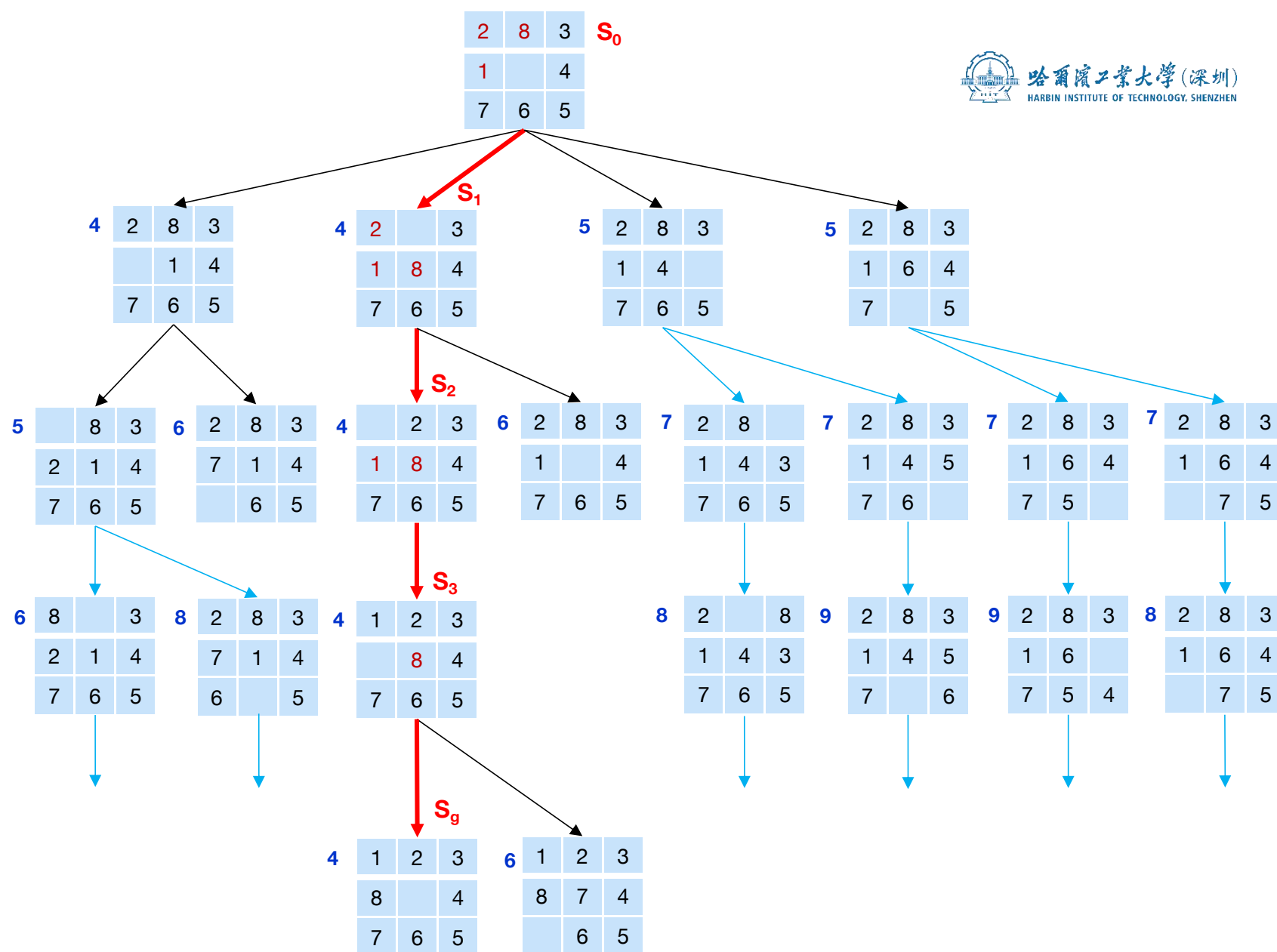


最优路径解: $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_g$

4.3.3 A算法

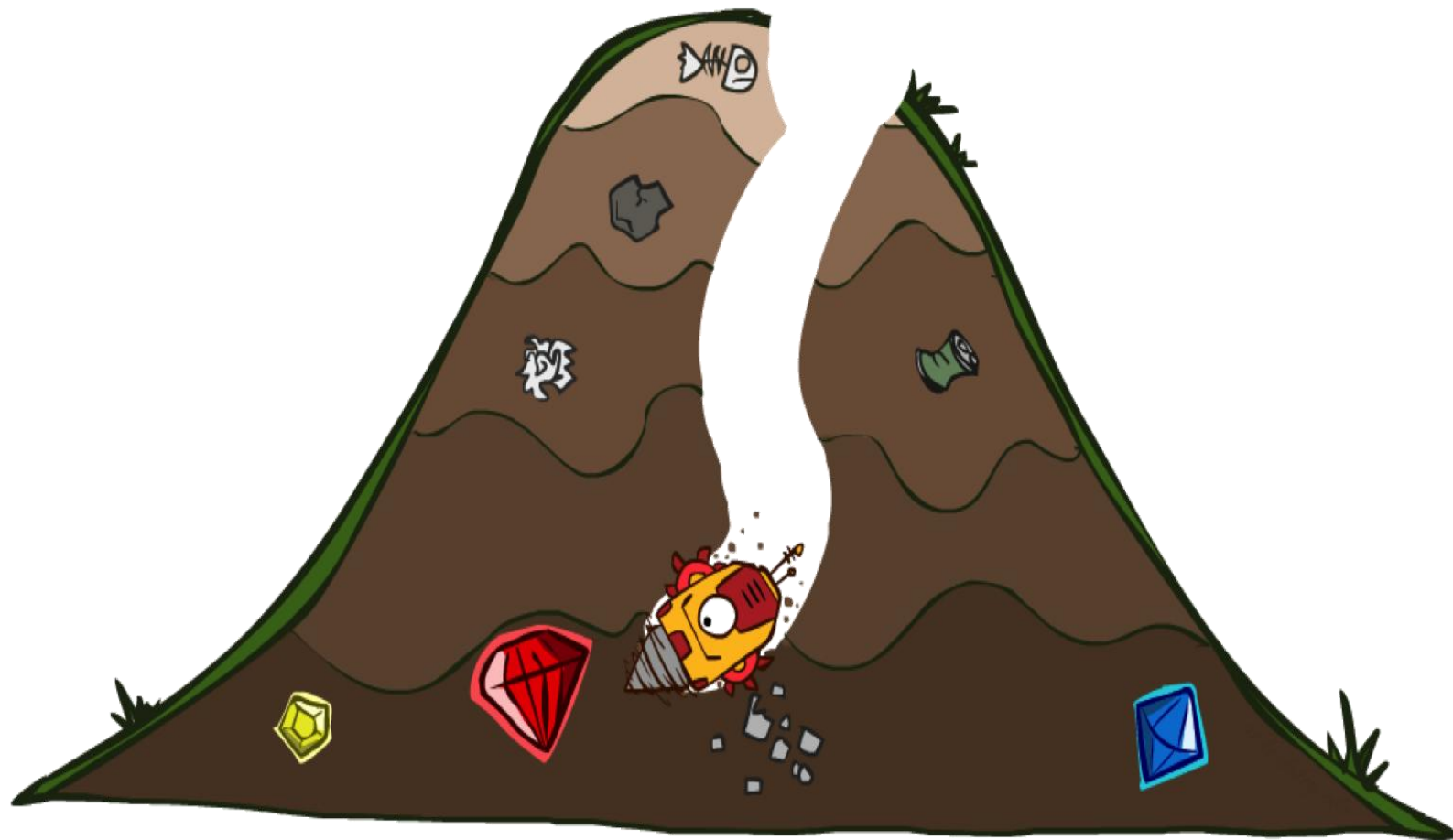
解：该问题的全局择优搜索树如下图所示。在该图中，每个节点旁边的数字是该节点的估价函数值。

例如，对节点 S_2 ，其估价函数值的计算为：
 $f(S_2)=d(S_2)+W(S_2)$
 $=2+2=4$



最优路径解: $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_g$

4.3.4 A*算法：缩小总评估代价



4.3.4 A*算法

A*算法是对A算法的估价函数 $f(n)=g(n)+h(n)$ 加上某些限制后得到的一种启发式搜索算法。

假设 $f^*(n)$ 是从初始节点 S_0 出发，约束经过节点 n 到达目标节点 S_g 的最小代价，估价函数 $f(n)$ 是对 $f^*(n)$ 的估计值。记

$$f^*(n)=g^*(n)+h^*(n)$$

其中， $g^*(n)$ 是从 S_0 出发到达 n 的最小代价, $h^*(n)$ 是 n 到 S_g 的最小代价

4.3.4 A*算法

A*算法是对A算法的估价函数 $f(n)=g(n)+h(n)$ 加上某些限制后得到的一种启发式搜索算法。

假设 $f^*(n)$ 是从初始节点 S_0 出发，约束经过节点 n 到达目标节点 S_g 的最小代价，估价函数 $f(n)$ 是对 $f^*(n)$ 的估计值。记

$$f^*(n)=g^*(n)+h^*(n)$$

其中， $g^*(n)$ 是从 S_0 出发到达 n 的最小代价， $h^*(n)$ 是 n 到 S_g 的最小代价

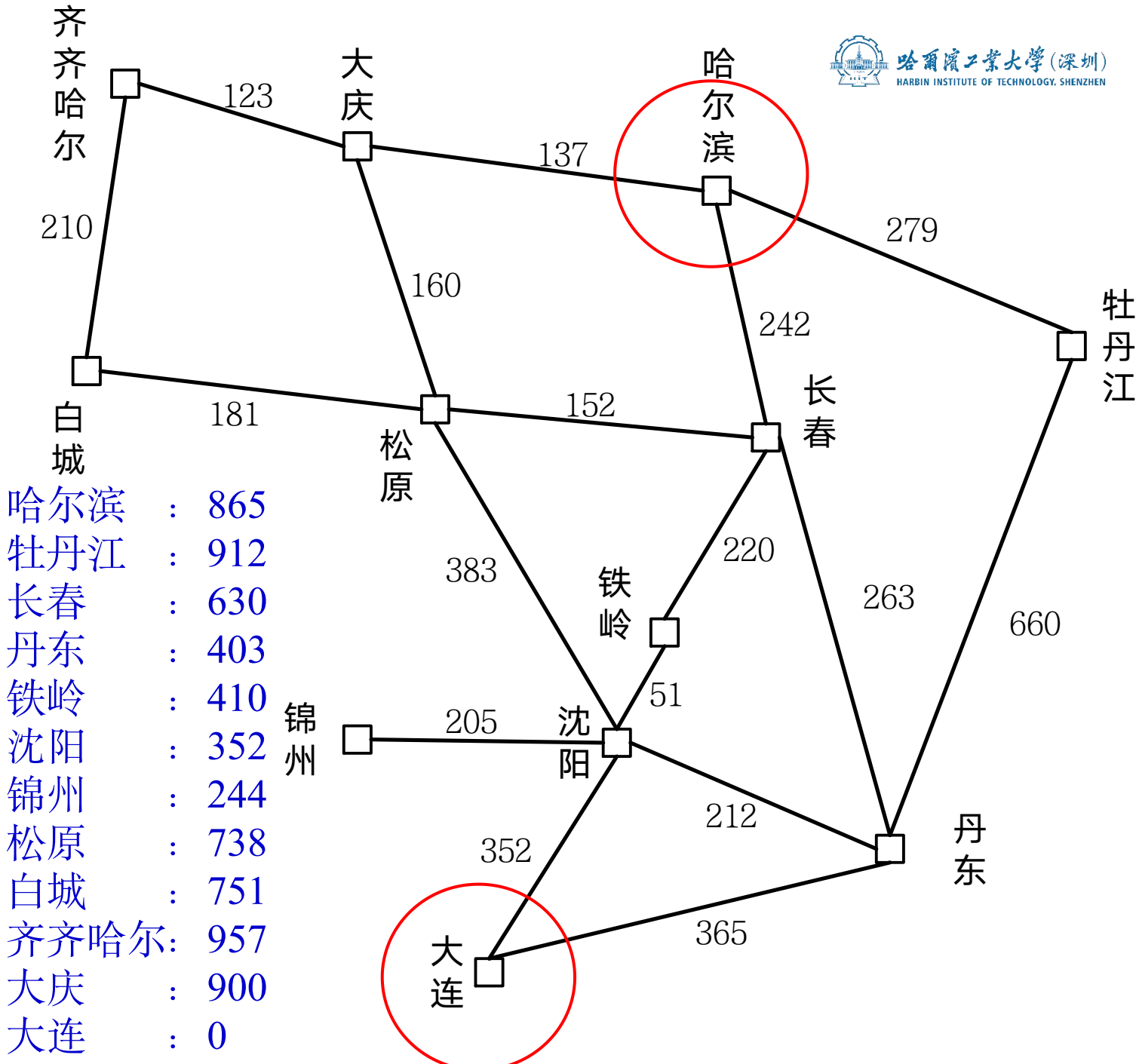
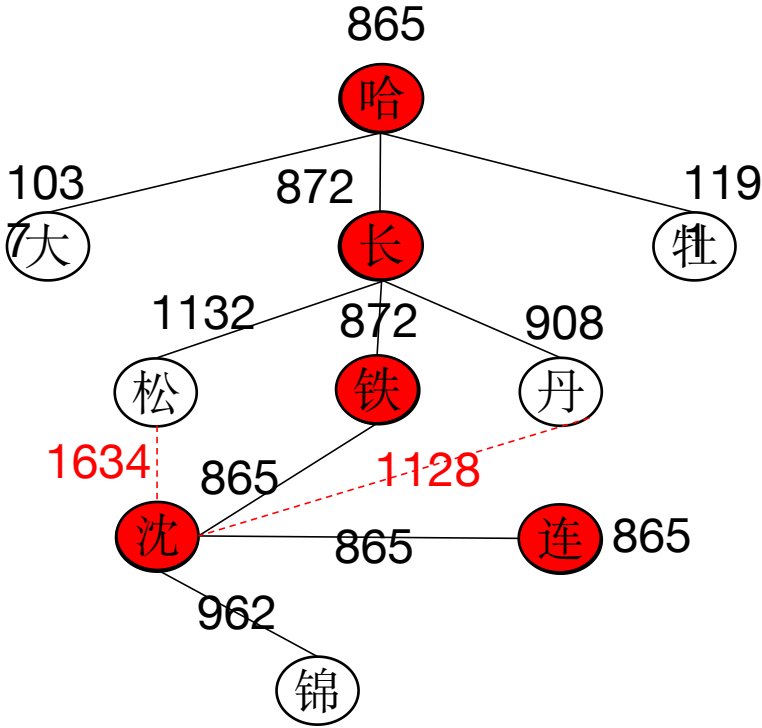
定义4.1 如果对A算法（全局择优）中的 $g(n)$ 和 $h(n)$ 分别提出如下限制：

第一， $g(n)$ 是对最小代价 $g^*(n)$ 的估计，且 $g(n)>0$ ；

第二， $h(n)$ 是最小代价 $h^*(n)$ 的下界，即对任意节点 n 均有 $h(n)\leq h^*(n)$ 。

则称满足上述两条限制的A算法为A*算法。

路径搜索问题:



对比



Greedy

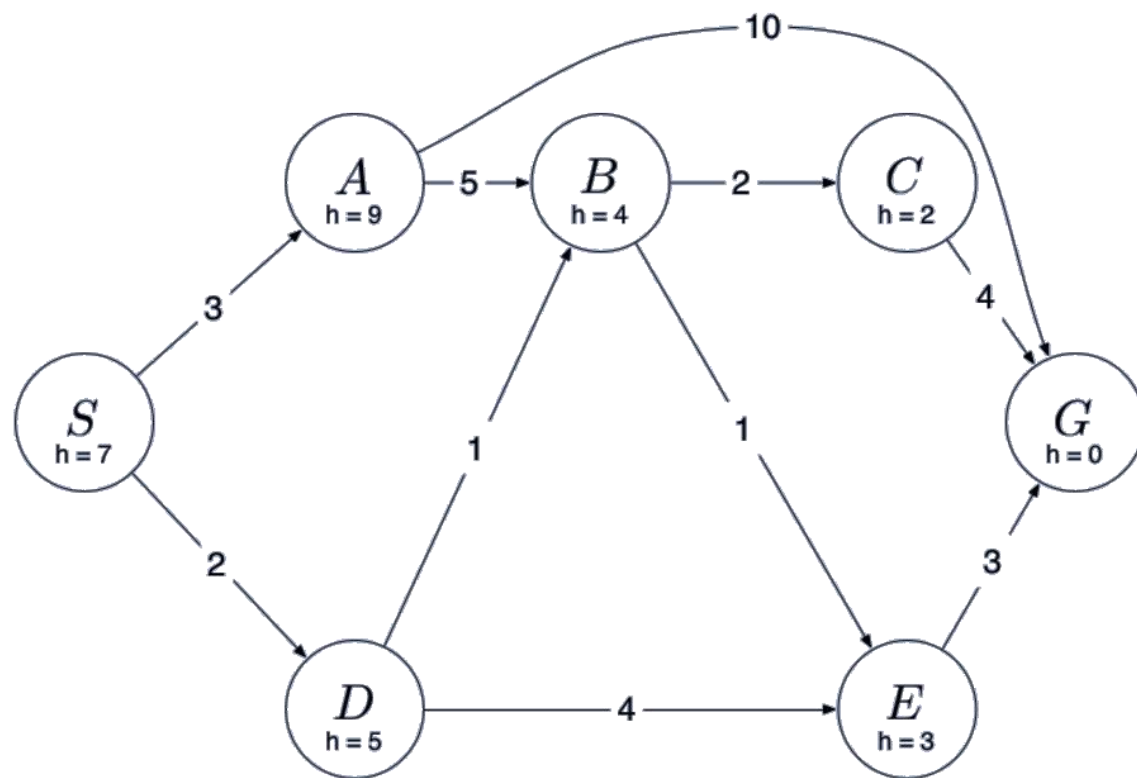


Uniform Cost



A^*

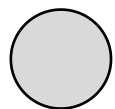
求下图所示从 S 到 G 的A*搜索树



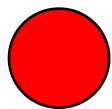
- 假设同一优先级扩展节点按字母顺序，即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展

作答

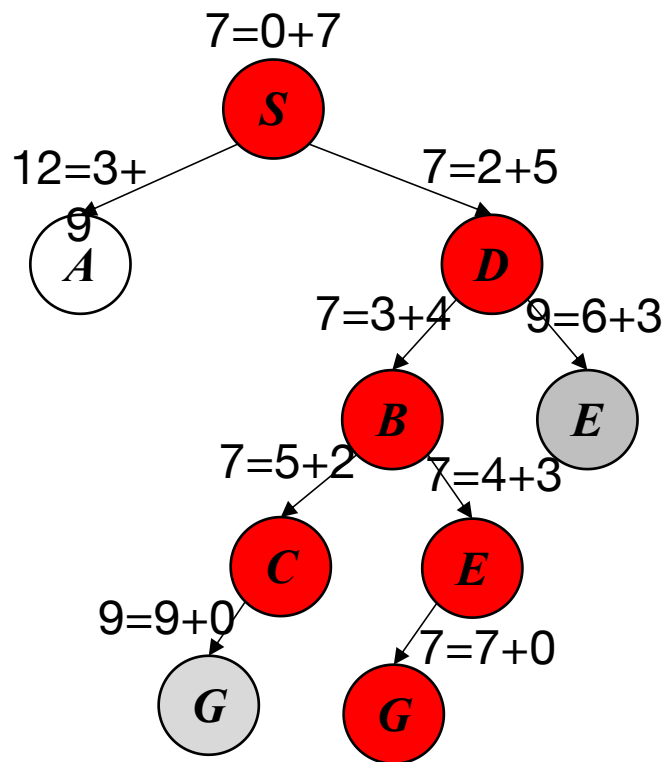
4.3.4 A*算法



因为修改父节点指向删掉的节点

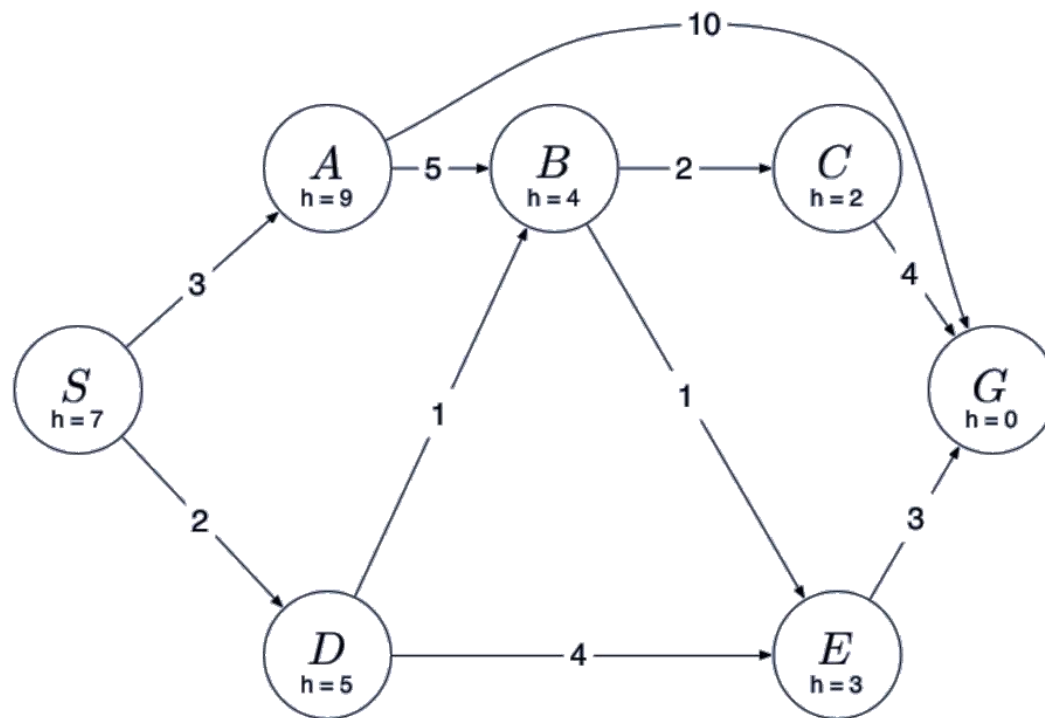


已扩展的节点



搜索顺序: $S \rightarrow D \rightarrow B \rightarrow C \rightarrow E \rightarrow G$

解: $S \rightarrow D \rightarrow B \rightarrow E \rightarrow G$



- 假设同一优先级扩展节点按字母顺序, 即 $S \rightarrow A$ 优先于 $S \rightarrow D$ 扩展

4.3.4.1 A*算法的可纳性

□ 可纳性的含义:

定义4.2 对任一状态空间图，当从初始节点到目标节点有路经存在时，如果搜索算法总能在有限步骤内找到一条从初始节点到目标节点的最佳路径，并在此路径上结束，则称该搜索算法是可采纳的。

A*算法可纳性的证明过程可分以下三步:

第一步，对有限图，A*算法一定能够成功结束。

第二步，对无限图，A*算法也一定能够成功结束。

第三步，A*算法一定能够结束在最佳路径上。

4.3.4.1 A*算法的可纳性

□ 可纳性的证明:

定理4.1 对有限图, 如果从初始节点 S_0 到目标节点 S_g 有路径存在, 则算法A*一定成功结束。

4.3.4.1 A*算法的可纳性

□ 可纳性的证明:

定理4.1 对有限图, 如果从初始节点 S_0 到目标节点 S_g 有路径存在, 则算法A*一定成功结束。

证明: 首先证明算法必然会结束。

由于搜索图为有限图, 如果算法能找到解, 则成功结束; 如果算法找不到解, 则必然会由于**Open**表变空而结束。因此, A*算法必然会结束。

4.3.4.1 A*算法的可纳性

□ 可纳性的证明:

定理4.1 对有限图, 如果从初始节点 S_0 到目标节点 S_g 有路径存在, 则算法A*一定成功结束。

证明: 首先证明算法必然会结束。

由于搜索图为有限图, 如果算法能找到解, 则成功结束; 如果算法找不到解, 则必然会由于**Open**表变空而结束。因此, A*算法必然会结束。

然后证明算法一定会成功结束。

由于至少存在一条有初始节点到目标节点的路径, 设此路径为
 $S_0 = n_0, n_1, \dots, n_k = S_g$

4.3.4.1 A*算法的可纳性

□ 可纳性的证明:

定理4.1 对有限图, 如果从初始节点 S_0 到目标节点 S_g 有路径存在, 则算法A*一定成功结束。

证明: 首先证明算法必然会结束。

由于搜索图为有限图, 如果算法能找到解, 则成功结束; 如果算法找不到解, 则必然会由于**Open**表变空而结束。因此, A*算法必然会结束。

然后证明算法一定会成功结束。

由于至少存在一条有初始节点到目标节点的路径, 设此路径为

$$S_0 = n_0, n_1, \dots, n_k = S_g$$

算法开始时, 节点 n_0 在**Open**表中, 且路径中任一节点 n_i 离开**Open**表后, 其后继节点 n_{i+1} 必然进入**Open**表, 这样, 在**Open**表变为空之前, 目标节点必然出现在**Open**表中。因此, 算法一定会成功结束。

4.3.4.1 A*算法的可纳性

引理4.1 对无限图，如果从初始节点 S_0 到目标节点 S_g 有路径存在，则算法A*算法不终止的话，则从**Open**表中选出的节点必将具有任意大的**f**值。

4.3.4.1 A*算法的可纳性

引理4.1 对无限图，如果从初始节点 S_0 到目标节点 S_g 有路径存在，则算法A*算法不终止的话，则从**Open**表中选出的节点必将具有任意大的**f**值。

证明： 设 $d^*(n)$ 是A*生成的从初始节点 S_0 到节点 n 的最短路径长度，由于搜索图中每条边的代价都是一个正数，令其中的最小的一个数是 e ，则有

$$g^*(n) \geq d^*(n) \times e$$

4.3.4.1 A*算法的可纳性

引理4.1 对无限图，如果从初始节点 S_0 到目标节点 S_g 有路径存在，则算法A*算法不终止的话，则从**Open**表中选出的节点必将具有任意大的**f**值。

证明： 设 $d^*(n)$ 是A*生成的从初始节点 S_0 到节点 n 的最短路径长度，由于搜索图中每条边的代价都是一个正数，令其中的最小的一个数是 e ，则有

$$g^*(n) \geq d^*(n) \times e$$

因为 $g^*(n)$ 是最佳路径的代价，故有

$$g(n) \geq g^*(n) \geq d^*(n) \times e$$

4.3.4.1 A*算法的可纳性

引理4.1 对无限图，如果从初始节点 S_0 到目标节点 S_g 有路径存在，则算法A*算法不终止的话，则从**Open**表中选出的节点必将具有任意大的**f**值。

证明： 设 $d^*(n)$ 是A*生成的从初始节点 S_0 到节点 n 的最短路径长度，由于搜索图中每条边的代价都是一个正数，令其中的最小的一个数是 e ，则有

$$g^*(n) \geq d^*(n) \times e$$

因为 $g^*(n)$ 是最佳路径的代价，故有

$$g(n) \geq g^*(n) \geq d^*(n) \times e$$

又因为 $h(n) \geq 0$ ，故有

$$f(n) = g(n) + h(n) \geq g(n) \geq d^*(n) \times e$$

4.3.4.1 A*算法的可纳性

引理4.1 对无限图，如果从初始节点 S_0 到目标节点 S_g 有路径存在，则算法A*算法不终止的话，则从**Open**表中选出的节点必将具有任意大的**f**值。

证明： 设 $d^*(n)$ 是A*生成的从初始节点 S_0 到节点 n 的最短路径长度，由于搜索图中每条边的代价都是一个正数，令其中的最小的一个数是 e ，则有

$$g^*(n) \geq d^*(n) \times e$$

因为 $g^*(n)$ 是最佳路径的代价，故有

$$g(n) \geq g^*(n) \geq d^*(n) \times e$$

又因为 $h(n) \geq 0$ ，故有

$$f(n) = g(n) + h(n) \geq g(n) \geq d^*(n) \times e$$

如果A*算法不终止的话，从**Open**表中选出的节点必将具有任意大的 $d^*(n)$ 值，因此，也将具有任意大的**f**值。

4.3.4.1 A*算法的可纳性

引理4.2 在A*算法终止前的任何时刻, **Open**表中总存在节点 n' , 它是从初始节点 S_0 到目标节点的最佳路径上的一个节点, 且满足 $f(n') \leq f^*(S_0)$ 。

4.3.4.1 A*算法的可纳性

引理4.2 在A*算法终止前的任何时刻, **Open**表中总存在节点 n' , 它是从初始节点 S_0 到目标节点的最佳路径上的一个节点, 且满足 $f(n') \leq f^*(S_0)$ 。

证明: 设从初始节点 S_0 到目标节点 t 的一条最佳路径序列为

$$S_0 = n_0, n_1, \dots, n_k = S_g$$

算法开始时, 节点 S_0 在**Open**表中, 当节点 S_0 离开**Open**表进入**Closed**表时, 节点 n_1 进入**Open**表。因此, A*没有结束以前, 在**Open**表中必存在最佳路径上的节点。

4.3.4.1 A*算法的可纳性

引理4.2 在A*算法终止前的任何时刻，**Open**表中总存在节点 n' ，它是从初始节点 S_0 到目标节点的最佳路径上的一个节点，且满足 $f(n') \leq f^*(S_0)$ 。

证明： 设从初始节点 S_0 到目标节点 t 的一条最佳路径序列为

$$S_0 = n_0, n_1, \dots, n_k = S_g$$

算法开始时，节点 S_0 在**Open**表中，当节点 S_0 离开**Open**表进入**Closed**表时，节点 n_1 进入**Open**表。因此，A*没有结束以前，在**Open**表中必存在最佳路径上的节点。

设这些节点中排在最前面的节点为 n' ，则有

$$f(n') = g(n') + h(n')$$

由于 n' 在最佳路径上，故有 $g(n') = g^*(n')$ ，从而

$$f(n') = g^*(n') + h(n')$$

4.3.4.1 A*算法的可纳性

引理4.2 在A*算法终止前的任何时刻，**Open**表中总存在节点 n' ，它是从初始节点 S_0 到目标节点的最佳路径上的一个节点，且满足 $f(n') \leq f^*(S_0)$ 。

证明： 设从初始节点 S_0 到目标节点 t 的一条最佳路径序列为

$$S_0 = n_0, n_1, \dots, n_k = S_g$$

算法开始时，节点 S_0 在**Open**表中，当节点 S_0 离开**Open**表进入**Closed**表时，节点 n_1 进入**Open**表。因此，A*没有结束以前，在**Open**表中必存在最佳路径上的节点。

设这些节点中排在最前面的节点为 n' ，则有

$$f(n') = g(n') + h(n')$$

由于 n' 在最佳路径上，故有 $g(n') = g^*(n')$ ，从而

$$f(n') = g^*(n') + h(n')$$

又由于A*算法满足 $h(n') \leq h^*(n')$ ，故有

$$f(n') \leq g^*(n') + h^*(n') = f^*(n')$$

因为在最佳路径上的所有节点的 f^* 值都应相等，因此有

$$f(n') \leq f^*(S_0)$$

4.3.4.1 A*算法的可纳性

定理4.2 对无限图，若从初始节点 S_0 到目标节点 S_g 有路径存在，则A*算法必然会结束。

证明：（反证法）假设A*不结束，由引理4.1知Open表中的节点有任意大的 f 值，这与引理4.2的结论相矛盾，因此，A*算法只能成功结束。

推论4.1 Open表中任一具有 $f(n) < f^*(S_0)$ 的节点 n ，最终都被A* 算法选作为扩展的节点。

(证明略)

下面给出A*算法的可纳性

4.3.4.1 A*算法的可纳性

定理4.3 A*算法是可采纳的，即若存在从初始节点 S_0 到目标节点 S_g 的路径，则A*算法必能结束在最佳路径上。

4.3.4.1 A*算法的可纳性

定理4.3 A*算法是可采纳的，即若存在从初始节点 S_0 到目标节点 S_g 的路径，则A*算法必能结束在最佳路径上。

证明：证明过程分以下两步进行：

先证明A*算法一定能够终止在某个目标节点上。

由定理4.1和定理4.2可知，无论是对有限图还是无限图，A*算法都能够找到某个目标节点而结束。

4.3.4.1 A*算法的可纳性

定理4.3 A*算法是可采纳的，即若存在从初始节点 S_0 到目标节点 S_g 的路径，则A*算法必能结束在最佳路径上。

证明：证明过程分以下两步进行：

先证明A*算法一定能够终止在某个目标节点上。

由定理4.1和定理4.2可知，无论是对有限图还是无限图，A*算法都能够找到某个目标节点而结束。

再证明A*算法只能终止在最佳路径上。（反证法）

假设A*算法未能终止在最佳路径上，而是终止在某个目标节点 t 处，则有

$$f(t)=g(t)>f^*(S_0)$$

但由引理4.2可知，在A*算法结束前，必有最佳路径上的一个节点 n' 在Open表中，且有

$$f(n')\leq f^*(S_0)<f(t)$$

4.3.4.1 A*算法的可纳性

定理4.3 A*算法是可采纳的，即若存在从初始节点 S_0 到目标节点 S_g 的路径，则A*算法必能结束在最佳路径上。

证明：证明过程分以下两步进行：

先证明A*算法一定能够终止在某个目标节点上。

由定理4.1和定理4.2可知，无论是对有限图还是无限图，A*算法都能够找到某个目标节点而结束。

再证明A*算法只能终止在最佳路径上。（反证法）

假设A*算法未能终止在最佳路径上，而是终止在某个目标节点 t 处，则有

$$f(t)=g(t)>f^*(S_0)$$

但由引理4.2可知，在A*算法结束前，必有最佳路径上的一个节点 n' 在Open表中，且有

$$f(n')\leq f^*(S_0)<f(t)$$

这时，A*算法一定会选择 n' 来扩展，而不可能选择 t ，从而也不会去测试目标节点 t ，这就与假设A*算法终止在目标节点 t 相矛盾。因此，A*算法只能终止在最佳路径上

4.3.4.1 A*算法的可纳性

推论4.2 在A*算法中, 对任何被扩展的节点 n , 都有 $f(n) \leq f^*(S_0)$ 。

4.3.4.1 A*算法的可纳性

推论4.2 在A*算法中, 对任何被扩展的节点 n , 都有 $f(n) \leq f^*(S_0)$ 。

证明: 令 n 是由A*选作扩展的任一节点, 因此 n 不会为目标节点, 且搜索没有结束。

由引理4.2可知, 在Open表中有满足

$$f(n') \leq f^*(S_0)$$

的节点 n' 。

4.3.4.1 A*算法的可纳性

推论4.2 在A*算法中, 对任何被扩展的节点 n , 都有 $f(n) \leq f^*(S_0)$ 。

证明: 令 n 是由A*选作扩展的任一节点, 因此 n 不会为目标节点, 且搜索没有结束。

由引理4.2可知, 在Open表中有满足

$$f(n') \leq f^*(S_0)$$

的节点 n' 。

若 $n=n'$, 则有 $f(n) \leq f^*(S_0)$; 否则, 选择 n 扩展 (选择 f 较小的节点扩展), 必有

$$f(n) \leq f(n')$$

4.3.4.1 A*算法的可纳性

推论4.2 在A*算法中, 对任何被扩展的节点 n , 都有 $f(n) \leq f^*(S_0)$ 。

证明: 令 n 是由A*选作扩展的任一节点, 因此 n 不会为目标节点, 且搜索没有结束。

由引理4.2可知, 在Open表中有满足

$$f(n') \leq f^*(S_0)$$

的节点 n' 。

若 $n=n'$, 则有 $f(n) \leq f^*(S_0)$; 否则, 选择 n 扩展 (选择 f 较小的节点扩展), 必有

$$f(n) \leq f(n')$$

所以有

$$f(n) \leq f^*(S_0)$$

4.3.4.2 A*算法的最优性

A*算法的搜索效率很大程度上取决于估价函数 $h(n)$ 。一般来说，在满足 $h(n) \leq h^*(n)$ 的前提下， $h(n)$ 的值越大越好。 $h(n)$ 的值越大，说明它携带的启发性信息越多，A*算法搜索时扩展的节点就越少，搜索效率就越高。A*算法的这一特性被称为最优性。

4.3.4.2 A*算法的最优性

A*算法的搜索效率很大程度上取决于估价函数 $h(n)$ 。一般来说，在满足 $h(n) \leq h^*(n)$ 的前提下， $h(n)$ 的值越大越好。 $h(n)$ 的值越大，说明它携带的启发性信息越多，A*算法搜索时扩展的节点就越少，搜索效率就越高。A*算法的这一特性被称为最优性。

下面通过一个定理来描述这一特性。

定理4.4 设有两个A*算法 A_1^* 和 A_2^* ，它们有

$$A_1^*: f_1(n) = g_1(n) + h_1(n)$$

$$A_2^*: f_2(n) = g_2(n) + h_2(n)$$

如果 A_2^* 比 A_1^* 有更多的启发性信息，即对所有非目标节点均有

$$h_2(n) > h_1(n)$$

则在搜索过程中，被 A_2^* 扩展的节点也必然被 A_1^* 扩展，即 A_1^* 扩展的节点不会比 A_2^* 扩展的节点少，亦即 A_2^* 扩展的节点集是 A_1^* 扩展的节点集的子集。

4.3.4.2 A*算法的最优性

例 八数码难题。设问题的初始状态 S_0 和目标状态 S_g 如下图所示，且估价函数为
$$f(n)=d(n)+K(n)$$

其中： $d(n)$ 表示节点 n 在搜索树中的深度； $K(n)$ 表示不在位数码回原位需要的步数。

S_0 $K(n) = 4 = 1 + 1 + 2$

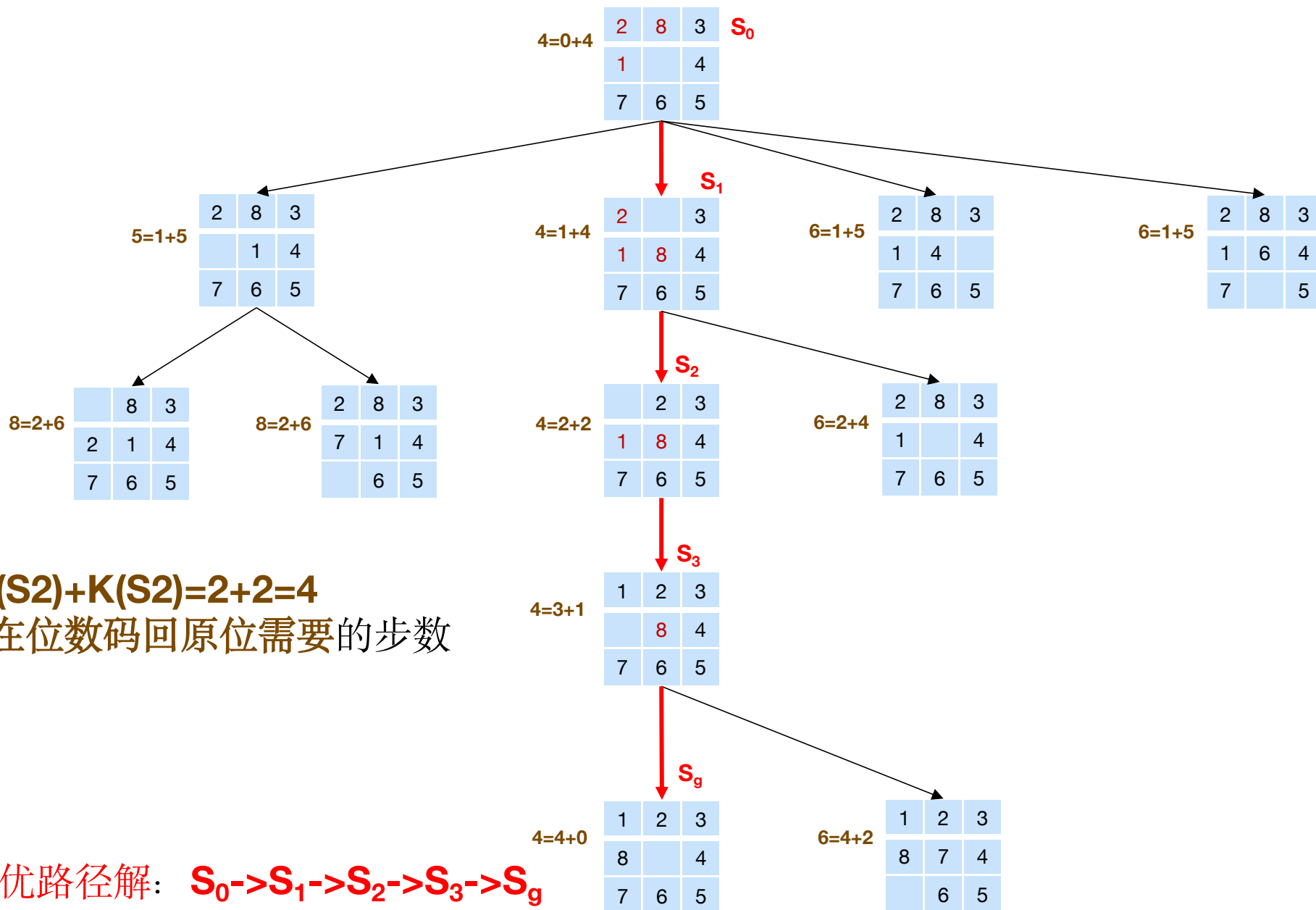
2	8	3
1		4
7	6	5

S_g

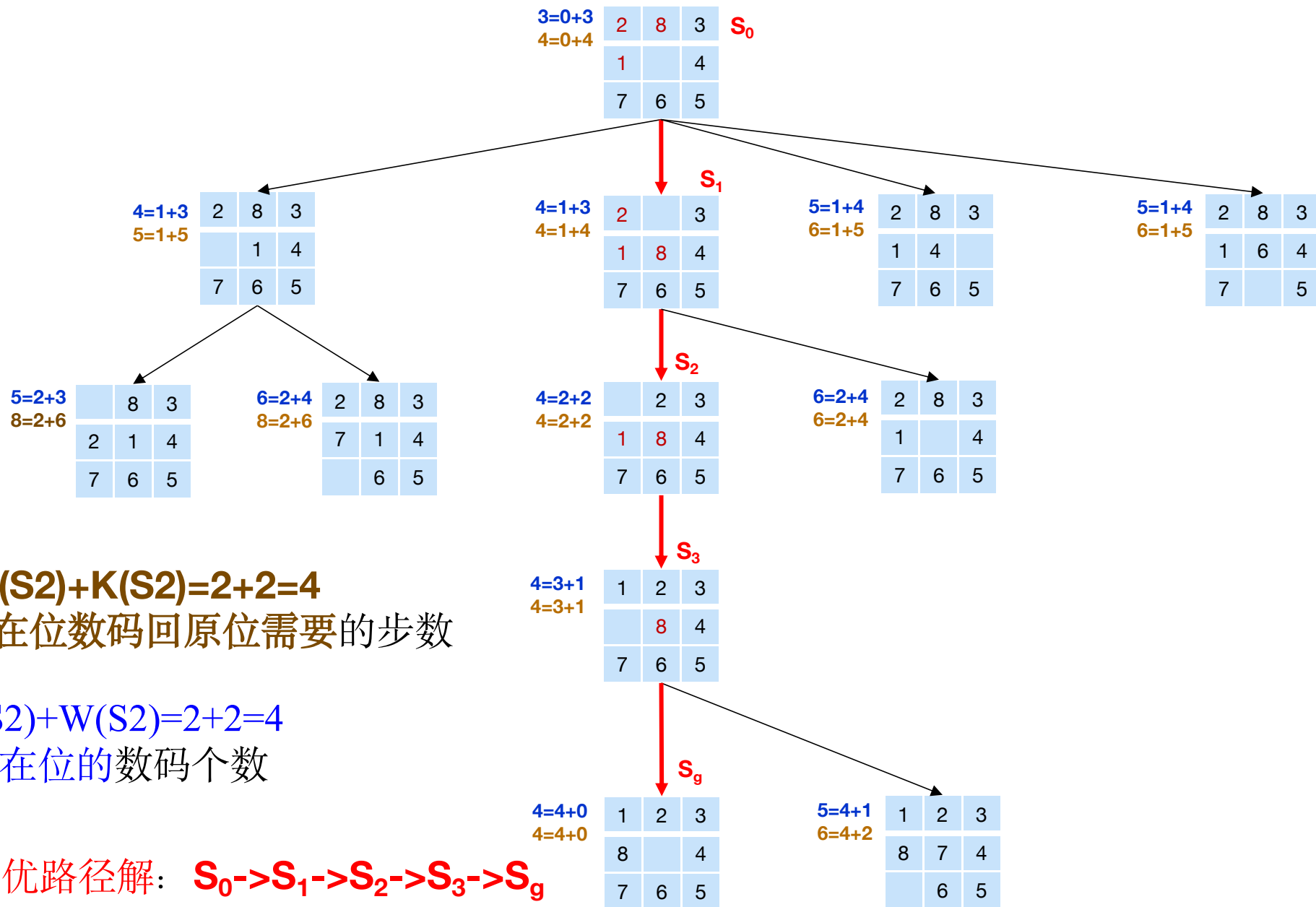
1	2	3
8		4
7	6	5

解：该问题的全局择优搜索树如下图所示。在该图中，每个节点旁边的数字是该节点的估价函数值。

4.3.4.2 A*算法的最优性



4.3.4.2 A*算法的最优性



A*: $f(S_2) = d(S_2) + K(S_2) = 2 + 2 = 4$

K(n)表示不在位数码回原位需要的步数

A: $f(S_2) = d(S_2) + W(S_2) = 2 + 2 = 4$

W(n)表示不在位的数码个数

最优路径解: $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_g$

4.3.4.3 $h(n)$ 的单调限制

在A*算法中，每当扩展一个节点 n 时，都需要检查其子节点是否已在**Open** 表或**Closed**表中。

对已在**Open**表中的子节点，需要决定是否调整指向其父节点的指针；

对已在**Closed**表中的子节点，除需要决定是否调整其指向父节点的指针外，还需要决定是否调整其子节点的后继节点的父指针。

如果能够保证，每当扩展一个节点时就已经找到了通往这个节点的最佳路径，就没有必要再去作上述检查。

4.3.4.3 $h(n)$ 的单调限制

在A*算法中，每当扩展一个节点 n 时，都需要检查其子节点是否已在**Open** 表或**Closed**表中。

对已在**Open**表中的子节点，需要决定是否调整指向其父节点的指针；

对已在**Closed**表中的子节点，除需要决定是否调整其指向父节点的指针外，还需要决定是否调整其子节点的后继节点的父指针。

如果能够保证，每当扩展一个节点时就已经找到了通往这个节点的最佳路径，就没有必要再去作上述检查。

为满足这一要求，我们需要对启发函数 $h(n)$ 增加单调性限制。

定义4.1 如果启发函数满足以下两个条件：

(1) $h(S_g)=0$;

(2) 对任意节点 n_i 及其任一子节点 n_j ，都有 $0 \leq h(n_i) - h(n_j) \leq c(n_i, n_j)$

其中 $c(n_i, n_j)$ 是 n_i 到其子节点 n_j 的边代价，则称 $h(n)$ 满足单调限制。

4.3.4.3 $h(n)$ 的单调限制

定理4.5 如果 h 满足单调条件，则当A*算法扩展节点 n 时，该节点就已经找到了通往它的最佳路径，即 $g(n)=g^*(n)$ 。

定理4.6 如果 $h(n)$ 满足单调限制，则A*算法扩展的节点序列的 f 值是非递减的，即 $f(n_i) \leq f(n_{i+1})$ 。

4.3.4.3 $h(n)$ 的单调限制

定理4.5 如果 h 满足单调条件, 则当 A^* 算法扩展节点 n 时, 该节点就已经找到了通往它的最佳路径, 即 $g(n)=g^*(n)$ 。

4.3.4.3 $h(n)$ 的单调限制

定理4.5 如果 h 满足单调条件, 则当 A^* 算法扩展节点 n 时, 该节点就已经找到了通往它的最佳路径, 即 $g(n)=g^*(n)$ 。

证明: 设 A^* 正要扩展节点 n , 而节点序列

$$S_0 = n_0, n_1, \dots, n_k = n$$

是由初始节点 S_0 到节点 n 的最佳路径。其中, n_i 是这个序列中最后一个位于**Closed**表中的节点, 则上述节点序列中的 n_{i+1} 节点必定在**Open**表中, 则有

$$g^*(n_i) + h(n_i) \leq g^*(n_i) + c(n_i, n_{i+1}) + h(n_{i+1})$$

4.3.4.3 $h(n)$ 的单调限制

定理4.5 如果 h 满足单调条件, 则当 A^* 算法扩展节点 n 时, 该节点就已经找到了通往它的最佳路径, 即 $g(n)=g^*(n)$ 。

证明: 设 A^* 正要扩展节点 n , 而节点序列

$$S_0 = n_0, n_1, \dots, n_k = n$$

是由初始节点 S_0 到节点 n 的最佳路径。其中, n_i 是这个序列中最后一个位于**Closed**表中的节点, 则上述节点序列中的 n_{i+1} 节点必定在**Open**表中, 则有

$$g^*(n_i) + h(n_i) \leq g^*(n_i) + c(n_i, n_{i+1}) + h(n_{i+1})$$

由于节点 n_i 和 n_{i+1} 都在最佳路径上, 故有

$$g^*(n_{i+1}) = g^*(n_i) + c(n_i, n_{i+1})$$

所以 $g^*(n_i) + h(n_i) \leq g^*(n_{i+1}) + h(n_{i+1})$

4.3.4.3 $h(n)$ 的单调限制

定理4.5 如果 h 满足单调条件, 则当 A^* 算法扩展节点 n 时, 该节点就已经找到了通往它的最佳路径, 即 $g(n)=g^*(n)$ 。

证明: 设 A^* 正要扩展节点 n , 而节点序列

$$S_0 = n_0, n_1, \dots, n_k = n$$

是由初始节点 S_0 到节点 n 的最佳路径。其中, n_i 是这个序列中最后一个位于**Closed**表中的节点, 则上述节点序列中的 n_{i+1} 节点必定在**Open**表中, 则有

$$g^*(n_i) + h(n_i) \leq g^*(n_i) + c(n_i, n_{i+1}) + h(n_{i+1})$$

由于节点 n_i 和 n_{i+1} 都在最佳路径上, 故有

$$g^*(n_{i+1}) = g^*(n_i) + c(n_i, n_{i+1})$$

所以 $g^*(n_i) + h(n_i) \leq g^*(n_{i+1}) + h(n_{i+1})$

一直推导下去可得 $g^*(n_{i+1}) + h(n_{i+1}) \leq g^*(n_k) + h(n_k)$

由于节点 n_{i+1} 在最佳路径上, 故有 $f(n_{i+1}) \leq g^*(n) + h(n)$

4.3.4.3 $h(n)$ 的单调限制

定理4.5 如果 h 满足单调条件, 则当 A^* 算法扩展节点 n 时, 该节点就已经找到了通往它的最佳路径, 即 $g(n)=g^*(n)$ 。

证明: 设 A^* 正要扩展节点 n , 而节点序列

$$S_0 = n_0, n_1, \dots, n_k = n$$

是由初始节点 S_0 到节点 n 的最佳路径。其中, n_i 是这个序列中最后一个位于**Closed**表中的节点, 则上述节点序列中的 n_{i+1} 节点必定在**Open**表中, 则有

$$g^*(n_i) + h(n_i) \leq g^*(n_i) + c(n_i, n_{i+1}) + h(n_{i+1})$$

由于节点 n_i 和 n_{i+1} 都在最佳路径上, 故有

$$g^*(n_{i+1}) = g^*(n_i) + c(n_i, n_{i+1})$$

所以 $g^*(n_i) + h(n_i) \leq g^*(n_{i+1}) + h(n_{i+1})$

一直推导下去可得 $g^*(n_{i+1}) + h(n_{i+1}) \leq g^*(n_k) + h(n_k)$

由于节点 n_{i+1} 在最佳路径上, 故有 $f(n_{i+1}) \leq g^*(n) + h(n)$

因为这时 A^* 扩展节点 n 而不扩展节点 n_{i+1} , 则有

$$f(n) = g(n) + h(n) \leq f(n_{i+1}) \leq g^*(n) + h(n)$$

即 $g(n) \leq g^*(n)$

4.3.4.3 $h(n)$ 的单调限制

定理4.5 如果 h 满足单调条件, 则当 A^* 算法扩展节点 n 时, 该节点就已经找到了通往它的最佳路径, 即 $g(n)=g^*(n)$ 。

证明: 设 A^* 正要扩展节点 n , 而节点序列

$$S_0 = n_0, n_1, \dots, n_k = n$$

是由初始节点 S_0 到节点 n 的最佳路径。其中, n_i 是这个序列中最后一个位于**Closed**表中的节点, 则上述节点序列中的 n_{i+1} 节点必定在**Open**表中, 则有

$$g^*(n_i) + h(n_i) \leq g^*(n_i) + c(n_i, n_{i+1}) + h(n_{i+1})$$

由于节点 n_i 和 n_{i+1} 都在最佳路径上, 故有

$$g^*(n_{i+1}) = g^*(n_i) + c(n_i, n_{i+1})$$

$$\text{所以 } g^*(n_i) + h(n_i) \leq g^*(n_{i+1}) + h(n_{i+1})$$

$$\text{一直推导下去可得 } g^*(n_{i+1}) + h(n_{i+1}) \leq g^*(n_k) + h(n_k)$$

$$\text{由于节点 } n_{i+1} \text{ 在最佳路径上, 故有 } f(n_{i+1}) \leq g^*(n) + h(n)$$

因为这时 A^* 扩展节点 n 而不扩展节点 n_{i+1} , 则有

$$f(n) = g(n) + h(n) \leq f(n_{i+1}) \leq g^*(n) + h(n)$$

$$\text{即 } g(n) \leq g^*(n)$$

但是 $g^*(n)$ 是最小代价值, 应当有 $g(n) \geq g^*(n)$, 所以有 $g(n) = g^*(n)$

4.3.4.3 $h(n)$ 的单调限制

定理4.6 如果 $h(n)$ 满足单调限制, 则A*算法扩展的节点序列的 f 值是非递减的, 即
 $f(n_i) \leq f(n_{i+1})$ 。

证明: 假设节点 n_{i+1} 在节点 n_i 之后立即扩展, 由单调限制条件可知

$$h(n_i) - h(n_{i+1}) \leq c(n_i, n_{i+1})$$

4.3.4.3 $h(n)$ 的单调限制

定理4.6 如果 $h(n)$ 满足单调限制, 则A*算法扩展的节点序列的 f 值是非递减的, 即 $f(n_i) \leq f(n_{i+1})$ 。

证明: 假设节点 n_{i+1} 在节点 n_i 之后立即扩展, 由单调限制条件可知

$$h(n_i) - h(n_{i+1}) \leq c(n_i, n_{i+1})$$

$$\text{即 } f(n_i) - g(n_i) - f(n_{i+1}) + g(n_{i+1}) \leq c(n_i, n_{i+1})$$

$$\text{亦即 } f(n_i) - g(n_i) - f(n_{i+1}) + g(n_i) + c(n_i, n_{i+1}) \leq c(n_i, n_{i+1})$$

4.3.4.3 $h(n)$ 的单调限制

定理4.6 如果 $h(n)$ 满足单调限制, 则A*算法扩展的节点序列的 f 值是非递减的, 即 $f(n_i) \leq f(n_{i+1})$ 。

证明: 假设节点 n_{i+1} 在节点 n_i 之后立即扩展, 由单调限制条件可知

$$h(n_i) - h(n_{i+1}) \leq c(n_i, n_{i+1})$$

$$\text{即 } f(n_i) - g(n_i) - f(n_{i+1}) + g(n_{i+1}) \leq c(n_i, n_{i+1})$$

$$\text{亦即 } f(n_i) - g(n_i) - f(n_{i+1}) + g(n_i) + c(n_i, n_{i+1}) \leq c(n_i, n_{i+1})$$

$$\text{所以 } f(n_i) - f(n_{i+1}) \leq 0$$

$$\text{即 } f(n_i) \leq f(n_{i+1})$$

以上两个定理都是在 $h(n)$ 满足单调性限制的前提下才成立的。如果 $h(n)$ 不满足单调性限制, 则它们不一定成立。

4.3.4.3 $h(n)$ 的单调限制

定理4.6 如果 $h(n)$ 满足单调限制, 则A*算法扩展的节点序列的 f 值是非递减的, 即 $f(n_i) \leq f(n_{i+1})$ 。

证明: 假设节点 n_{i+1} 在节点 n_i 之后立即扩展, 由单调限制条件可知

$$h(n_i) - h(n_{i+1}) \leq c(n_i, n_{i+1})$$

$$\text{即 } f(n_i) - g(n_i) - f(n_{i+1}) + g(n_{i+1}) \leq c(n_i, n_{i+1})$$

$$\text{亦即 } f(n_i) - g(n_i) - f(n_{i+1}) + g(n_i) + c(n_i, n_{i+1}) \leq c(n_i, n_{i+1})$$

$$\text{所以 } f(n_i) - f(n_{i+1}) \leq 0$$

$$\text{即 } f(n_i) \leq f(n_{i+1})$$

以上两个定理都是在 $h(n)$ 满足单调性限制的前提下才成立的。如果 $h(n)$ 不满足单调性限制, 则它们不一定成立。

在 $h(n)$ 满足单调性限制下的A*算法常被称为改进的A*算法。

目录

01

4.1 概述

02

4.2 状态空间盲目搜索

03

4.3 状态空间启发式搜索

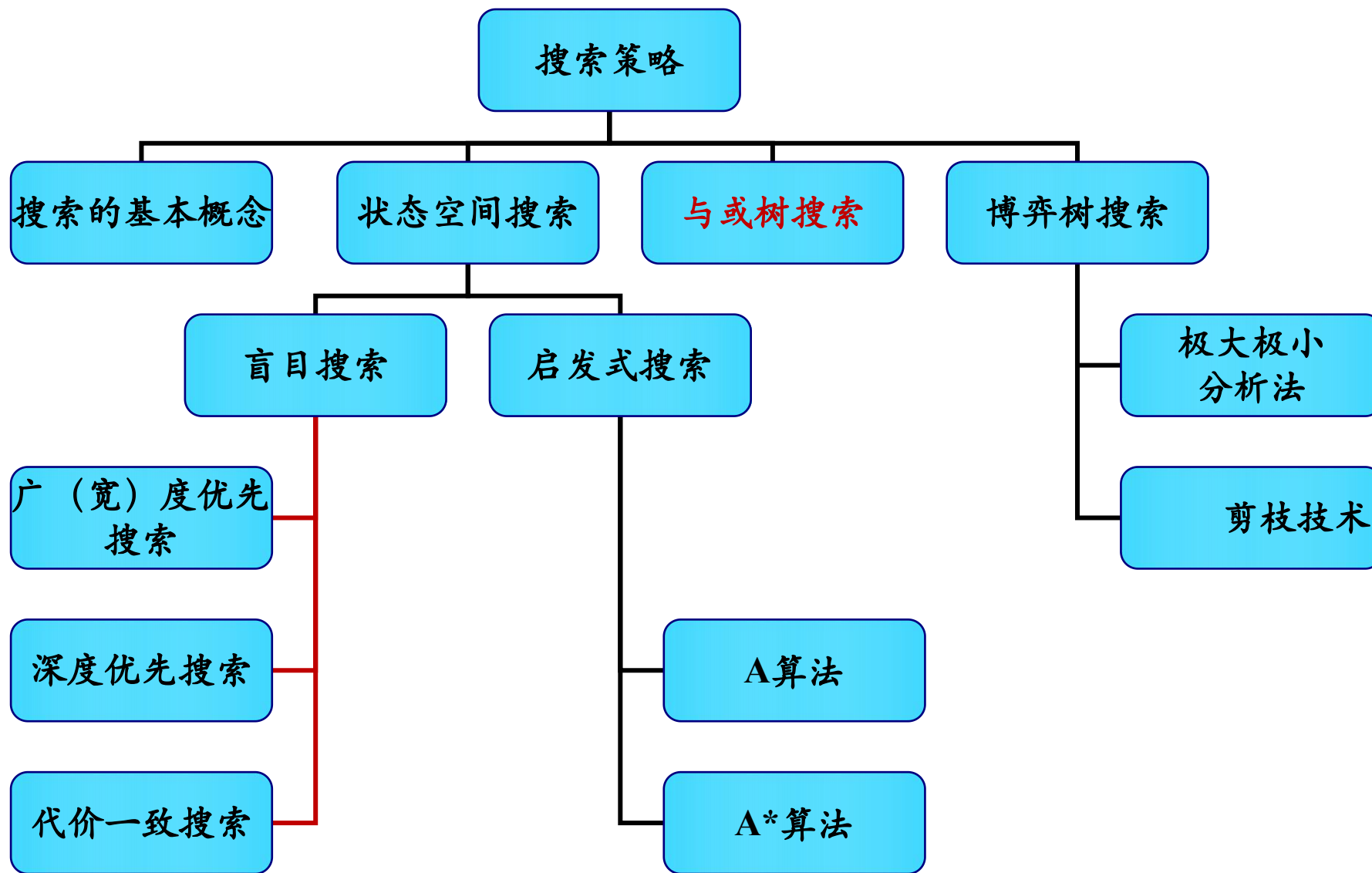
04

4.4 与/或树搜索

05

4.5 博弈树的启发式搜索

本章知识结构



4.4.1 与/或树的一般搜索

与/或树的搜索过程实际上是一个不断寻找解树的过程。其一般搜索过程如下：

- (1) 把原始问题作为初始节点 S_0 ，并把它作为当前节点；
- (2) 应用分解或等价变换操作对当前节点进行扩展；
- (3) 为每个子节点设置指向父节点的指针；
- (4) **选择合适的子节点作为当前节点**，反复执行第(2)步和第(3)步，在此期间需要多次调用可解标记过程或不可解标记过程，直到初始节点被标记为可解节点或不可解节点为止。

上述搜索过程将形成一颗与/或树，这种由搜索过程所形成的与/或树称为搜索树。

4.4.2 广度优先搜索

□ 与状态空间的广度优先搜索类似

Setp 1: 把初始节点 S_0 放入Open表中;

Setp 2: 把Open表的第一个节点取出放入Closed表中, 并记该节点为节点n;

Step 3: 若节点n可扩展, 则做下列工作:

①扩展节点n, 将其子节点放入Open表的尾部, 并为每一个子节点设置指向父节点的指针;

②考察这些子节点是否有终止节点。若有, 则标记这些终止节点为可解节点, 并用可解标记过程对其父节点或先辈节点中可解节点进行标记。如初始节点 S_0 被标记为可解节点, 得到解树, 成功退出。如不能确定 S_0 为可解节点, 则从Open表中删除具有可解先辈的节点;

③转Step 2

4.4.2 广度优先搜索

□ 与状态空间的广度优先搜索类似

Step 4: 如果节点不可扩展, 则做下列工作:

① 标记节点 n 为不可解节点;

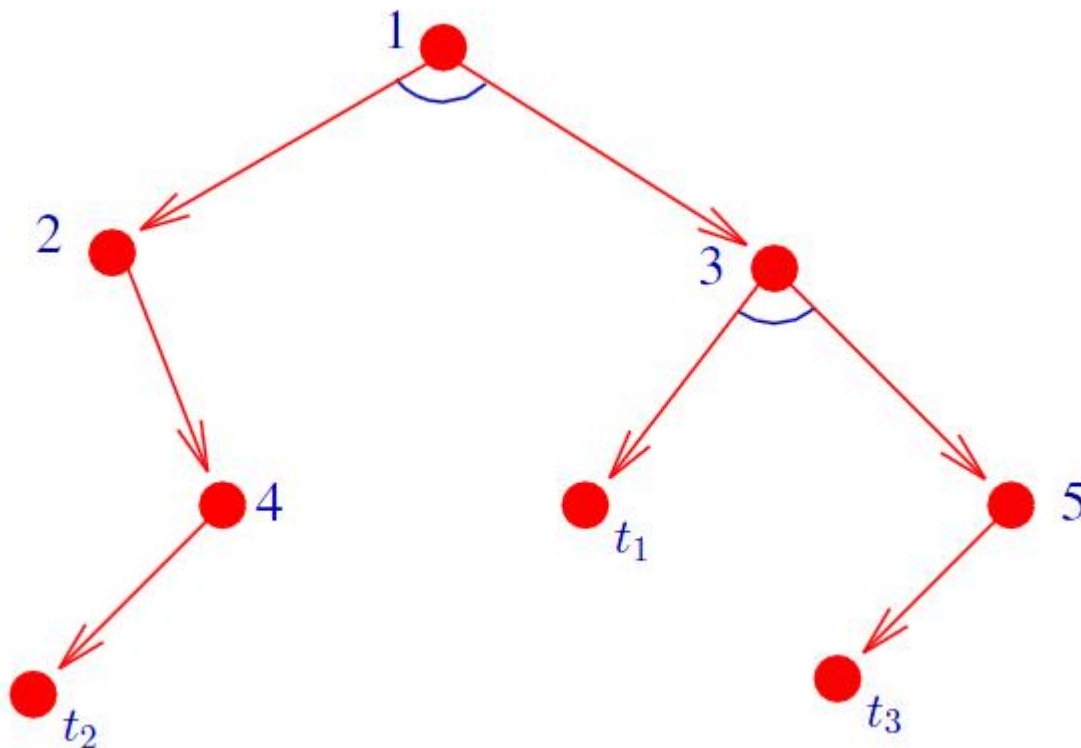
② 用不可解标记过程对节点 n 的先辈节点中不可解节点进行标记。如初始节点 S_0 被标记为不可解节点, 原问题无解而失败退出。如不能确定 S_0 为不可解节点, 则从Open表中删除具有不可解先辈的节点;

③ 转Step 2

4.4.2 广度优先搜索

【例】与或树广度优先搜索：

设有下图所示的与或树，节点按标注顺序进行扩展，其中标有 t_1 、 t_2 、 t_3 的节点是终止节点，A、B、C为不可解的端节点。



4.4.2 与或树的深度优先搜索

与广度优先搜索主要区别在于Open表中节点的排列顺序不同。
在扩展节点时，与或树的深度优先搜索总是把刚生成的节点放在Open表的首部。同状态空间的深度优先搜索类似，与或树的深度优先搜索也可以带有深度限制 d_m ，其搜索步骤如下：

Step 1：把初始节点 S_0 放入Open表中；

Step 2：把Open表的第一个节点取出放入Closed表中，并记该节点为节点 n ；

Step 3：如果节点 n 的深度等于 d_m ，则转Step5的第①点；

4.4.2 与或树的深度优先搜索

Step 4: 若节点 n 可扩展, 则做下列工作:

①扩展节点 n , 将其子节点放入Open表的**首部**, 并为每一个子节点设置指向父节点的指针;

②考察这些子节点是否有**终止节点**。若有, 则标记这些终止节点为可解节点, 并用**可解标记过程**对其父节点或先辈节点中可解节点进行标记。如初始节点 S_0 被标记为可解节点, 得到解树, 成功退出。如不能确定 S_0 为可解节点, 则从Open表中删除具有可解先辈的节点;

③转Step 2

4.4.2 与或树的深度优先搜索

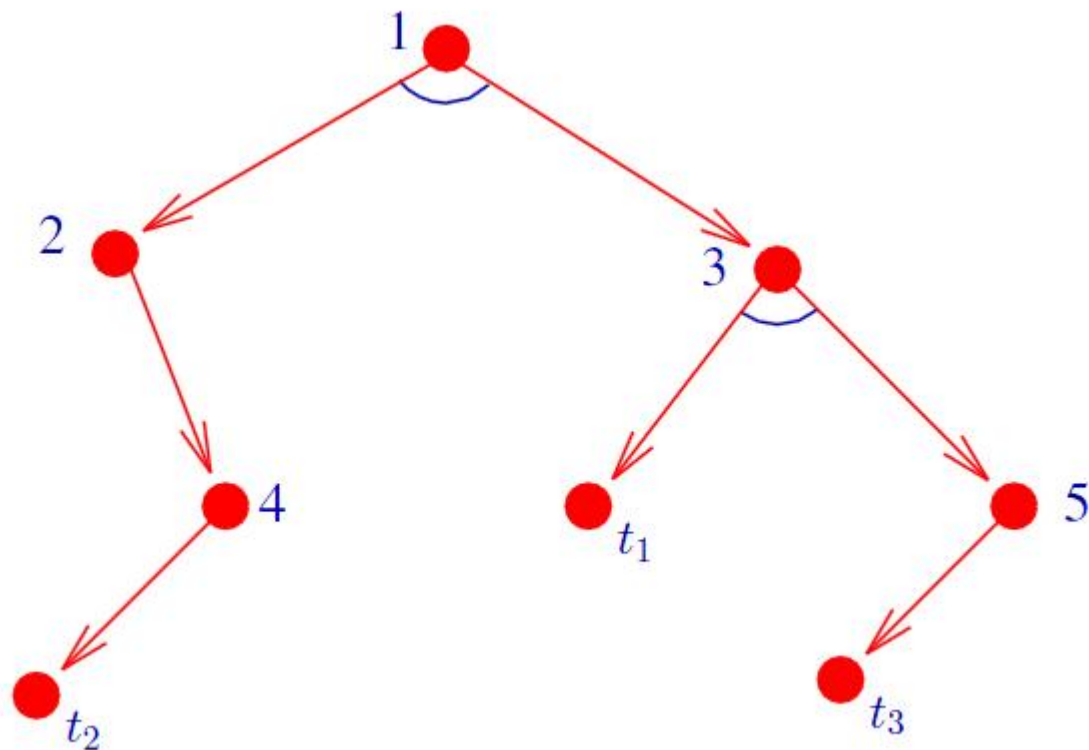
Step 5: 如果节点**不可扩展**, 则做下列工作:

- ① 标记节点 n 为**不可解节点**;
- ② 用**不可解标记过程**对节点 n 的先辈节点中不可解节点进行标记。如初始节点 S_0 被标记为不可解节点, 原问题无解而失败退出。如不能确定 S_0 为不可解节点, 则从Open表中删除具有不可解先辈的节点;
- ③ 转Step 2

4.4.2 与或树的深度优先搜索

【例】与或树深度优先搜索：

设有下图所示的与或树，节点按标注顺序进行扩展，其中标有 t_1 、 t_2 、 t_3 的节点是终止节点，A、B、C为不可解的端节点。



4.4.2 与或树的盲目搜索的特点

- 从初始节点开始，自上而下的搜索，寻找终止节点以及端节点；然后自下而上的进行可解标记，一旦初始节点被标记为可解或不可解节点，搜索停止。
- 按照确定路线进行，没有考虑付出的代价，不能够保证找到代价最小的解树。

4.4.3 与或树的启发式搜索

- ❖ 与/或树的**启发式搜索**过程实际上是一种利用搜索过程所得到的启发性信息寻找最优解树的过程。
- ❖ 算法的每一步都试图找到一个最有希望成为**最优解树的子树**。
- ❖ 最优解树是**指代价最小**的那棵解树。
- ❖ 它涉及到**解树的代价与希望树**。

4.4.3 与或树的启发式搜索

□ 解树的代价

❖ 若 n 为终止节点，则其代价 $h(n) = 0$

❖ 若 n 为或节点，且子节点为 $n_1, n_2, \dots, n_k$ ，则 n 的代价为：

$$h(n) = \min_{1 \leq i \leq k} \{c(n, n_i) + h(n_i)\}$$

其中， $c(n, n_i)$ 是节点 n 到其子节点 n_i 的代价。

❖ 若 n 为与节点，且子节点为 $n_1, n_2, \dots, n_k$ ，则 n 的代价可用和代价法或最大代价法求出来：

若用和代价法，则计算公式为： $h(n) = \sum_{i=1}^k [c(n, n_i) + h(n_i)]$ ；

若用最大代价法，则其计算公式为： $h(n) = \max_{1 \leq i \leq k} \{c(n, n_i) + h(n_i)\}$ 。

❖ 若 n 是端节点，但又不是终止节点，则 n 不可扩展，其代价定义为 $h(n) = +\infty$

❖ 根节点的代价即为解树的代价。

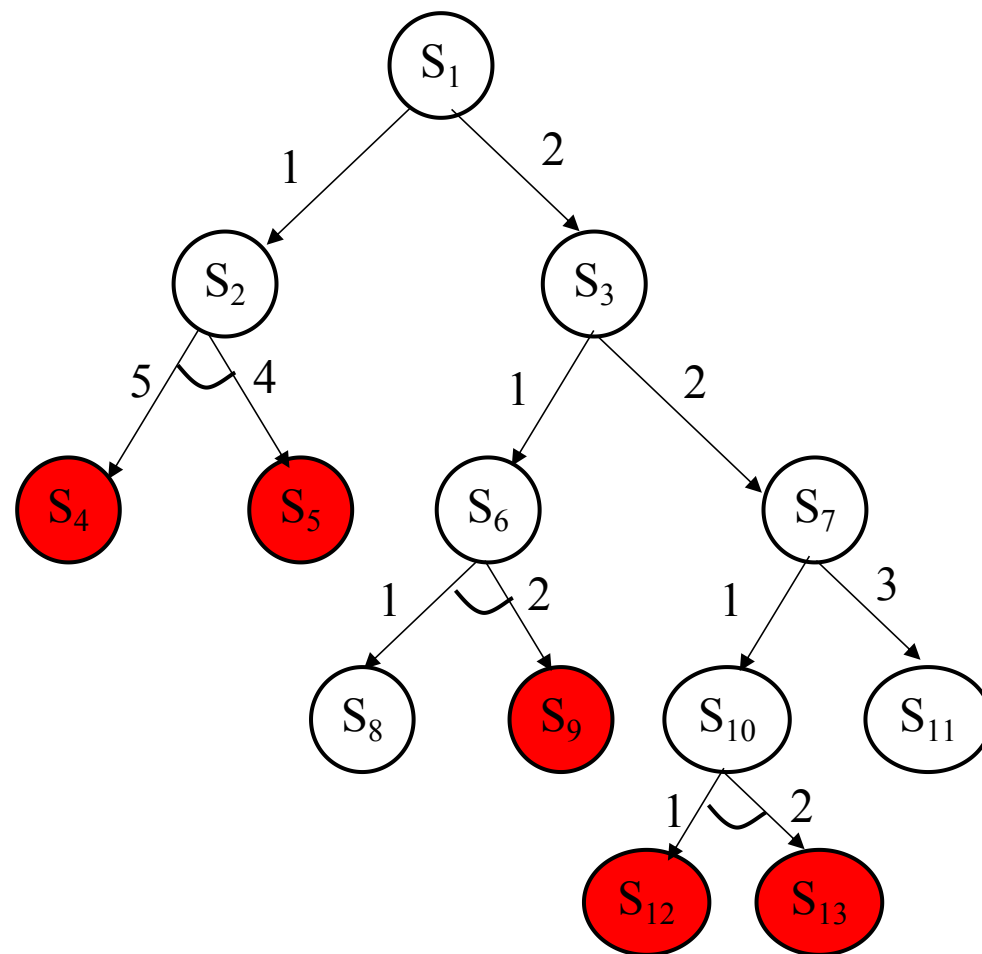
4.4.3 与或树的启发式搜索

□ 解树的代价实例

如图所示一棵与/或树，已知节点 **S4、S5、S9、S12、S13** 是本原问题，边上的数字是操作的代价。

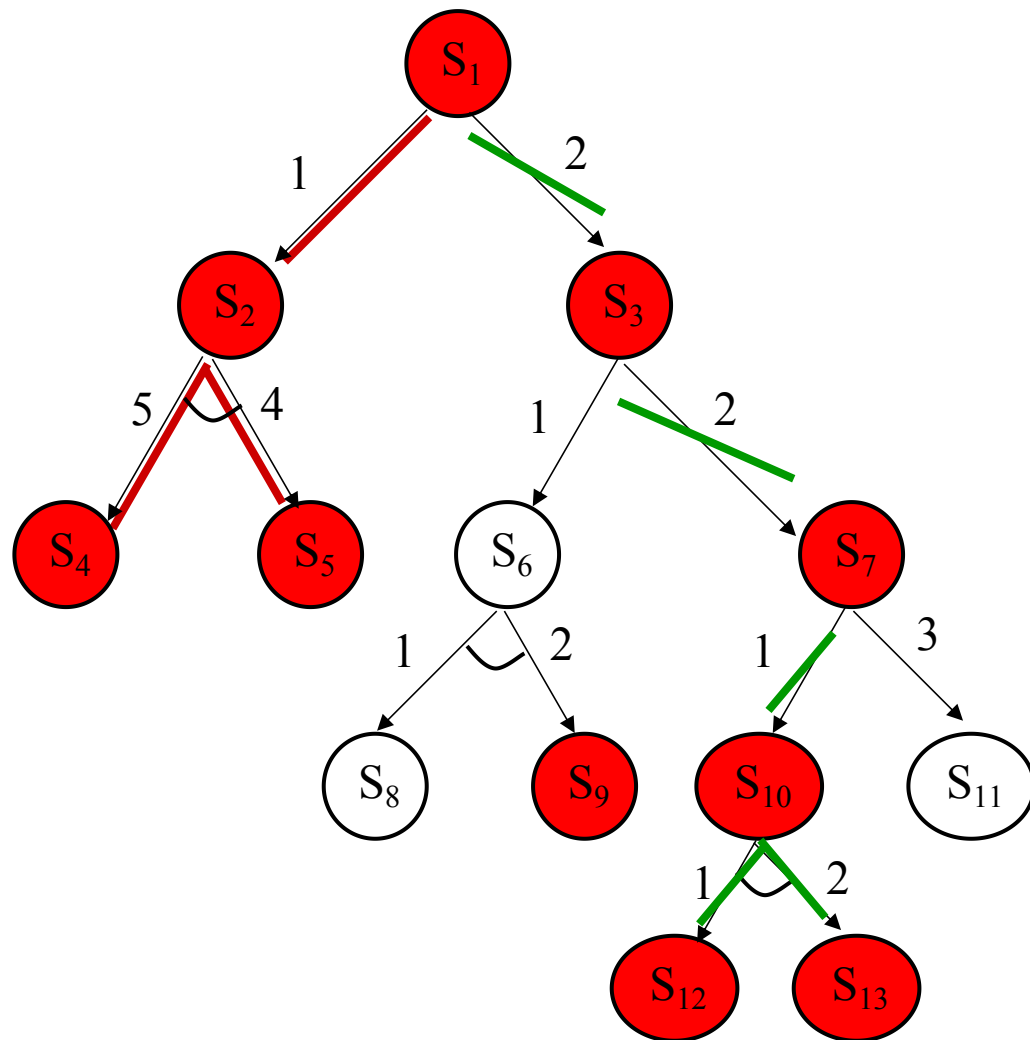
试求：

- (1) 按**和代价法**求出最优解树；
- (2) 按**最大代价法**求出最优解树。



4.4.3 与或树的启发式搜索

□ 和代价法计算解树代价



左边解树:

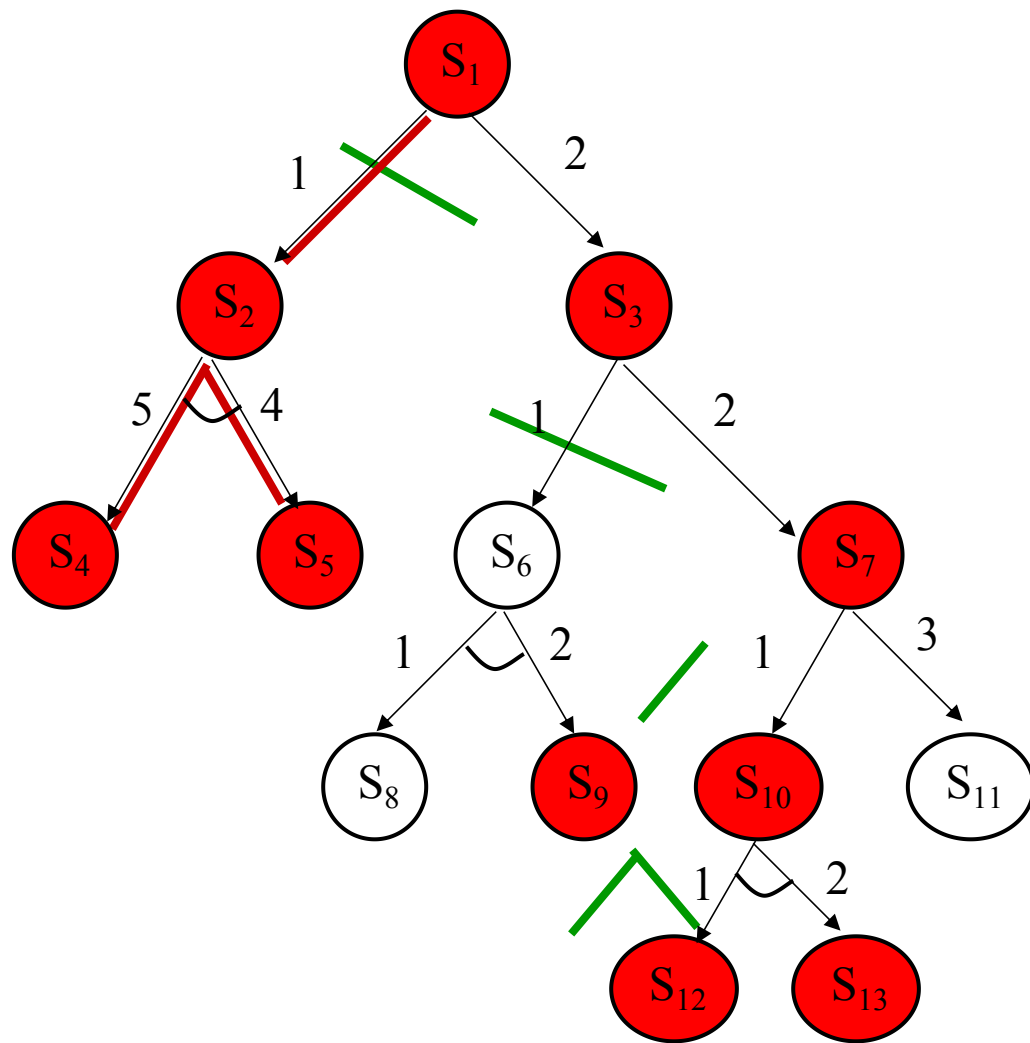
$$h_L(S_1)=10$$

右边解树:

$$h_R(S_1)=8$$

4.4.3 与或树的启发式搜索

□ 最大代价法计算解树代价



左边解树:

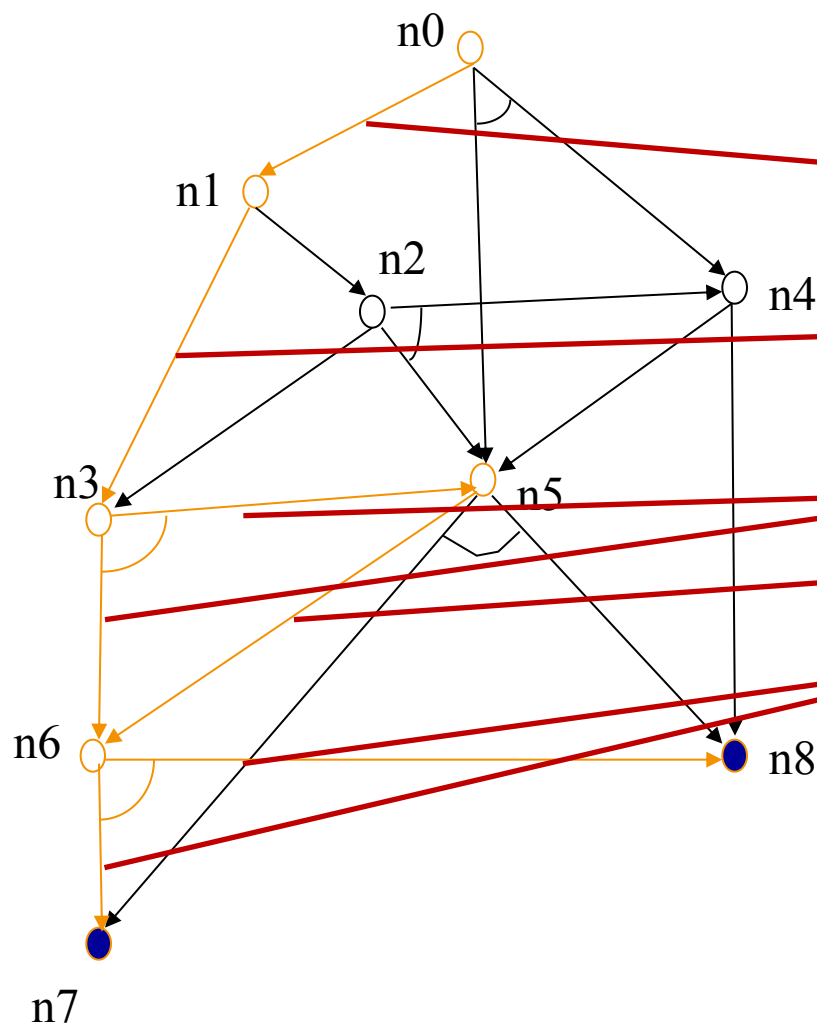
$$h_L(S_1)=6$$

右边解树:

$$h_R(S_1)=7$$

4.4.3 与或树的启发式搜索

□ 和代价法



$h(n_0)$

$$\begin{aligned} &= C(n_0, n_1) + h(n_1) \\ &= 1 + h(n_1) \end{aligned}$$

$$\begin{aligned} &= 1 + C(n_1, n_3) + h(n_3) \\ &= 1 + 1 + h(n_3) \end{aligned}$$

$$= 1 + 1 + 2 + h(n_5) + h(n_6)$$

$$= 1 + 1 + 2 + 1 + h(n_6) + h(n_6)$$

$$= 1 + 1 + 2 + 1 + 2 + h(n_7) + h(n_8) + h(n_6)$$

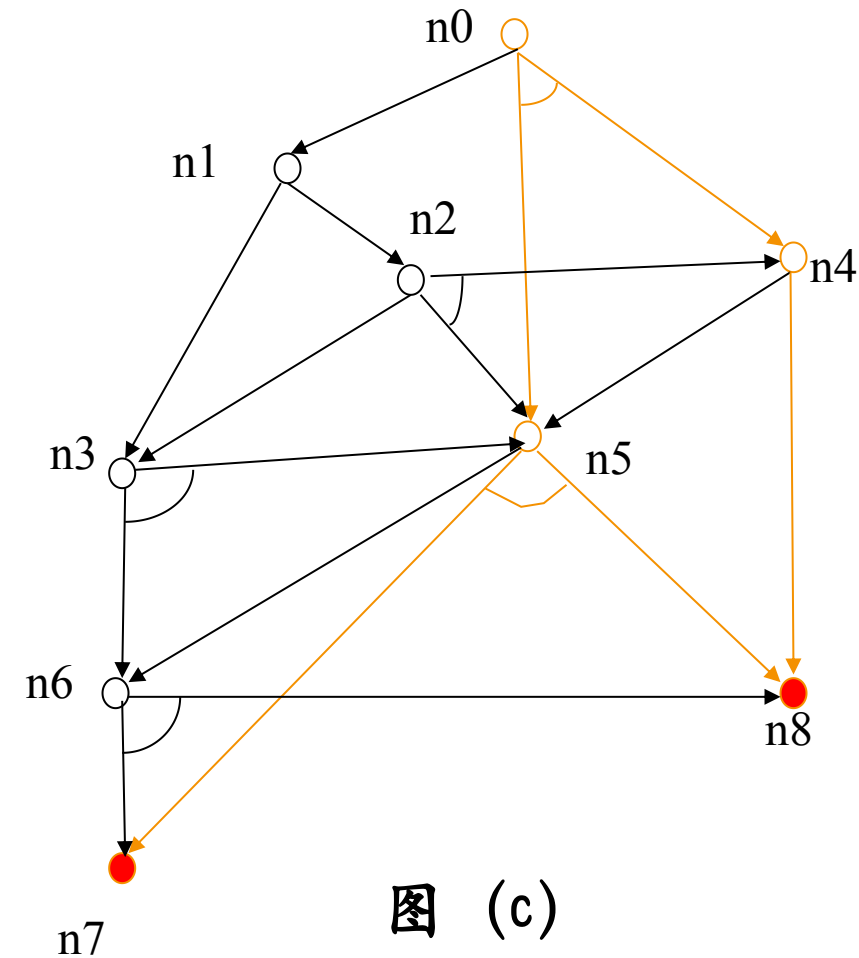
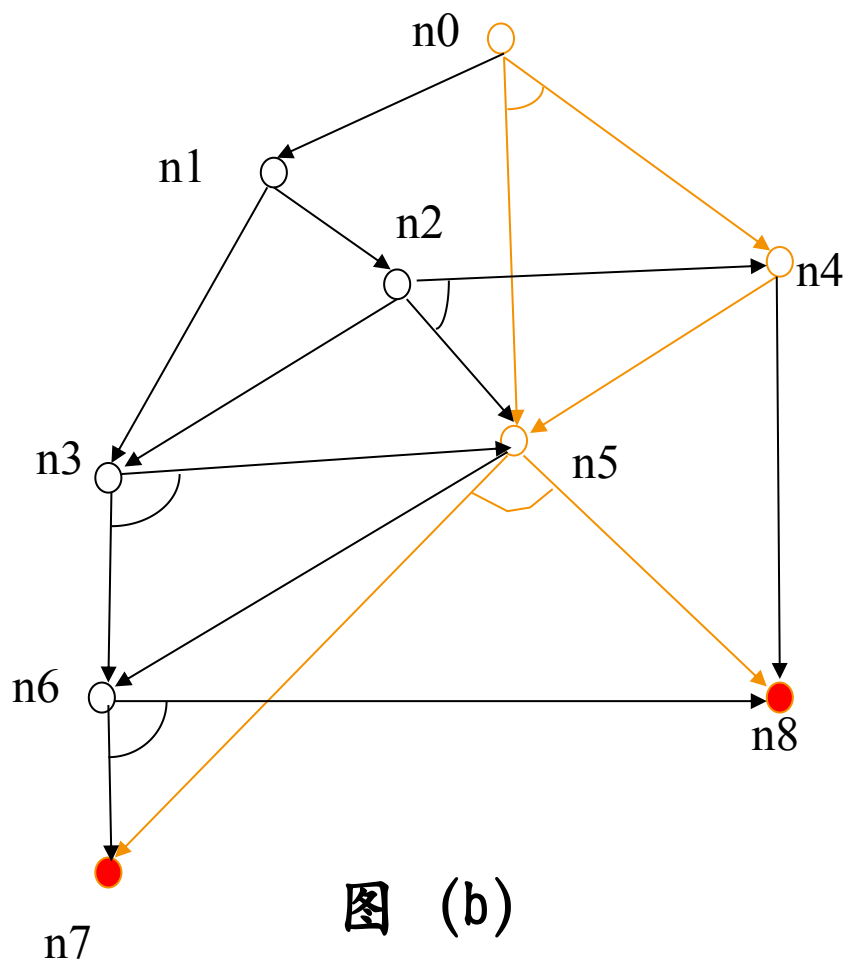
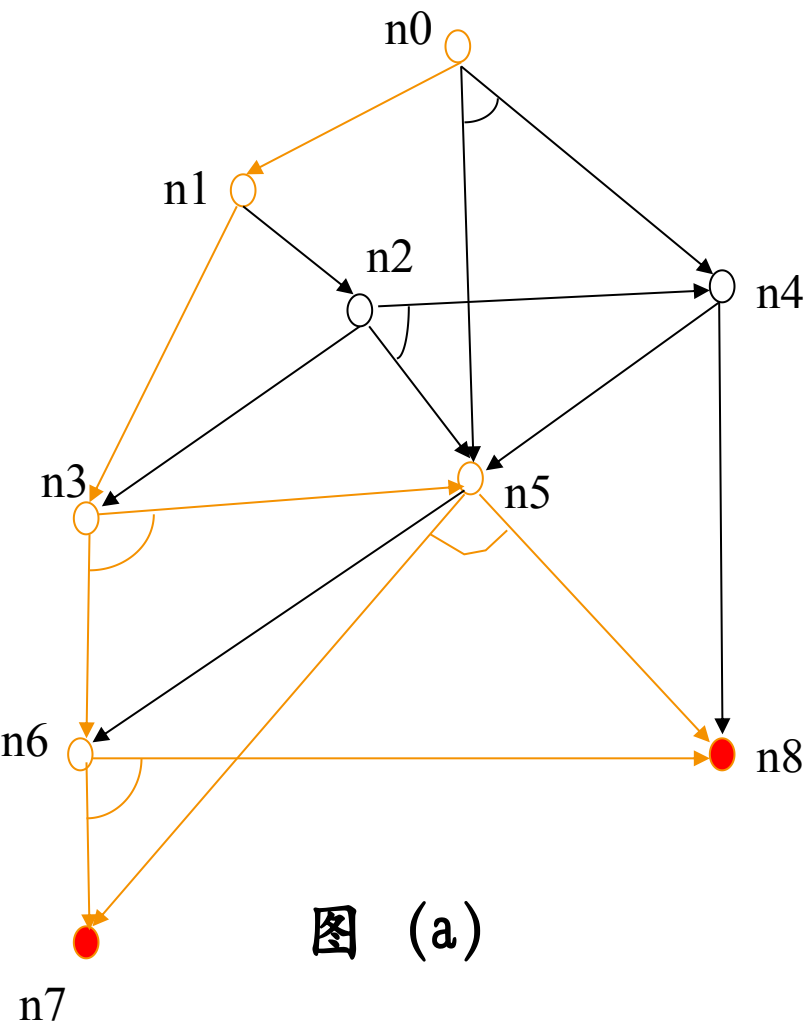
$$= 1 + 1 + 2 + 1 + 2 + h(n_6)$$

$$= 1 + 1 + 2 + 1 + 2 + 2 + h(n_7) + h(n_8)$$

$$= 1 + 1 + 2 + 1 + 2 + 2 = 9$$

【课堂练习】求解树的代价

已知 n_7 、 n_8 为本原节点。



4.4.4 希望树

- 为找到最佳解树，搜索过程的任何时刻都应该选择那些最有希望成为最佳解树一部分的节点进行扩展。由于这些节点及其父节点所构成的与或树最有可能成为最佳解树的一部分，因此称之为**希望树**。
- 与或树的启发式搜索过程就是不断地**选择、修正希望树的过程**，在该过程中，**希望树是不断变化的**。

4.4.2 广度优先搜索

□ 与状态空间的广度优先搜索类似

Setp 1: 把初始节点 S_0 放入Open表中;

Setp 2: 把Open表的第一个节点取出放入Closed表中, 并记该节点为节点n;

Step 3: 若节点n可扩展, 则做下列工作:

①扩展节点n, 将其子节点放入Open表的尾部, 并为每一个子节点设置指向父节点的指针;

②考察这些子节点是否有终止节点。若有, 则标记这些终止节点为可解节点, 并用可解标记过程对其父节点或先辈节点中可解节点进行标记。如初始节点 S_0 被标记为可解节点, 得到解树, 成功退出。如不能确定 S_0 为可解节点, 则从Open表中删除具有可解先辈的节点;

③转Step 2

4.4.2 广度优先搜索

□ 与状态空间的广度优先搜索类似

Step 4: 如果节点不可扩展, 则做下列工作:

① 标记节点 n 为不可解节点;

② 用不可解标记过程对节点 n 的先辈节点中不可解节点进行标记。如初始节点 S_0 被标记为不可解节点, 原问题无解而失败退出。如不能确定 S_0 为不可解节点, 则从Open表中删除具有不可解先辈的节点;

③ 转Step 2

4.4.3 与或树的启发式搜索

□ 解树的代价

❖ 若 n 为终止节点，则其代价 $h(n) = 0$

❖ 若 n 为或节点，且子节点为 $n_1, n_2, \dots, n_k$ ，则 n 的代价为：

$$h(n) = \min_{1 \leq i \leq k} \{c(n, n_i) + h(n_i)\}$$

其中， $c(n, n_i)$ 是节点 n 到其子节点 n_i 的代价。

❖ 若 n 为与节点，且子节点为 $n_1, n_2, \dots, n_k$ ，则 n 的代价可用和代价法或最大代价法求出来：

若用和代价法，则计算公式为： $h(n) = \sum_{i=1}^k [c(n, n_i) + h(n_i)]$ ；

若用最大代价法，则其计算公式为： $h(n) = \max_{1 \leq i \leq k} \{c(n, n_i) + h(n_i)\}$ 。

❖ 若 n 是端节点，但又不是终止节点，则 n 不可扩展，其代价定义为 $h(n) = +\infty$

❖ 根节点的代价即为解树的代价。

4.4.4 希望树

□ 希望树定义

1. 初始节点 S_0 在希望树 T 中;
2. 如果节点 x 在希望树 T 中, 则一定有:
 - (1) 如果 x 是具有子节点 $y_1, y_2, \dots, y_k$ 的“或”节点, 则具有
$$\min\{c(x, y_i) + h(y_i)\} \quad (i=1, 2, \dots, k)$$
值的那个节点 y_i 也应在 T 中;
 - (2) 如果 x 是“与”节点, 则 x 的全部子节点都在希望树 T 中。

4.4.4 希望树

□ 与或树的启发式搜索过程

- (1) 把初始节点 S_0 放入Open表中, 计算 $h(S_0)$;
- (2) 计算希望树 T ;
- (3) 依次在Open表中取出 T 的端节点放入Closed表, 并记该节点为 n ;
- (4) 如果节点 n 为终止节点, 则做下列工作:
 - ① 标记节点 n 为可解节点;
 - ② 在 T 上应用可解标记过程, 对 n 的先辈节点中的所有可解节点进行标记;
 - ③ 如果初始解节点 S_0 能够被标记为可解节点, 则 T 就是最优解树, 成功退出;
 - ④ 否则, 从Open表中删去具有可解先辈的所有节点。
 - ⑤ 转第(2)步。

4.4.4 希望树

□ 与或树的启发式搜索过程

(5) 如果节点 n 不是终止节点，但可扩展，则做下列工作：

- ① 扩展节点 n ，生成 n 的所有子节点；
- ② 把这些子节点都放入Open表中，并为每一个子节点设置指向父节点 n 的指针
- ③ 计算这些子节点及其先辈节点的 h 值；
- ④ 转第(2)步。

(6) 如果节点 n 不是终止节点，且不可扩展，则做下列工作：

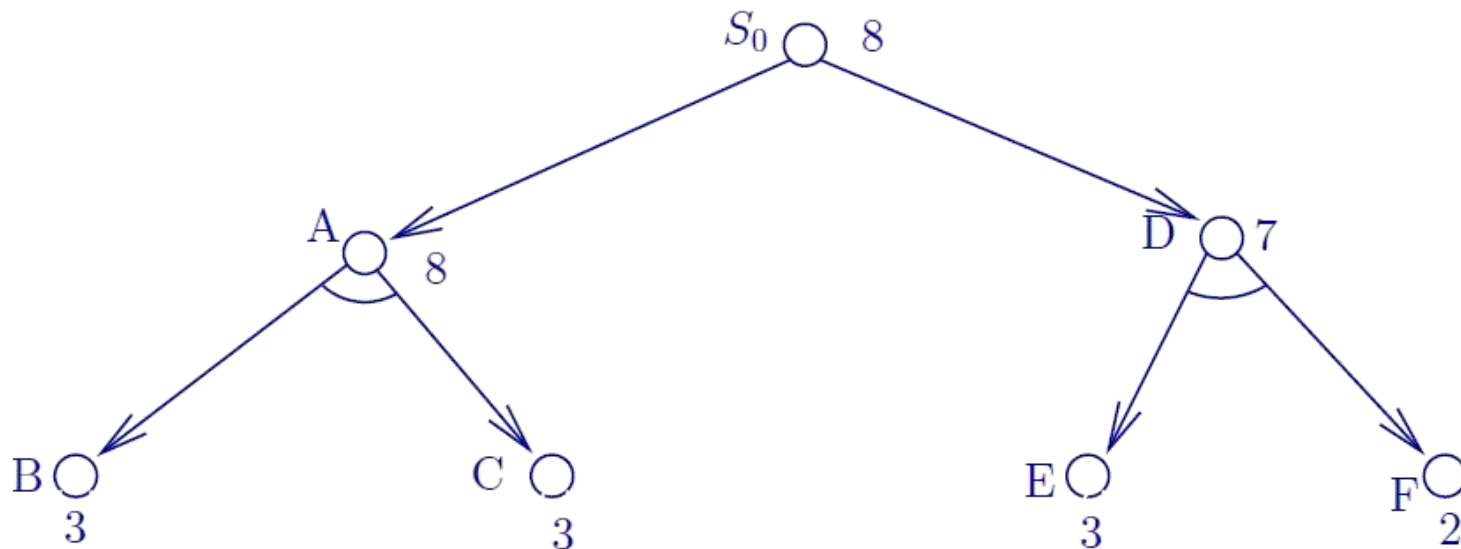
- ① 标记节点 n 为不可解节点；
- ② 在T上应用不可解标记过程，对 n 的先辈节点中的所有不可解节点进行标记；
- ③ 如果初始节点 S_0 能够被标记为不可解，则问题无解，失败退出；
- ④ 否则，从Open表中删去具有不可解先辈的所有节点。
- ⑤ 转第(2)步。

4.4.4 希望树

□ 与或树的启发式搜索举例

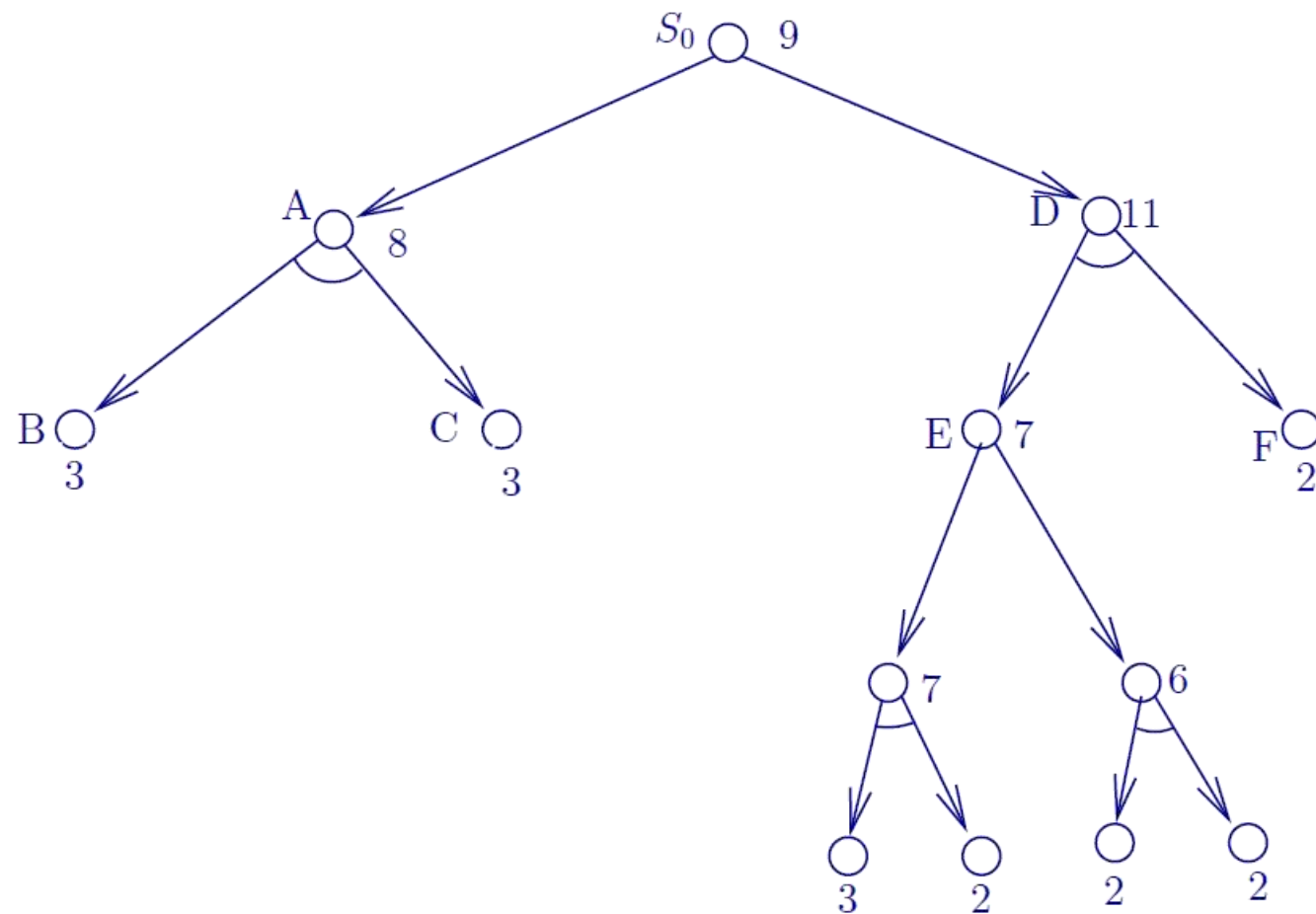
例：要求搜索过程每次扩展节点时都**同时扩展两层**，且按一层或节点、一层与节点的间隔方式进行扩展。它实际上就是下一节将要讨论的博弈树的结构。

设初始节点为 S_0 ，对 S_0 扩展后得到的与/或树如图所示。其中，端节点**B**、**C**、**E**、**F**，下面的数字是用启发函数估算出的 h 值，节点 S_0 、**A**、**D**旁边的数字是按和代价法计算出来的节点代价。



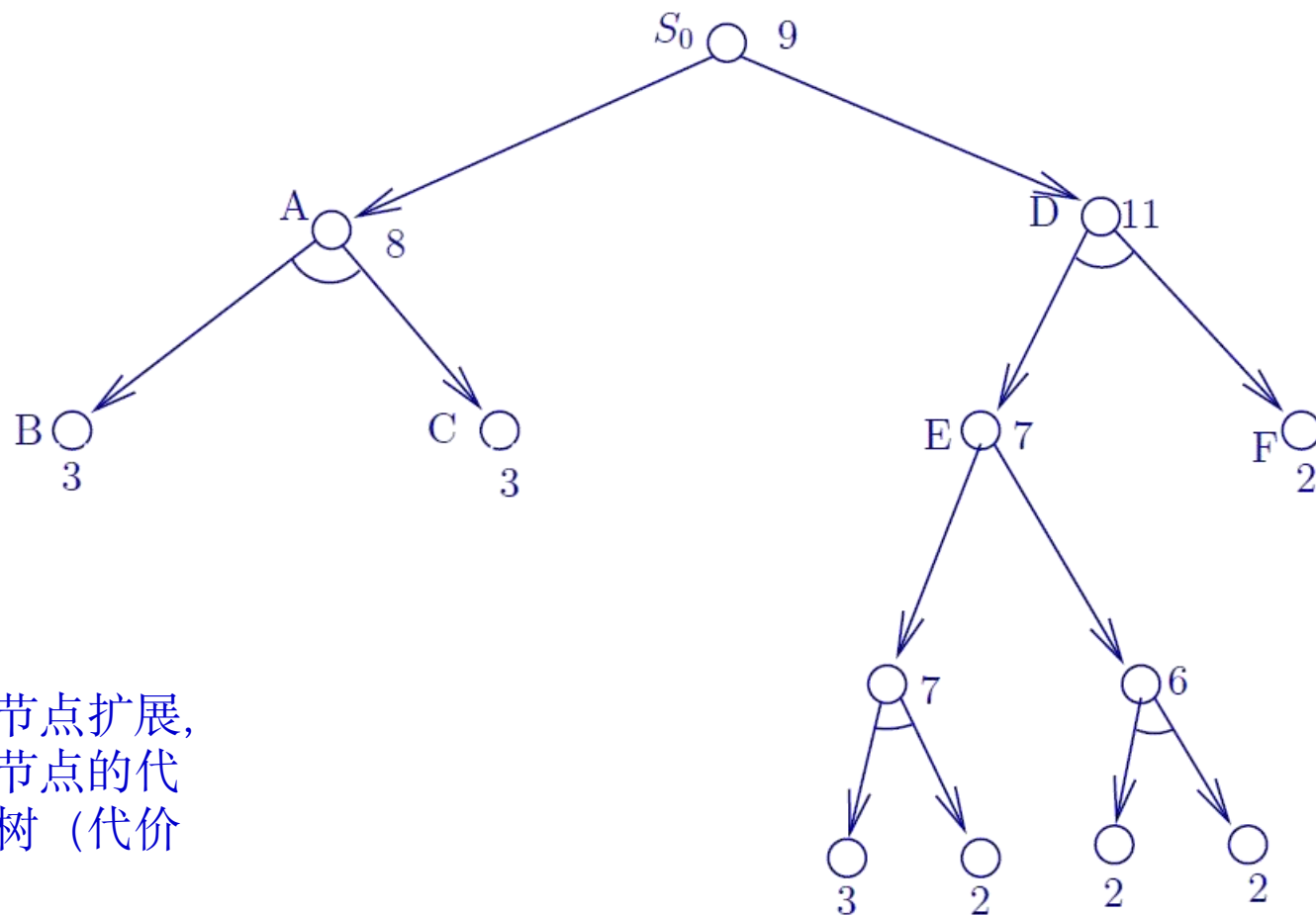
4.4.4 希望树

□ 与或树的启发式搜索举例



4.4.4 希望树

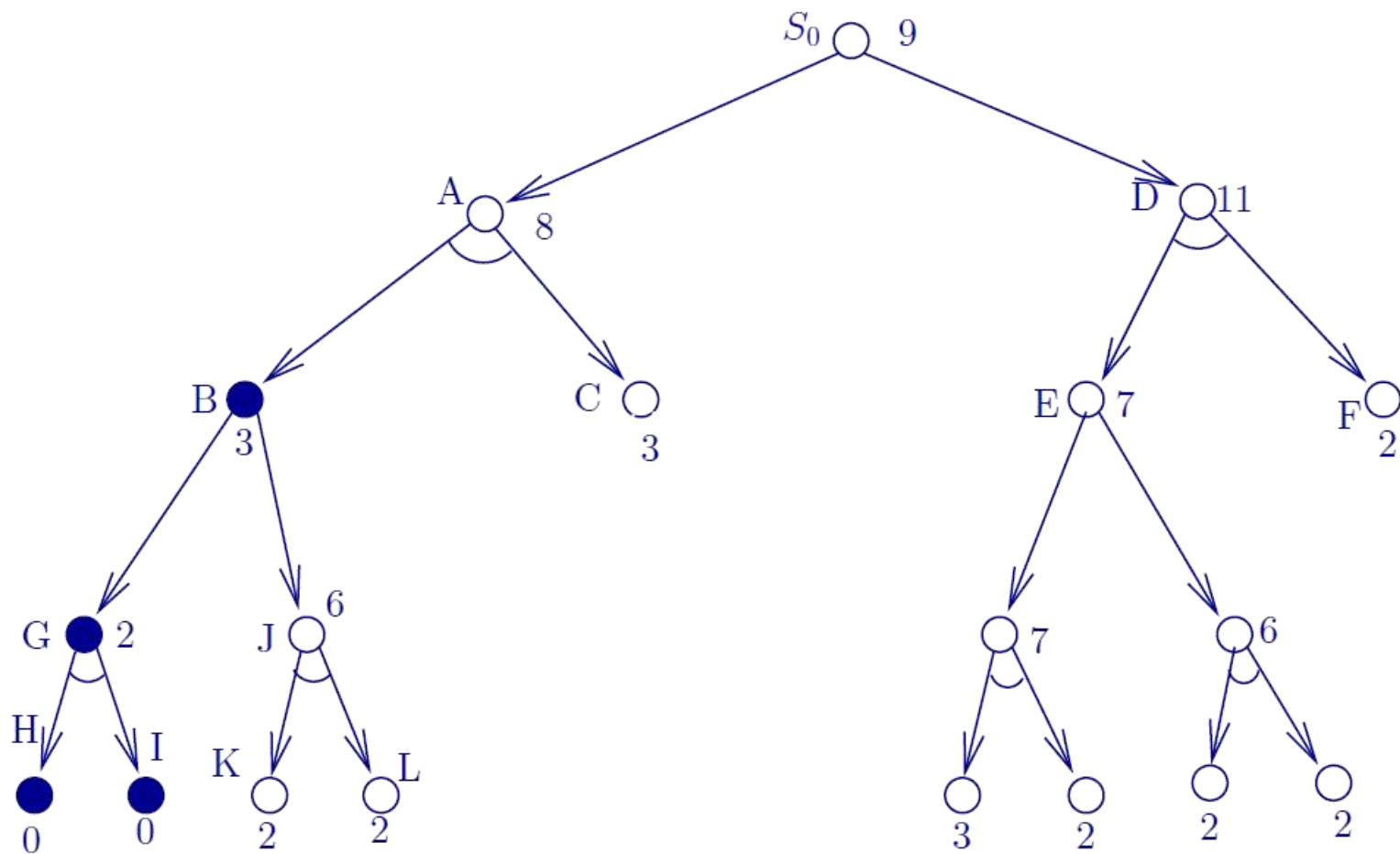
□ 与或树的启发式搜索举例



对右边子树进行或节点和与节点扩展，
通过右边子树计算出的初始节点的代
价为12，希望树变成左边子树（代价
为9）

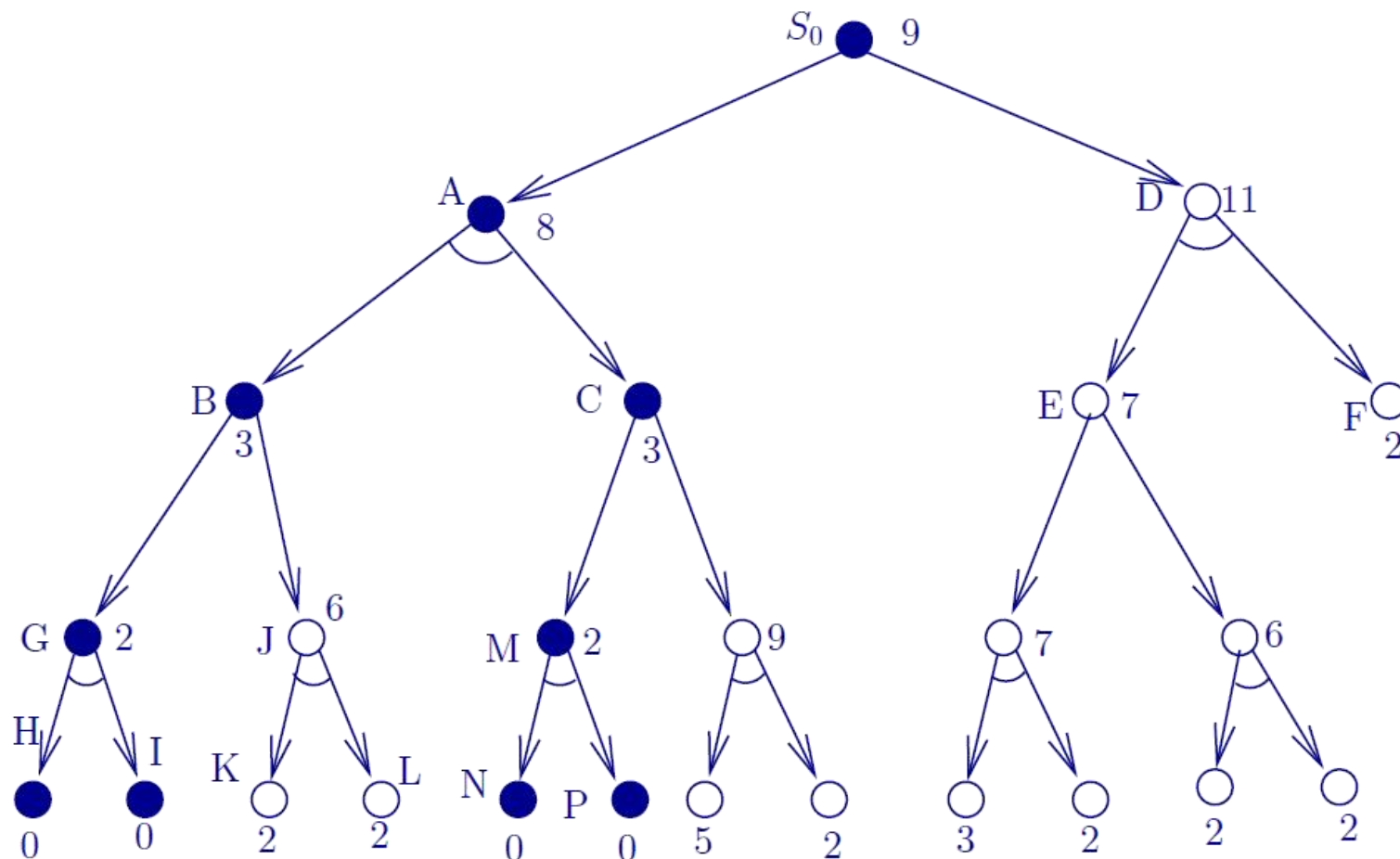
4.4.4 希望树

□ 与或树的启发式搜索举例



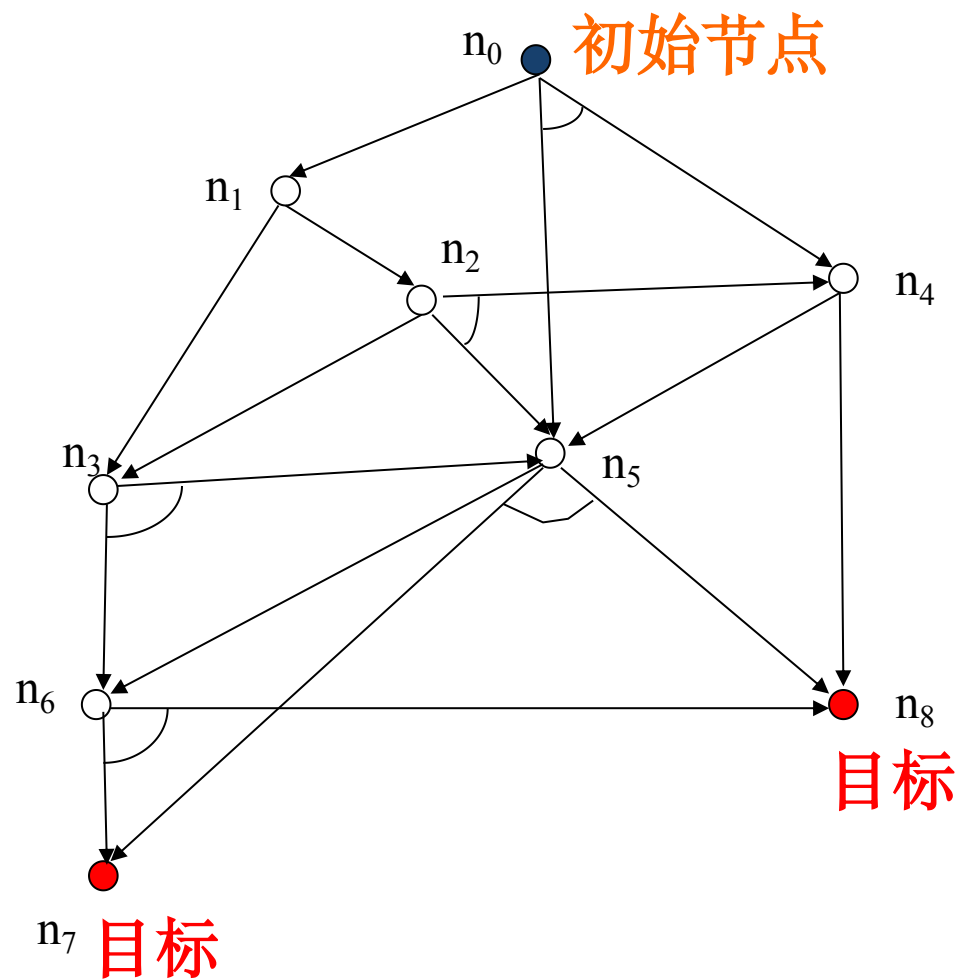
4.4.4 希望树

□ 与或树的启发式搜索举例



主观题

请给出下图所示问题的与或树的启发式搜索树



已知其中:

$$h(n_1)=2$$

$$h(n_2)=4$$

$$h(n_3)=4$$

$$h(n_4)=1$$

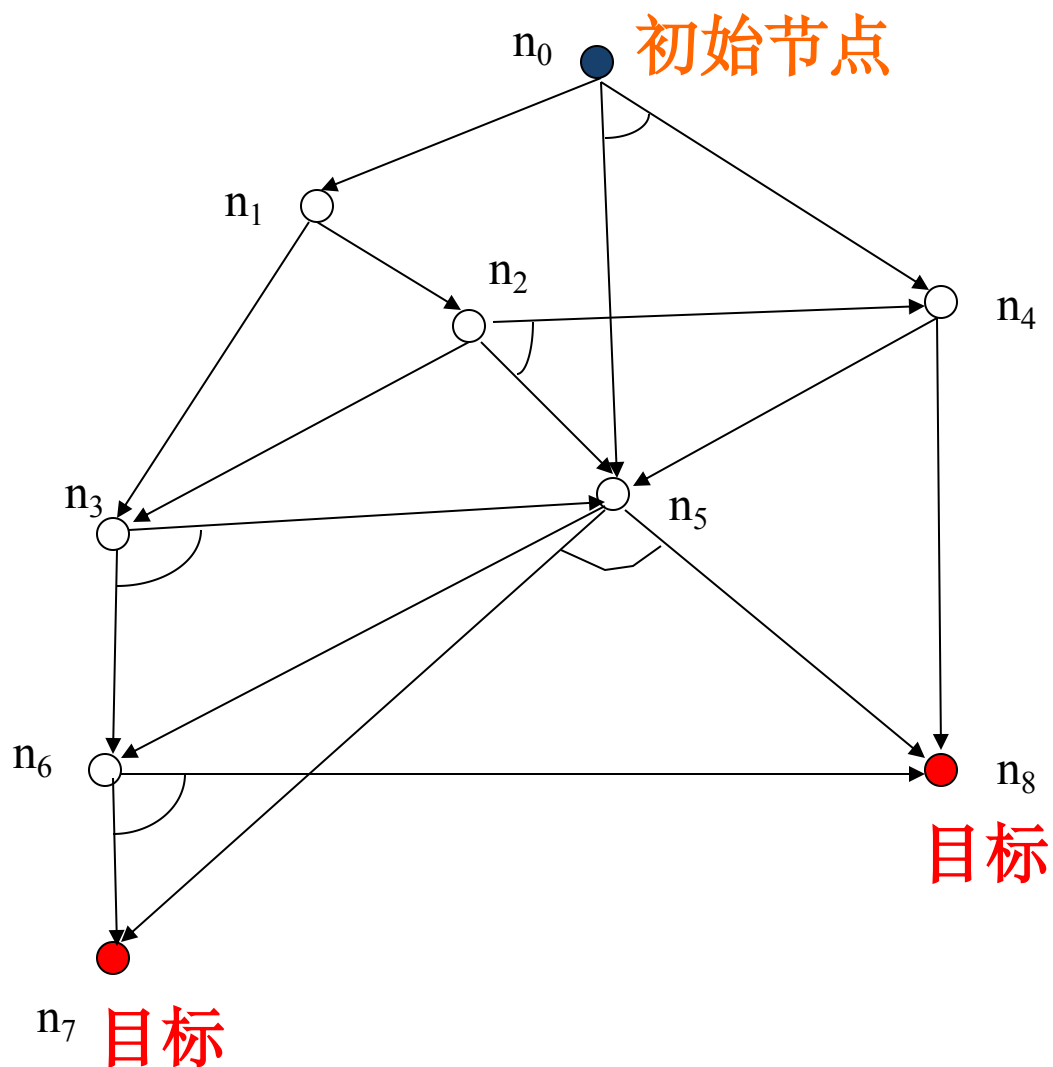
$$h(n_5)=1$$

$$h(n_6)=2$$

$$h(n_7)=0$$

$$h(n_8)=0$$

【与或树的启发式搜索举例】



已知其中:

$$h(n_1)=2$$

$$h(n_2)=4$$

$$h(n_3)=4$$

$$h(n_4)=1$$

$$h(n_5)=1$$

$$h(n_6)=2$$

$$h(n_7)=0$$

$$h(n_8)=0$$

已知:

$$h(n_1)=2$$

$$h(n_2)=4$$

$$h(n_3)=4$$

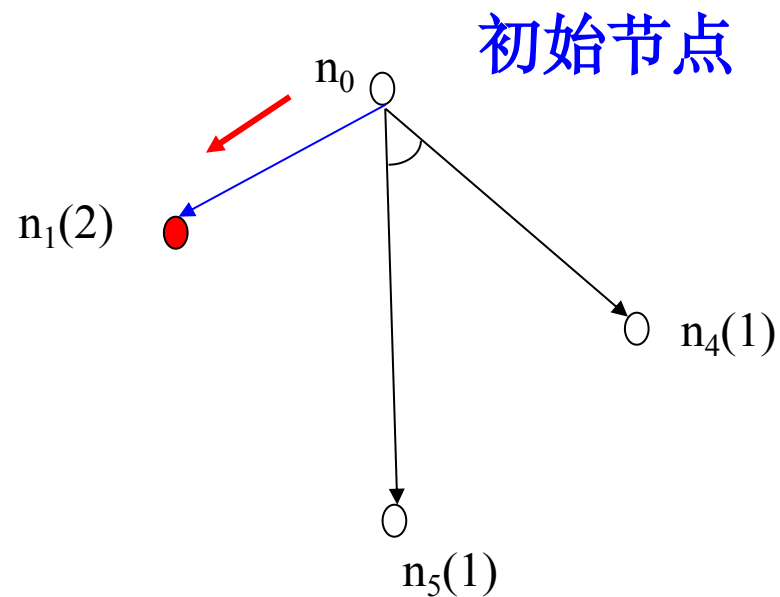
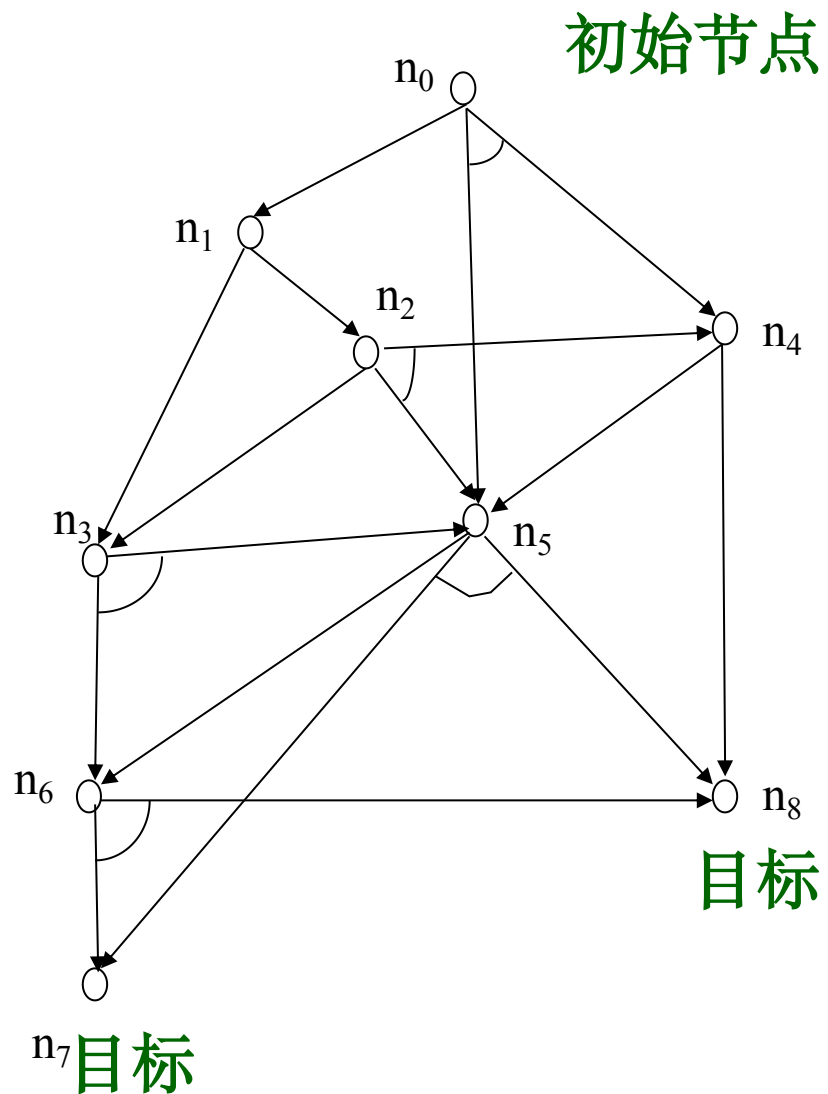
$$h(n_4)=1$$

$$h(n_5)=1$$

$$h(n_6)=2$$

$$h(n_7)=0$$

$$h(n_8)=0$$



已知:

$$h(n_1)=2$$

$$h(n_2)=4$$

$$h(n_3)=4$$

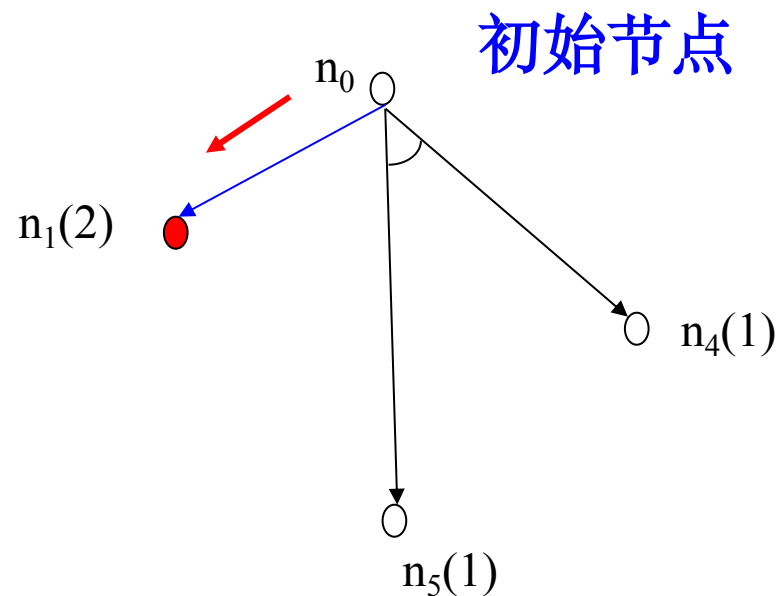
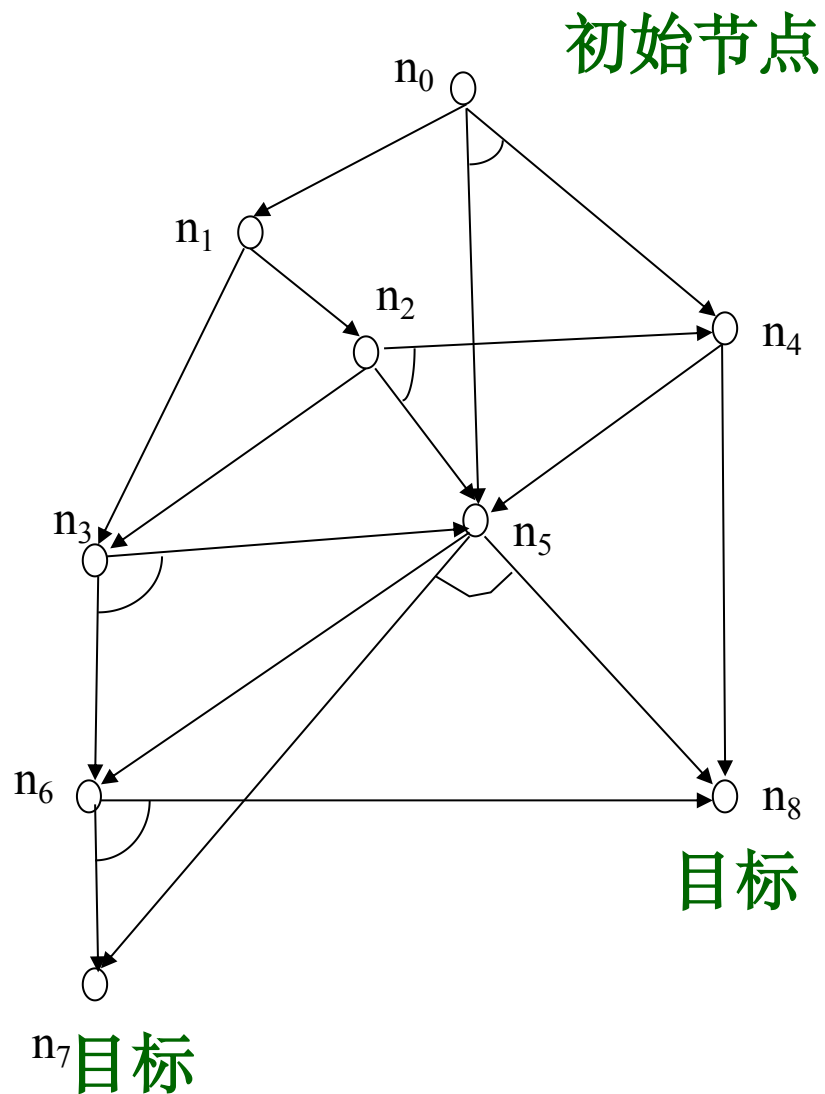
$$h(n_4)=1$$

$$h(n_5)=1$$

$$h(n_6)=2$$

$$h(n_7)=0$$

$$h(n_8)=0$$



黑色: $h(n_0)=(1+1)+(1+1)=4$

蓝色: $h(n_0)=1+2=3$

已知:

$$h(n_1)=2$$

$$h(n_2)=4$$

$$h(n_3)=4$$

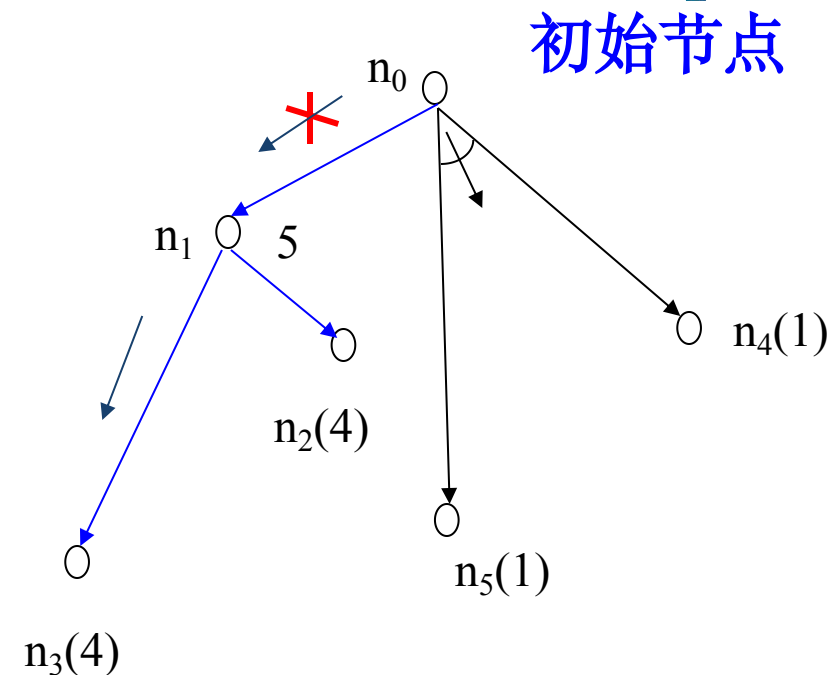
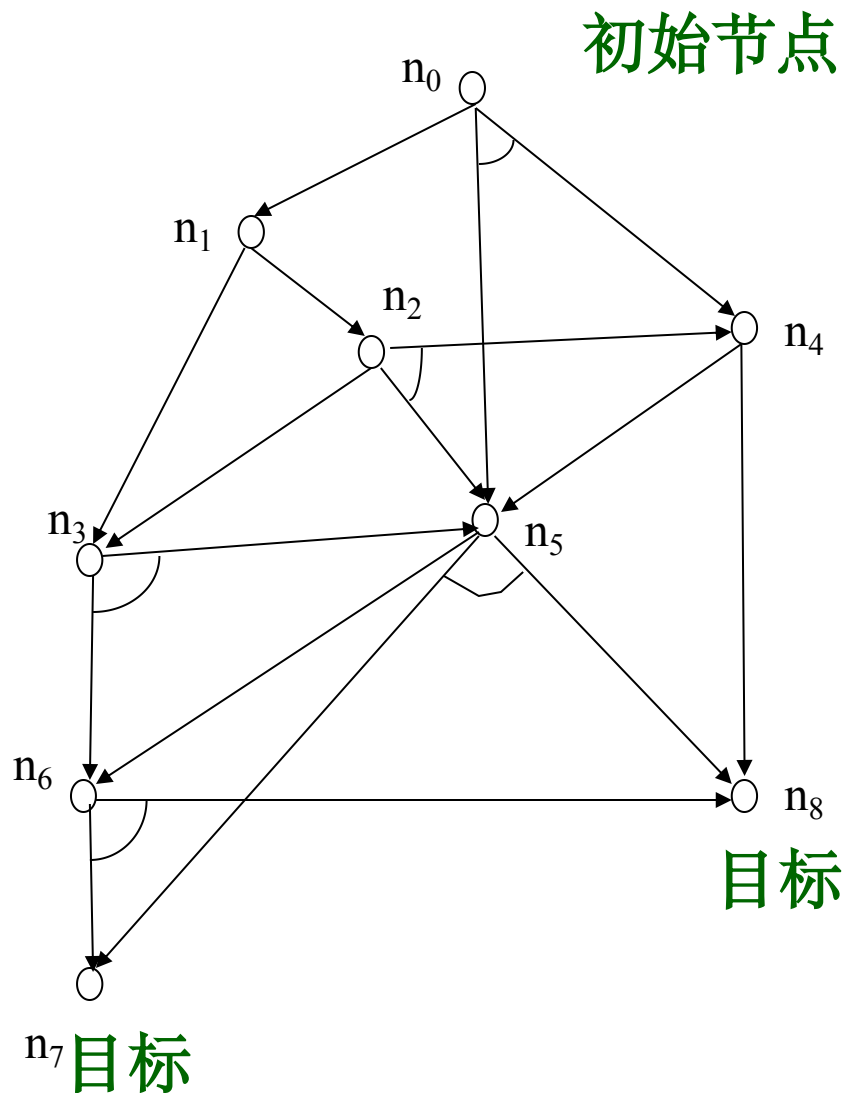
$$h(n_4)=1$$

$$h(n_5)=1$$

$$h(n_6)=2$$

$$h(n_7)=0$$

$$h(n_8)=0$$



已知:

$$h(n_1)=2$$

$$h(n_2)=4$$

$$h(n_3)=4$$

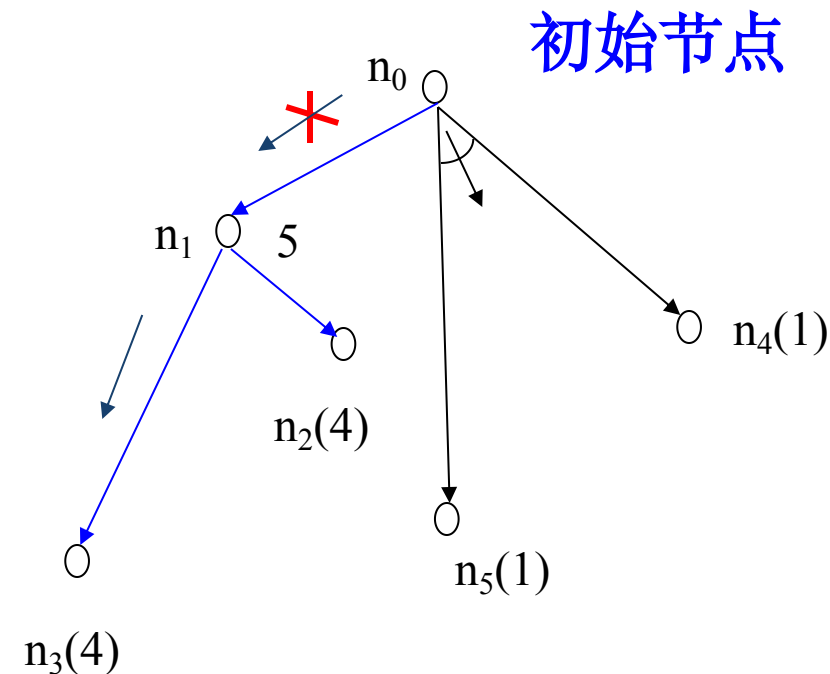
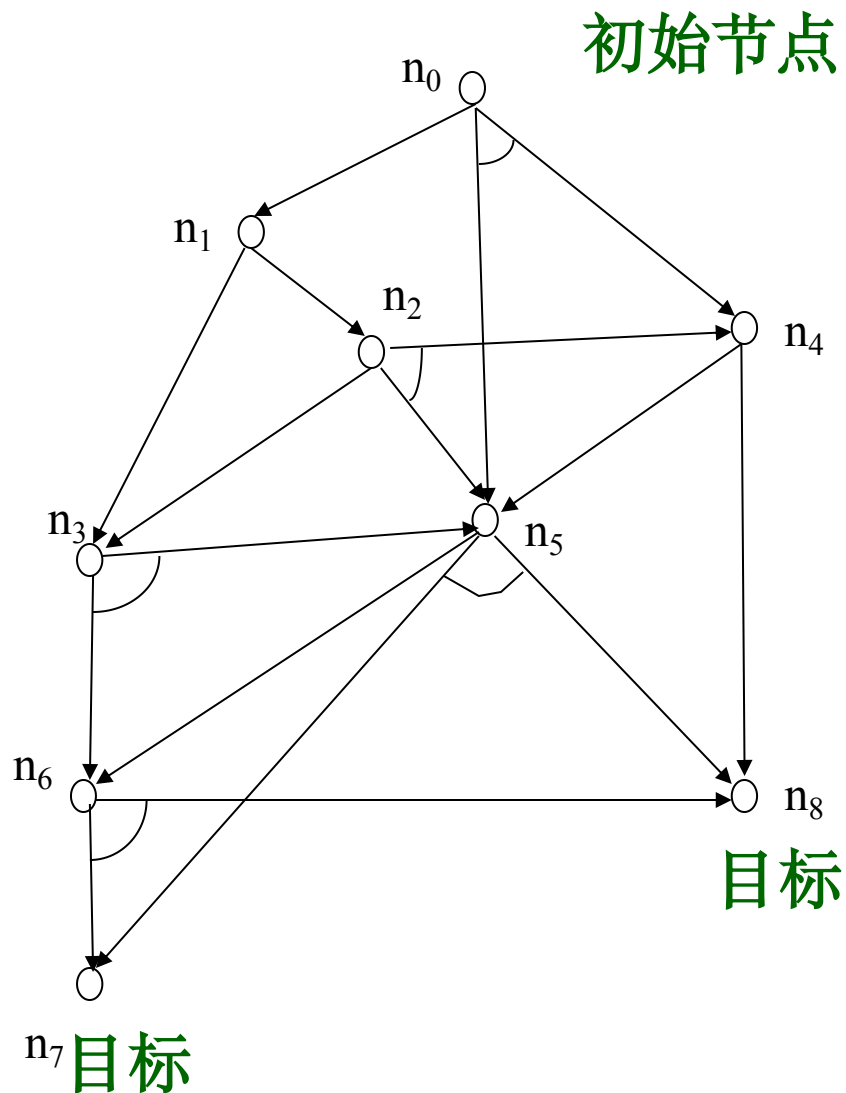
$$h(n_4)=1$$

$$h(n_5)=1$$

$$h(n_6)=2$$

$$h(n_7)=0$$

$$h(n_8)=0$$



黑色: $h(n_0)=2+2=4$

蓝色: $h(n_0)=1+5=6$

已知:

$$h(n_1)=2$$

$$h(n_2)=4$$

$$h(n_3)=4$$

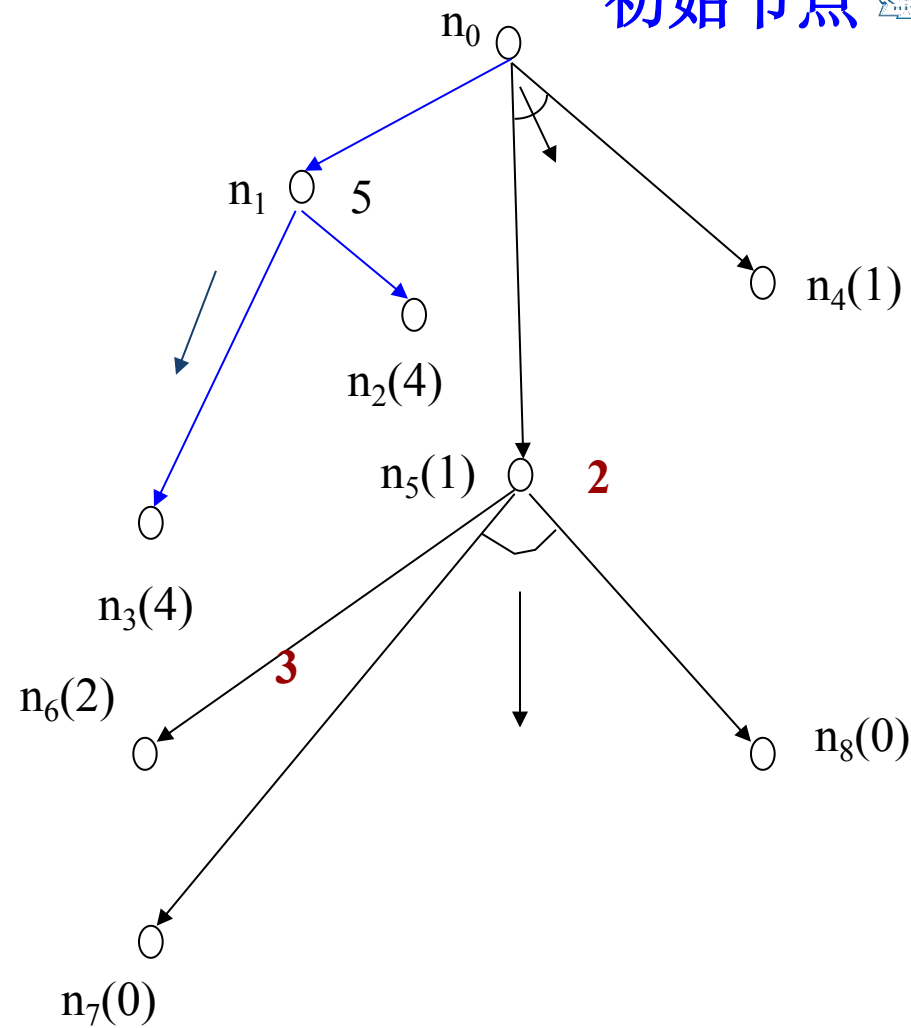
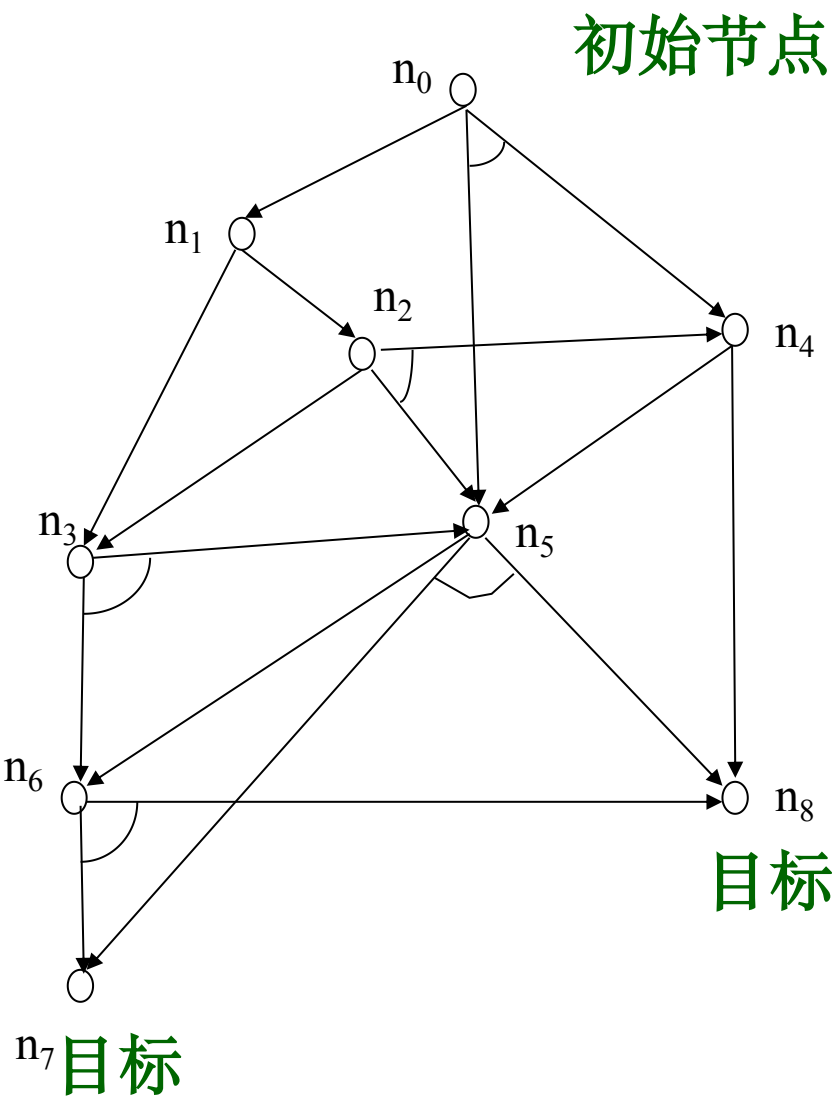
$$h(n_4)=1$$

$$h(n_5)=1$$

$$h(n_6)=2$$

$$h(n_7)=0$$

$$h(n_8)=0$$



初始节点

已知:

$$h(n_1)=2$$

$$h(n_2)=4$$

$$h(n_3)=4$$

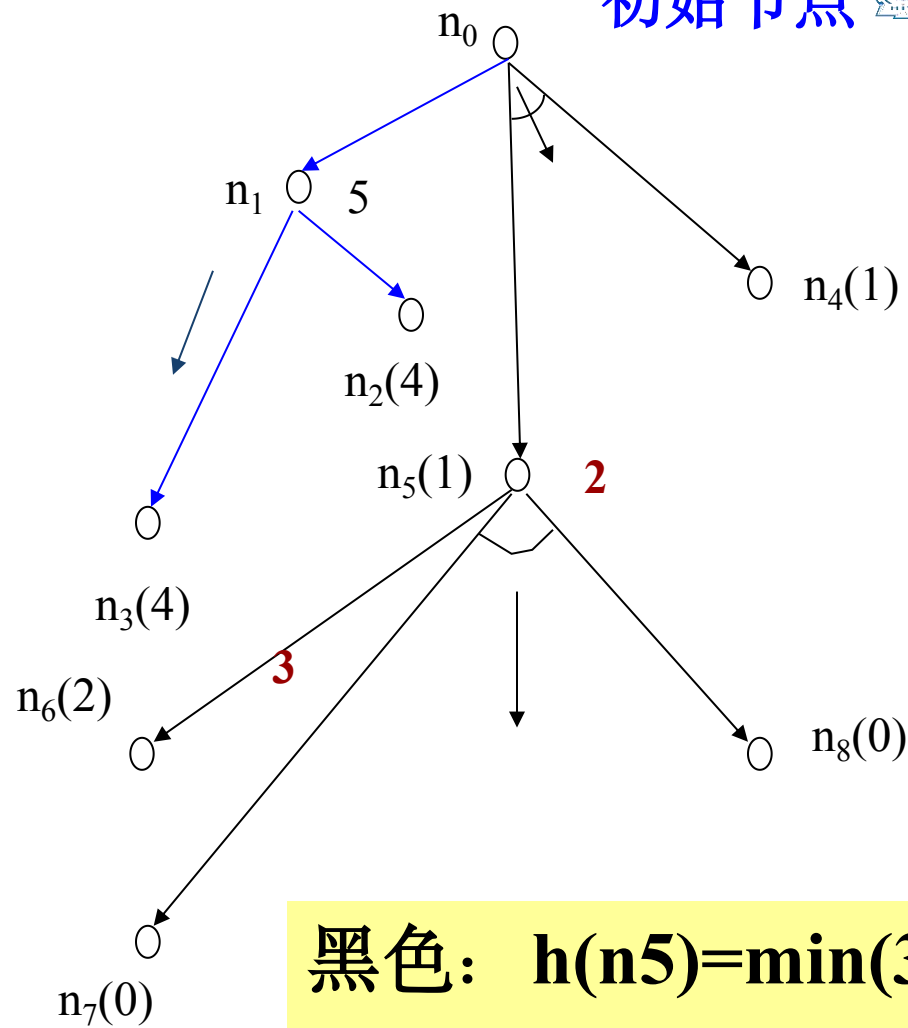
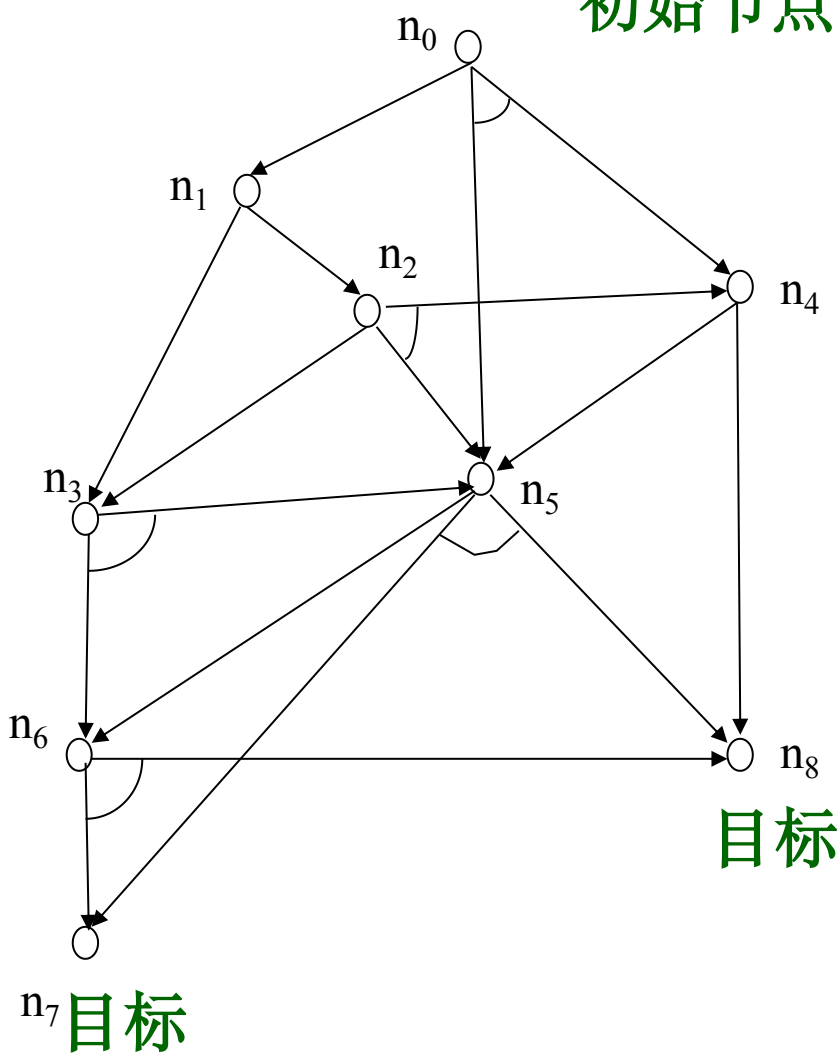
$$h(n_4)=1$$

$$h(n_5)=1$$

$$h(n_6)=2$$

$$h(n_7)=0$$

$$h(n_8)=0$$



黑色: $h(n_5)=\min(3,2)=2$

$h(n_0)=[1+h(n_5)]+[1+h(n_4)]=5$

蓝色: $1+5=6$

已知:

$$h(n_1)=2$$

$$h(n_2)=4$$

$$h(n_3)=4$$

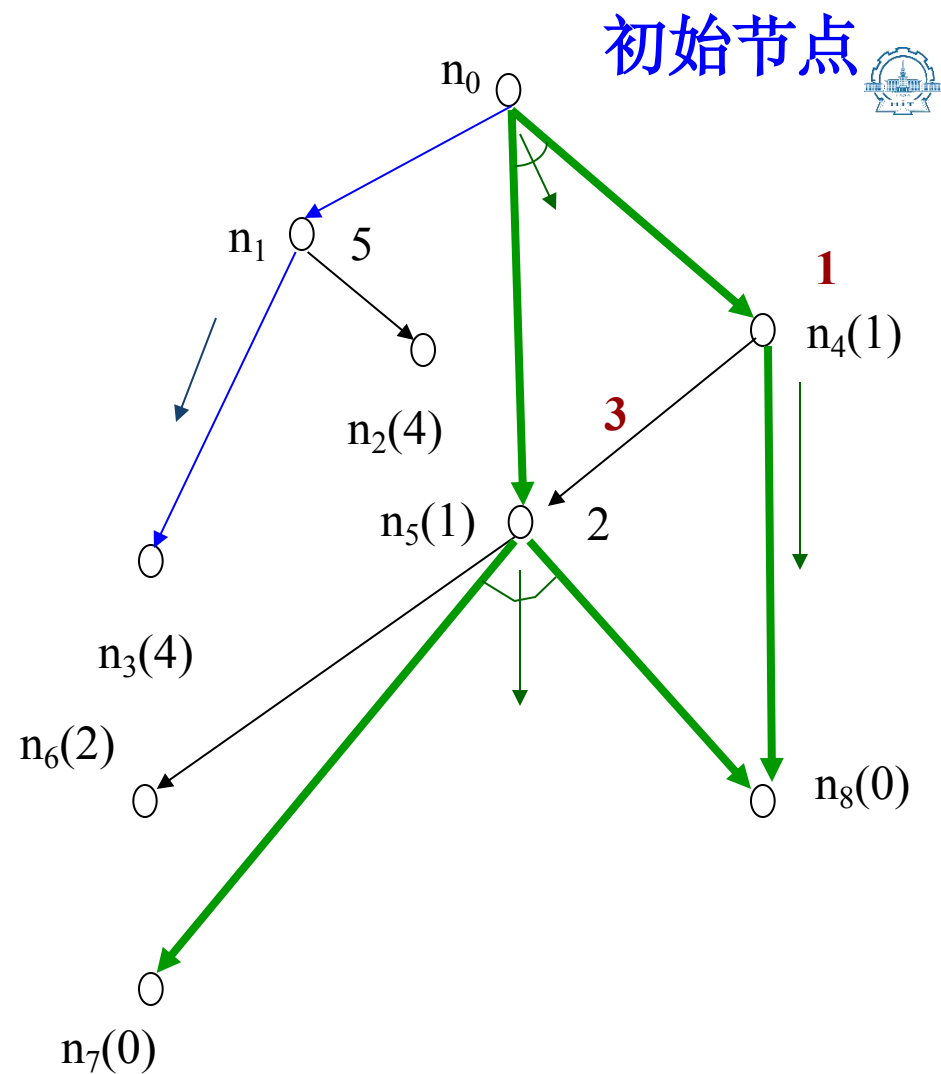
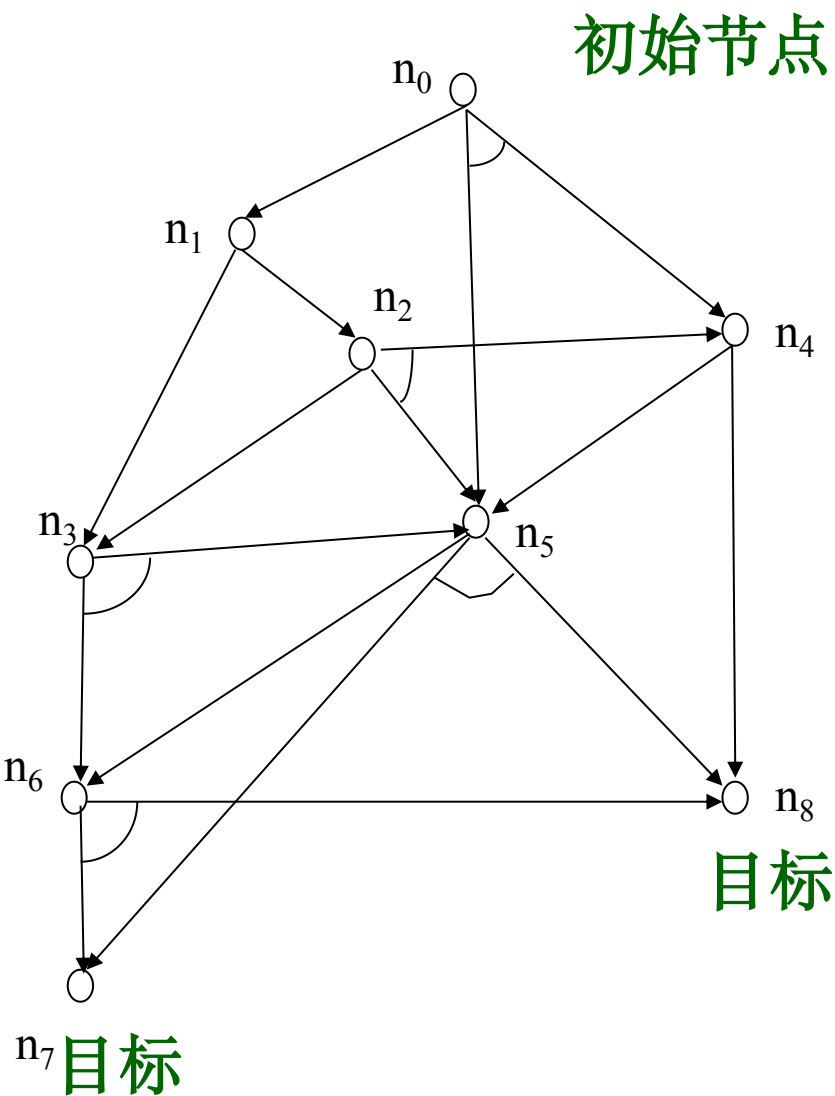
$$h(n_4)=1$$

$$h(n_5)=1$$

$$h(n_6)=2$$

$$h(n_7)=0$$

$$h(n_8)=0$$





初始节点

初始节点

已知:

$$h(n_1)=2$$

$$h(n_2)=4$$

$$h(n_3)=4$$

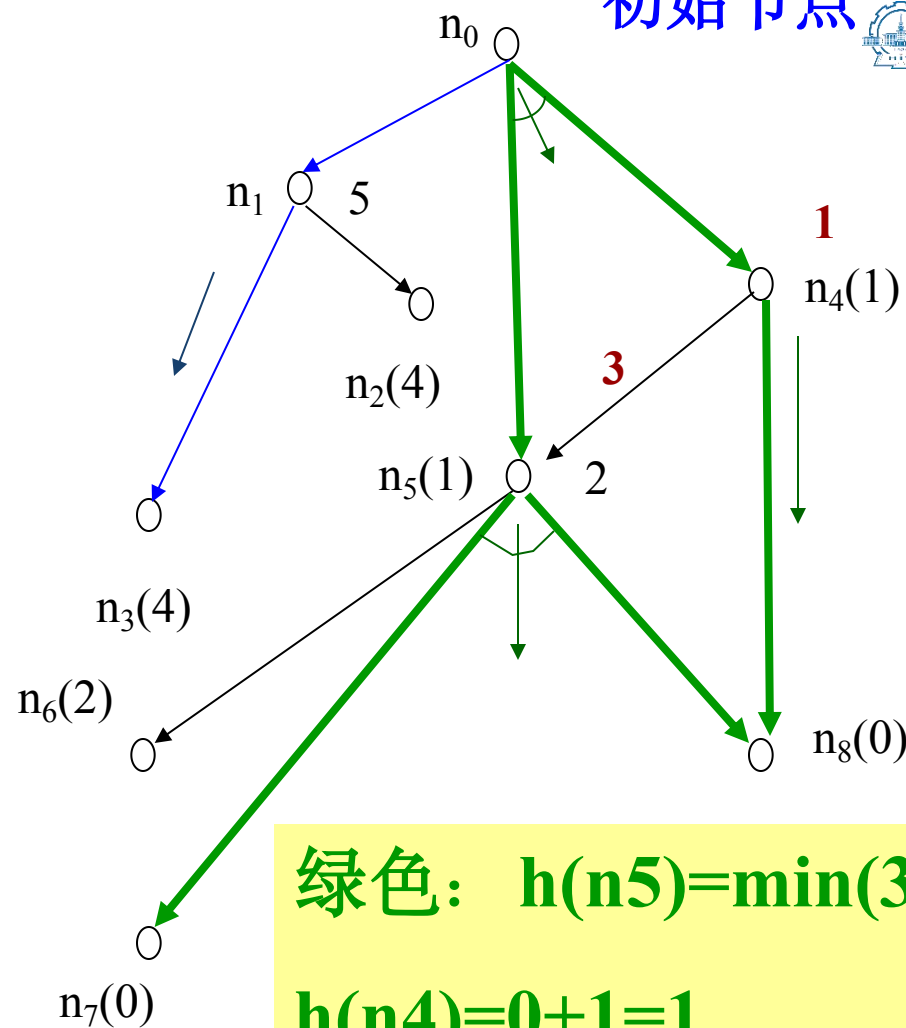
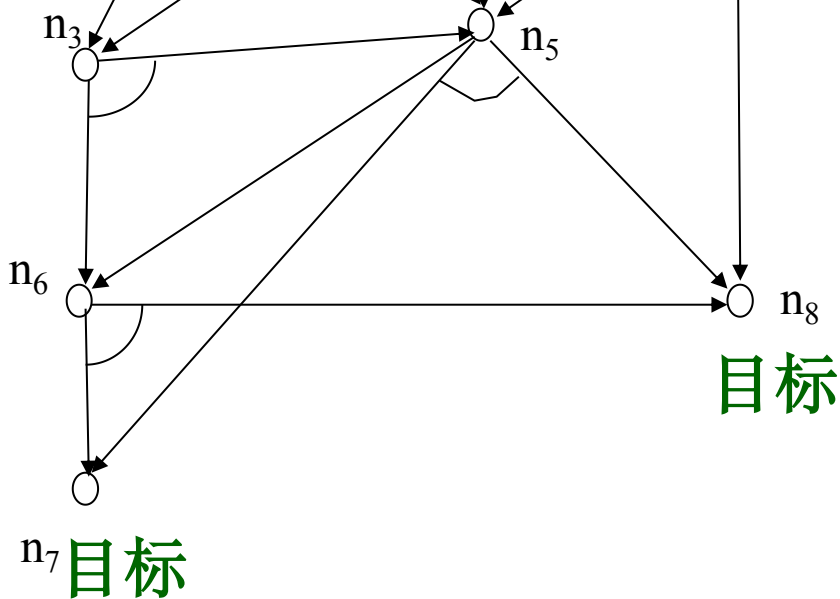
$$h(n_4)=1$$

$$h(n_5)=1$$

$$h(n_6)=2$$

$$h(n_7)=0$$

$$h(n_8)=0$$



绿色: $h(n_5)=\min(3,2)=2$

$h(n_4)=0+1=1$

$h(n_0)=[1+h(n_5)]+(1+h(n_4))=5$

蓝色: $1+5=6$

目录

01

4.1 概述

02

4.2 状态空间盲目搜索

03

4.3 状态空间启发式搜索

04

4.4 与/或树搜索

05

4.5 博弈树的启发式搜索

4.5.1概述

❖ 博弈的概念

博弈是一类具有智能行为的竞争活动，如下棋、战争等。

❖ 博弈的类型

双人完备信息博弈：两位选手（例如**MAX**和**MIN**）对垒，轮流走步，每一方不仅知道对方已经走过的棋步，而且还能估计出对方未来的走步。

机遇性博弈：存在不可预测性的博弈，例如掷币等。

❖ 博弈树

若把双人完备信息博弈过程用图表示出来，就得到一棵与/或树，这种与/或树被称为博弈树。在博弈树中，那些下一步该**MAX**走步的节点称为**MAX**节点，下一步该**MIN**走步的节点称为**MIN**节点。

❖ 博弈树的特点

- (1) 博弈的初始状态是**初始节点**；
- (2) 博弈树中的“或”节点和“与”节点是**逐层交替**出现的；
- (3) 整个博弈过程始终站在某一方的立场上，例如**MAX**方。所有能使自己一方获胜的终局都是**本原问题**，相应的节点是**可解节点**；所有使对方获胜的终局都是**不可解节点**。

4.5.2 极大极小过程

对简单的博弈问题，可生成整个博弈树，找到**必胜的策略**。

对于复杂的博弈问题，不可能生成整个搜索树，如国际象棋，大约有 10^{120} 个节点。一种可行的方法是用当前正在考察的节点生成一棵**部分博弈树**，并利用**估价函数 $f(n)$** 对叶节点进行静态估值。

对叶节点的估值方法，那些对**MAX**有利的节点，其估价函数取**正值**；那些对**MIN**有利的节点，其估价函数取**负值**；那些使双方**均等**的节点，其估价函数取**接近于0**的值。

为非叶节点的值，必须从叶节点开始向上倒退。其倒退方法是：

对于**MAX**节点，由于**MAX**方总是选择估值最大的走步，因此，**MAX**节点的倒推值应该取其后继节点估值的最大值。

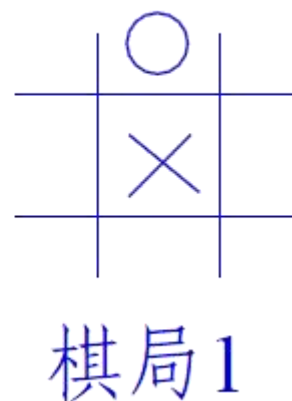
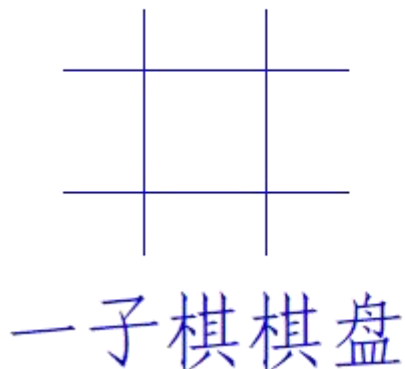
对于**MIN**节点，由于**MIN**方总是选择使估值最小的走步，因此，**MIN**节点的倒推值应取其后继节点估值的最小值。

这样一步一步的计算倒推值，直至求出初始节点的倒推值为止。这一过程称为**极大极小过程**。

4.5.2 极大极小过程

例：一子棋游戏

设有一个三行三列的棋盘，如下图所示，两个棋手轮流走步，每个棋手走步时往空格上摆一个自己的棋子，谁先使自己的棋子成三子一线为赢。设**MAX**方的棋子用×标记，**MIN**方的棋子用○标记，并规定**MAX**方先走步。



$$e(P)=e(+P)-e(-P)$$

解：估价函数 **$e(+P)$** ：**P**上有可能使×成三子为一线的数目；

$e(-P)$ ：**P**上有可能使○成三子为一线的数目；

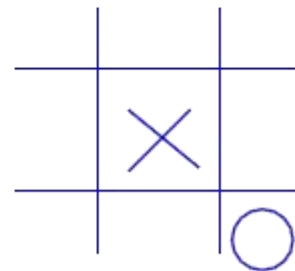
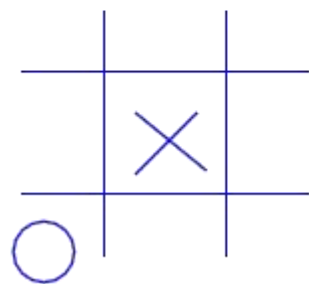
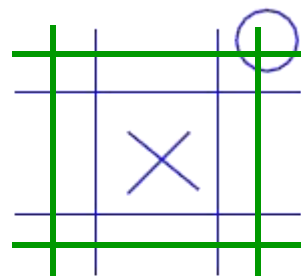
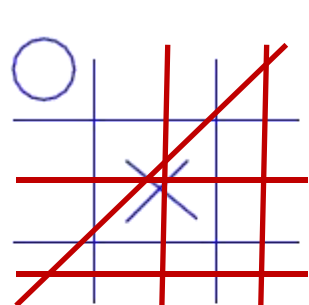
当**MAX** 必胜 **$e(P)$** 为正无穷大；

MIN 必胜 **$e(P)$** 为负无穷大。

4.5.2 极大极小过程

棋局即估价函数

具有对称性的棋盘可认为是同一棋盘。如下图所示:



× 为**MAX**方的棋子

○ 为**MIN**方的棋子

-使× 成三子一线的可能情况

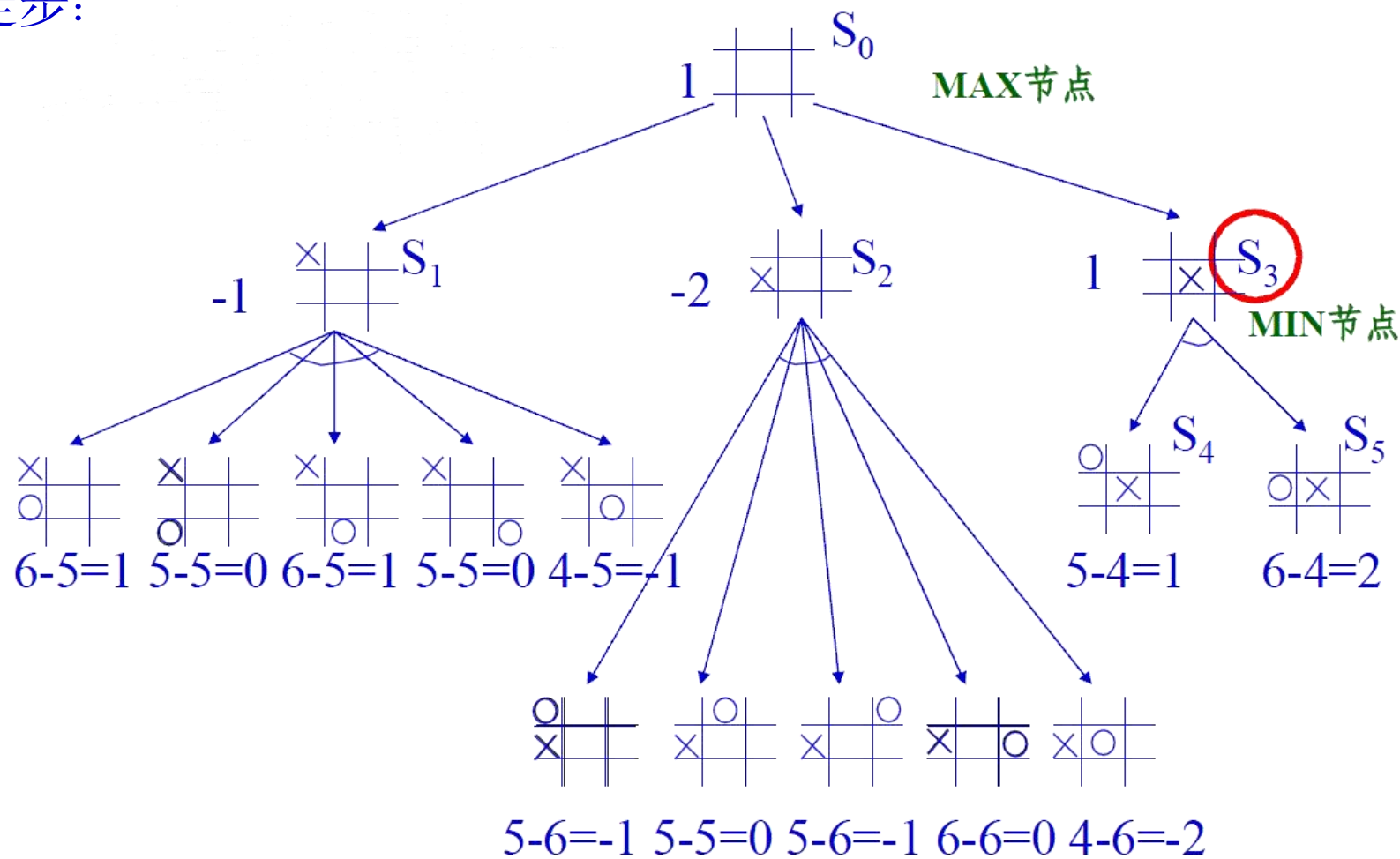
-使○ 成三子一线的可能情况

$$\text{其 } e(P) = e(+P) - e(-P) = 5 - 4 = 1$$

4.5.2 极大极小过程

一子棋的极大极小搜索

设MAX方先走步:



一子棋的极大极小搜索

4.5.3 α - β 剪枝

❖ 剪枝的概念

在极大极小搜索过程中，剪去一些没用的分枝，减少了搜索空间，使得算法在同样时间内可以搜索更深层，这种技术被称为 α - β 剪枝。

❖ 剪枝方法

(1) **MAX**节点（或节点）的 α 值为当前子节点的最大倒推值；

(2) **MIN**节点（与节点）的 β 值为当前子节点的最小倒推值；

(3) α - β 剪枝的规则如下：

β 剪枝

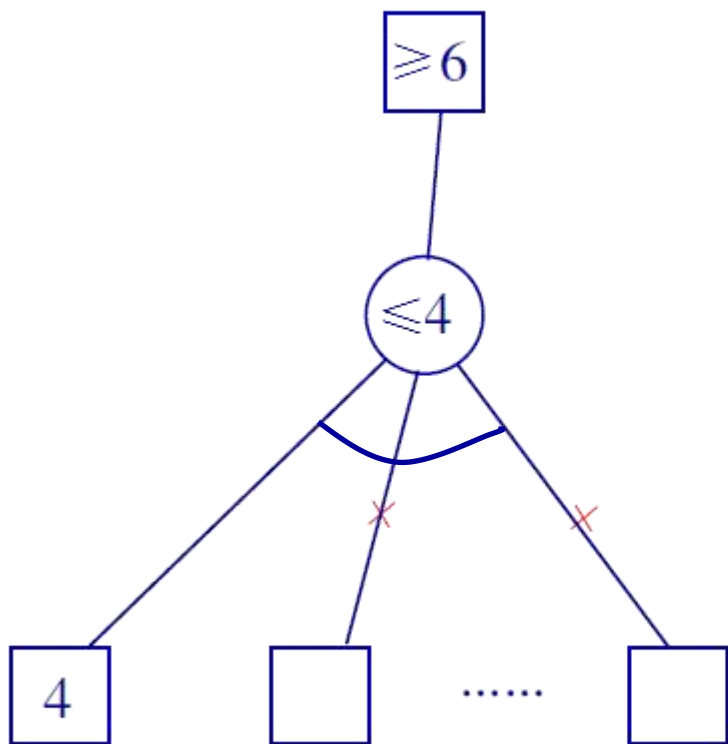
任何**MAX**节点 n 的 α 值大于或等于它先辈节点的 β 值，则 n 以下的分枝可停止搜索，并令节点 n 的倒推值为 α 。这种剪枝称为 β 剪枝。

α 剪枝

任何**MIN**节点 n 的 β 值小于或等于它先辈节点的 α 值，则 n 以下的分枝可停止搜索，并令节点 n 的倒推值为 β 。这种剪枝称为 α 剪枝。

4.5.3 α - β 剪枝

α 剪枝



β 剪枝

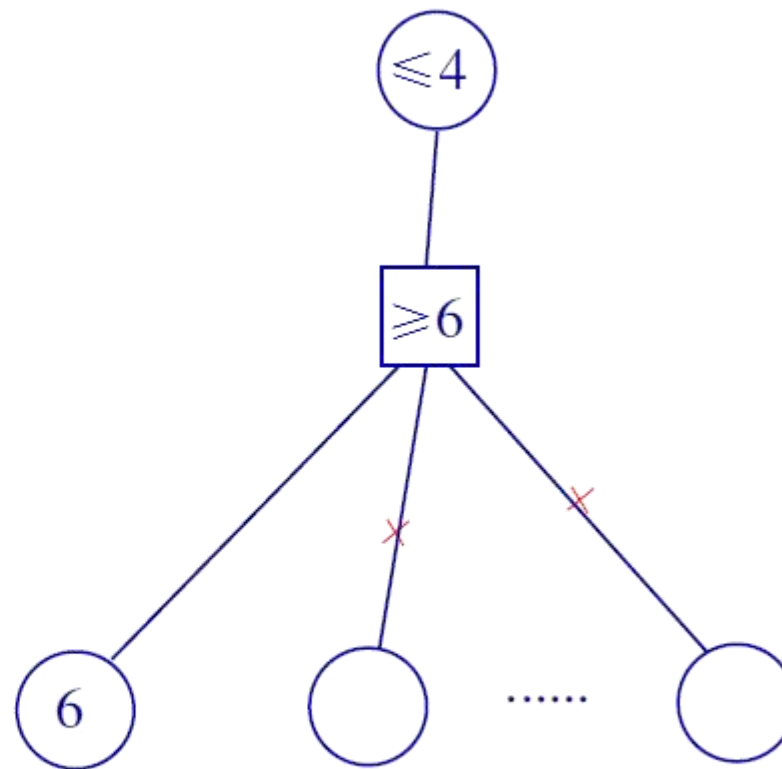


Figure: $\alpha - \beta$ 剪枝

主观题

一个 α - β 剪枝的具体例子，如下图所示。其中最下面一层端节点旁边的数字是假设的估值。请确定哪些节点和边可以被剪掉

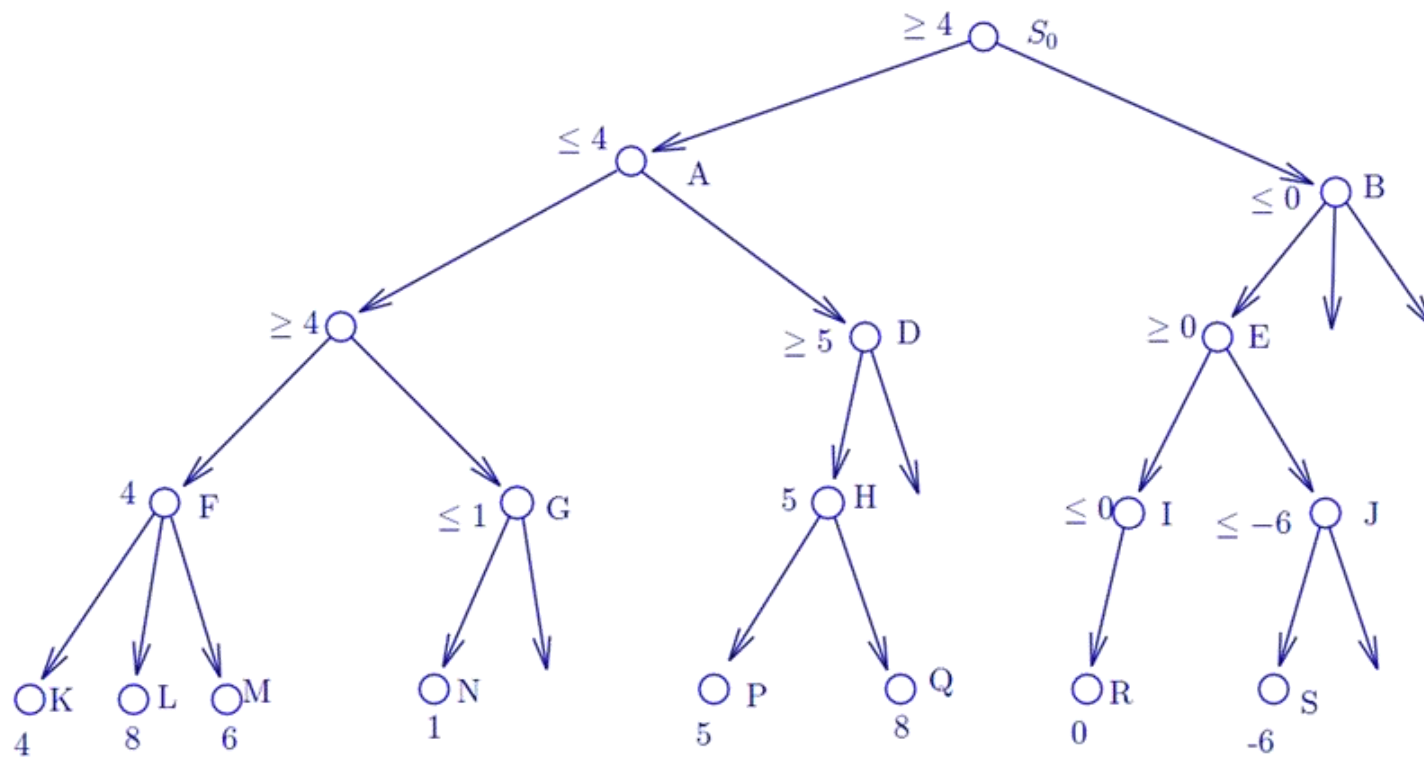


Figure: $\alpha \sim \beta$ 剪枝

4.5.3 α - β 剪枝

一个 α - β 剪枝的具体例子，如下图所示。其中最下面一层端节点旁边的数字是假设的估值。

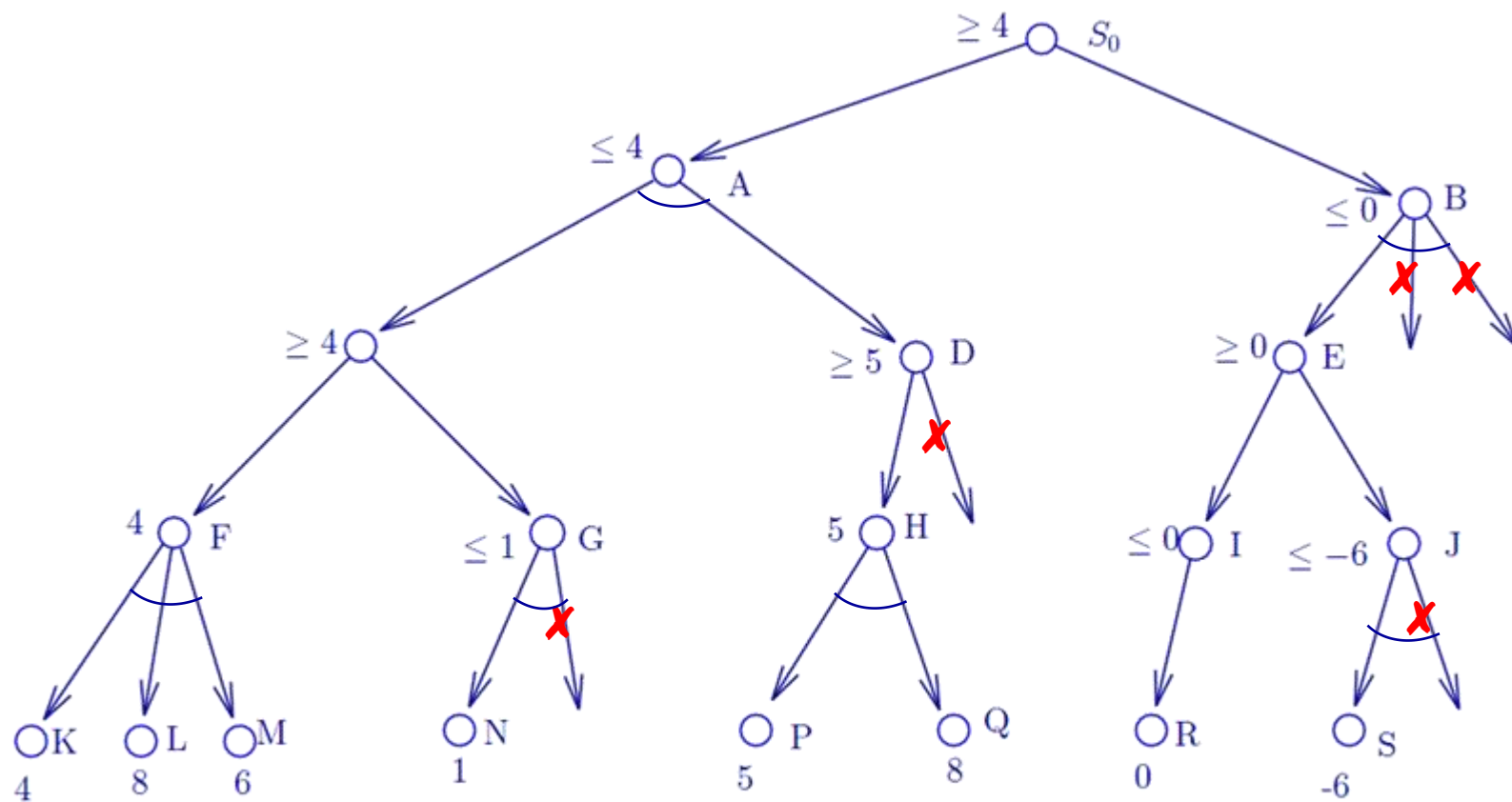


Figure: $\alpha \sim \beta$ 剪枝